Forecasting Stock Returns Using Machine Learning

A Comparative Analysis of Different Methods

Master's Thesis
M.Sc. Finance and Investments
Copenhagen Business School

Peter Mistol (120194)
Stefan Bernd Möhler (149397)

Supervisor: Kasper Lund-Jensen
Date: May 15, 2023

Number of Pages: 97
Characters (with Spaces): 217,273

Abstract

This thesis investigates the potential of machine learning models to predict stock returns in the U.S. market. The main objective of the study is to identify the model with the best prediction performance by comparing a set of candidate models. Furthermore, we identify the firm characteristics and macroeconomic variables that influence the performance of the models and evaluate whether machine learning-based trading strategies outperform a linear benchmark model. The study relies on a large panel analysis of more than 32,000 U.S. stocks over 65 years from 1957 to 2021. We find that neural networks are the most effective models for predicting stock returns, while combining the predictions of all methods in the study into an ensemble can lead to even better performance. The most important predictor variables are those related to price trends, liquidity measures, and investment and growth variables. Finally, we show that forming long-short trading strategies based on the predictions of machine learning methods can lead to significant economic benefits for investors. Although machine learning models can provide promising results in asset pricing, they come with risks and challenges. In particular, their limited interpretability can pose a challenge for practical applications in portfolio management. Also, they require more trading than simpler models, which reduces their performance when transaction costs are taken into account. In summary, our results suggest that the use of machine learning in asset pricing is a beneficial development, but it is critical to fully understand the underlying methods, including the associated risks and limitations.

Acknowledgements

We would like to thank our supervisor, Kasper Lund-Jensen, for his guidance, support, and expertise throughout the duration of our master's thesis. His insightful feedback and encouragement have been instrumental in shaping our research and enhancing the quality of our work.

We would also like to extend our thanks to UCloud, and particularly Kristoffer Gulmark Poulsen, for providing us with access to their high-performance computers. Their infrastructure played a crucial role in facilitating the computational aspects of our research, enabling us to analyze and process large datasets efficiently.

Contents

List of Tables **5**

List of Figures **5**

List of Abbreviations **6**

1. Introduction **8**

2. Literature Review **11**
2.1. Market Efficiency and Stock Return Predictability 12
2.2. Exploring Return Prediction Methods: A Review 13
2.3. Machine Learning in Asset Pricing 18

3. Data **22**
3.1. Data Sources and Preparation 22
3.2. Descriptive Statistics 25
3.3. Construction of Final Dataset 28

4. Methodology **29**
4.1. Overall Approach 29
4.2. Optimization of Machine Learning Models 31
4.3. Sample Splitting 35
4.4. Models 38
4.4.1. Multiple Linear Regression Model 38
4.4.2. Penalized Linear Model: Elastic Net 41
4.4.3. Dimension Reduction: PCR and PLS 42
4.4.4. Tree-Based Methods: Random Forests and XGBoost 45
4.4.5. Neural Networks 55
4.5. Comparative Performance Evaluation 61
4.5.1. Predictive Out-Of-Sample R^2 62
4.5.2. Diebold-Mariano Test 62
4.6. Variable Importance: SHAP Values 63
4.7. Investment Strategy 67
4.7.1. Backtesting Approach 67
4.7.2. Performance Metrics and Factor Analysis 68

5. Empirical Results **70**
5.1. Performance of Model Forecasts 70
5.2. Variable Importance 74
5.3. Performance of Machine Learning Portfolios 78

6. Discussion **88**
6.1. Critical Discussion of Findings 89
6.2. Implications of Findings 92
6.3. Further Research 94

7. Conclusion **96**

Appendix **98**
A. Detailed Description and Descriptive Statistics of Firm Characteristics . 98
B. Gauss-Markov Assumptions 102
C. Robustness Checks 103

References **105**

List of Tables

1. Firm Characteristics 23
2. SIC Divisions 24
3. Descriptive Statistics of Macroeconomic Factors 27
4. Monthly Out-Of-Sample Predictive R^2 Performance 71
5. Diebold-Mariano Test Results - Comparison of Predictive Performance . . . 72
6. Long-Short Portfolio Performance 79
7. Factor Analysis of Machine Learning Portfolios 85
8. Annualized Sharpe Ratios of the Best-performing Machine Learning Models and Common Factor Long-Short Strategies 87
9. Description and Descriptive Statistics of Firm Characteristics 99
10. Long-Short Portfolio Performance (Excluding Micro-cap Stocks) 103
11. Factor Analysis of Machine Learning Portfolios (Excluding Micro-cap Stocks) 104

List of Figures

1. Monthly Number of Firms over Time 26
2. Macroeconomic Factors over Time 27
3. Illustration of Bayesian Optimization 35
4. Sample Splitting 37
5. Structure of a Tree-Based Model 46
6. Splitting Process and Partitions of CART 47
7. Random Forest Model 51
8. Architecture of a Single Hidden Layer Neural Network 56
9. Illustration of SHAP Values 65
10. Variable Importance (SHAP) for Firm Characteristics (Top 20) by Model . . 75
11. Variable Importance (SHAP) for Macroeconomic Factors 77
12. Cumulative Log Returns of Machine Learning Portfolios 81
13. Cumulative Log Returns of Ensemble Strategy 83
14. Cumulative Log Returns of Machine Learning Portfolios in Comparison with Long-Short Strategies Based on Common Factor Portfolios 88

List of Abbreviations

AMEX	American Stock Exchange
AR	Autoregressive
ARCH	Autoregressive Conditional Heteroskedasticity
ARIMA	Autoregressive Integrated Moving Average
ARMA	Autoregressive Moving Average
bm	Book-to-Market Ratio
CAPM	Capital Asset Pricing Model
CMA	Conservative Minus Aggressive
CRSP	Center for Research in Security Prices
dfy	Default Yield Spread
dp	Dividend-Price Ratio
ep	Earnings-Price Ratio
FRED	Federal Reserve Economic Data
GBRT	Gradient Boosted Regression Trees
HML	High Minus Low
MA	Moving Average
MSE	Mean Squared Error
NASDAQ	National Association of Securities Dealers Automated Quotations Stock Market
ntis	Net Equity Expansion
NYSE	New York Stock Exchange
OLS	Ordinary Least Squares
OOS	Out-Of-Sample
PCA	Principal Component Analysis
PCR	Principal Component Regression
PLS	Partial Least Squares
R&D	Research and Development
ReLU	Rectified Linear Unit
RMW	Robust Minus Weak
S&P 500	Standard & Poor's 500
SGD	Stochastic Gradient Descent
SHAP	SHapley Additive exPlanations
SIC	Standard Industrial Classification

SMB	Small Minus Big
svar	Stock Variance
tbl	Treasury Bill
tms	Term Spread
VASA	Variable Subsample Aggregation
WRDS	Wharton Research Data Services

1. Introduction

Machine learning is a transformative technology with the potential to revolutionize many industries, from safely navigating autonomous vehicles to improving diagnosis and treatment in healthcare. At its core, machine learning refers to a subfield of artificial intelligence in which computer algorithms are trained to learn from data and make predictions or decisions without being explicitly programmed (Dey, 2016). In recent years, machine learning has evolved immensely thanks to advances in computing power and the increase in available data. For instance, computing power has doubled approximately every two years since the 1970s, while the amount of data generated has increased exponentially (James et al., 2021). This development has made machine learning an increasingly valuable tool for analyzing complex datasets.

In finance, machine learning is used in different areas, including derivative pricing, credit risk assessment, and fraud detection (Weigand, 2019). One of the most promising applications, however, is in asset pricing. Asset pricing models aim to explain the relationship between the expected returns of financial assets and their underlying risk characteristics. One of the key challenges in asset pricing is accurately predicting the returns of financial assets (Gu et al., 2020). Predicting returns is particularly difficult for stocks, as their returns are influenced by a variety of factors, including macroeconomic conditions, company-specific events, and changes in investor sentiment. Furthermore, common methods such as regression analysis and portfolio sorts have limitations when dealing with a large number of predictor variables and nonlinearity in the data, which are common issues in stock return prediction (Gu et al., 2020). Machine learning methods have the potential to overcome these challenges by analyzing a wide range of predictor variables simultaneously and accounting for nonlinear patterns in the data. As such, machine learning has emerged as a promising tool for predicting stock returns (James et al., 2021).

The literature on forecasting stock returns is extensive. For decades, researchers and practitioners have debated whether stock returns are predictable. The efficient market hypothesis introduced by Fama (1970) assumes that all available information is incorporated into asset prices, making it difficult to generate excess returns by predicting stock returns. While various studies demonstrate some degree of predictability in equity markets, this does not necessarily contradict the efficient market hypothesis. Rather, a modern interpretation of the hypothesis allows for the equity premium to vary countercyclically in a general

equilibrium model (Campbell & Cochrane, 1999). This is advantageous because even a small predictive component in returns can provide enormous economic benefits to investors (Campbell & Thompson, 2008). While the success of traditional time-series models in predicting stock returns is limited, several studies provide evidence of variables that have predictive power for the stock market in regression-based approaches. These variables include valuation ratios, price- and liquidity trends, and macroeconomic signals (Campbell, 2000). Welch and Goyal (2008) question the ability of these variables to predict the equity premium out-of-sample, as they can only find in-sample predictability when testing a wide range of well-known predictors. In contrast, Campbell and Thompson (2008) and Rapach and Zhou (2013) show that by adopting a slightly modified approach, the corresponding variables perform well in out-of-sample predictions and offer economic benefits to investors.

More recent research is investigating the benefits of various machine learning methods in asset pricing. The list of studied methods ranges from dimension reduction techniques to tree-based methods and neural networks. To gain a better understanding of which of these methods are most useful, Gu et al. (2020) have conducted a comparative study of different models for predicting U.S. stock returns. They conclude that tree-based methods and neural networks are the most effective methods and significantly outperform a multiple linear benchmark model. Similar studies were conducted for the European market by Drobetz and Otto (2021) and the Chinese market by Leippold et al. (2022), providing mostly consistent results.

Although previous research has shown promising results for tree-based methods and neural networks, it is not yet clear whether these results are robust and easily reproducible with a more recent dataset. Moreover, there are not many studies that have investigated the practical implications of the predictions generated by different machine learning methods. Motivated by this, the objective of our study is to investigate the predictive power of machine learning methods for stock returns. Specifically, we seek to provide solutions to the following research questions:

(1) Which machine learning models best predict stock returns out-of-sample?

(2) What are the firm characteristics and macroeconomic variables that drive the models' performance?

(3) Can long-short investment strategies based on machine learning outperform a strategy based on a linear benchmark model, and can common

risk factors explain their performance?

To answer the research questions we perform a grand-panel analysis where we examine more than 32,000 U.S. stocks over 65 years from 1957 to 2021. The motivation for this time horizon comes from the fact that we want to use as much data as available for the training of the models and most of our input variables were not available before 1957. For our analysis, we follow the empirical design of Gu et al. (2020) and compare different models that are well-suited candidates for the problem of excess return prediction. The models have their roots in both traditional statistics and the machine learning literature. The set of methods in our comparison includes linear regression, elastic net, dimension reduction via principal component regression (PCR) and partial least squares (PLS), tree-based models including random forests and XGBoost, and neural networks. Our study uses a notable amount of 479 predictor variables as signals to implement the models. They consist of 94 firm characteristics for each stock, 10 industry dummy variables, and interactions of each firm characteristic with 4 aggregate macroeconomic variables. Each model is used to make stock-level out-of-sample predictions for a time period of 35 years from 1987 to 2021. When making the predictions we use information from the entire panel across all stocks and over time. This is different from traditional asset pricing models that estimate a cross-sectional model for each time period or independent time-series models for each stock. Next, we analyze which of the 479 predictor variables has the highest importance in the various models to give insights into the forecast drivers and make some of the black-box models more interpretable. For this, we apply the relatively new but promising method of SHapley Additive exPlanations (SHAP) values. Thereafter, we sort the return predictions into long-short portfolios and investigate whether investment strategies based on machine learning models outperform strategies that are based on conventional models.

In our analysis of the predictive ability of the different machine learning models, we first find that neural networks outperform other methods in terms of out-of-sample R^2. Comparing the predictive ability to other studies we find less but still economically meaningful predictability in our sample. Second, the most influential variables in our models are stock-level characteristics that relate to price trends and liquidity. Another dominant group of predictors relates to investment and growth variables. Third, and even more importantly, we find that using forecasts from machine learning models can result in substantial economic gains for investors. A value-weighted long-short portfolio that invests based on neural network forecasts generates an annualized out-of-sample Sharpe ratio of 0.73, which is substantially higher than a Sharpe ratio of 0.35 for a conventional momentum

strategy. In addition, the neural network-based long-short portfolio earns a significant annualized excess return (alpha) of 9.6% against the Fama and French (2015) and Carhart (1997) six-factor model. Combining forecasts from all models in our comparison to an ensemble strategy can further increase the annualized Sharpe ratio to 0.81.

Our work contributes to the existing literature in many ways. We provide a comprehensive and up-to-date analysis of the effectiveness of machine learning methods in predicting stock returns and their practical implications for investment strategies. We show that most of the results of previous studies can be replicated. However, replication is challenging and requires a deep understanding of the underlying models. We also show how newer methods such as XGBoost perform relative to better-known methods. Finally, we demonstrate how SHAP values can be used to analyze the importance of the predictors for the models' forecasts.

The thesis will mostly follow the work by Gu et al. (2020) and extend their analysis with newer data and some variation in the methods. In Section 2 the related literature will be presented and discussed. Section 3 provides an overview of the data while Section 4 outlines the methodology of the analysis. Section 5 presents the empirical results and shows that machine learning can improve the predictive power of asset pricing models. Section 6 offers a critical examination of the usefulness and implications of the findings in practice and addresses some limitations of machine learning methods. Section 7 concludes.

2. Literature Review

To contextualize our analysis within the current literature, we begin with a comprehensive review of previous research on the predictability of stock returns and the application of machine learning in asset pricing. This literature review is an essential part of our analysis as it provides the necessary context and theoretical foundations for our research. First, we discuss the debate on market efficiency and stock return predictability, followed by a review of the literature on return prediction methods. We then review the literature on the use of machine learning models in asset pricing and return prediction.

2.1. Market Efficiency and Stock Return Predictability

The concept of market efficiency is central to the study of asset pricing. According to Fama (1970), a market in which prices always fully reflect available information is said to be efficient. This assumes that there are no transaction costs in trading securities and that all available information is available to all market participants at no cost. In addition, all market participants must agree on the impact of current information on the current price. If prices fully reflect all available information, price changes are random and prices would thus follow a random walk. Then it would not be possible to predict stock prices at all (Fama, 1970).

Fama (1970) summarizes the discussion on market efficiency by classifying three types of efficiency. In weakly efficient markets, asset prices reflect only historical price information. Therefore, it is not possible to obtain abnormal returns using only historical price information. In semi-strong efficient markets, all publicly available information is reflected in prices. This means that publicly available information cannot be used to generate abnormal returns. The final level of efficiency is strongly efficient markets. Here, both public and private information is incorporated into prices. However, this scenario is unlikely in reality.

Testing market efficiency is difficult, as it requires specifying what the market is supposed to do. Therefore, an asset pricing model that can specify expected returns is needed to test whether the returns implied by the model will be realized in reality. However, the outcome of this test does not necessarily indicate any information about the efficiency of the market. If the test is rejected, either the efficient market hypothesis does not hold or the model is not correctly specified. Unfortunately, there is no way to know which of the two scenarios is true. This is usually referred to as the joint hypothesis problem (Fama, 1991).

Although the degree of market efficiency is a topic of ongoing debate, several studies that are widely accepted suggest that there is at least some degree of predictability in financial markets (Campbell, 2000; Zhang et al., 2019). In this context, it is important to understand that this does not necessarily contradict the efficient market hypothesis. Instead, a modern interpretation of the efficient market hypothesis allows for the equity premium to vary countercyclically in a general equilibrium model (Campbell & Cochrane, 1999). This is because rational investors may exhibit more risk aversion to consumption and wealth volatility during recessions, leading to a countercyclical equilibrium equity

premium. Also, it is well known that even a small predictive component in excess stock returns can be economically relevant (Campbell & Thompson, 2008). Therefore, it is important to study and develop effective prediction models.

2.2. Exploring Return Prediction Methods: A Review

Given the extensive literature on return prediction, this section aims to provide a structured introduction to some of the most commonly used methods. Within this context, it is important to note that the term "predicting returns" in this thesis specifically refers to predicting excess returns, i.e. the equity premium, which is the focus of most studies on stock return prediction. Having this in mind, the section begins with a discussion of traditional econometric models and continues with regression-based approaches that use a variety of variables.

Traditional Time-Series Models

In traditional times-series forecasting, the objective is to forecast returns using only historical price information. This approach is based on the assumption that past trends and patterns in the data can help to predict future stock returns. The most popular time-series prediction methods are exponential smoothing, autoregressive integrated moving average (ARIMA) models, and autoregressive conditional heteroskedasticity (ARCH) models (De Gooijer & Hyndman, 2006).

The beginnings of exponential smoothing date back to various times-series techniques studied in the 1950s and 1960s (De Gooijer & Hyndman, 2006). Exponential smoothing combines the actual value of the current period with the predicted value from the previous period to calculate the forecast for the next period. The weight given to this combination of actual value and previous forecast decreases exponentially over time. This means that the model gives more importance to recent data and less importance to older data, which reflects the idea that recent stock returns are more relevant for predicting future returns than older ones (Gardner, 2006). Generally speaking, exponential smoothing is most suitable for forecasting data with no clear trend or seasonal pattern (Chatfield, 2000). However, there are several advanced variations of the exponential smoothing model that can account for trends and seasonality.

Although exponential smoothing is the most basic times-series forecasting method, its effectiveness has been tested in several studies. One such study by Leung et al. (2000) attempted to predict the sign of stock market returns. Leung et al. (2000) used adaptive exponential smoothing, in which the smoothing coefficient is continuously updated depending on the performance of the model in the previous period. The study tested different smoothing coefficients. Leung et al. (2000) found that classification models such as logit and probit performed better than exponential smoothing. Other studies have also failed to provide evidence supporting the notion that exponential smoothing is a suitable model for predicting stock returns. Therefore, other models may be more appropriate in this context.

As an alternative to exponential smoothing, ARIMA models are widely used for forecasting financial markets. ARIMA models were first introduced by Box and Jenkins (1970), who integrated existing knowledge of autoregressive (AR) and moving average (MA) models. An AR process is a weighted linear sum of the past values of a variable plus a random component. On the other hand, MA processes model the value of a variable as a linear combination of lagged error terms plus a random component. Combining the features of AR and MA processes results in an autoregressive moving average (ARMA) process. The ARMA model is useful in situations where the data shows both AR and MA patterns. The problem with ARMA models is that they assume that the times-series data is stationary, meaning that the statistical properties of the data, such as mean and variance, are constant over time. However, many real-world times-series data exhibit non-stationary behavior, such as trends or seasonality, which can lead to incorrect model specifications and unreliable predictions. This is where ARIMA comes in. The idea of ARIMA models is to differentiate the times-series data until it is stationary before fitting an ARMA model (Chatfield, 2000).

The research on ARIMA models is more comprehensive than that on exponential smoothing. Most of the research has shown that ARIMA models perform well in-sample but do not provide economic benefits when predicting equity returns out-of-sample (Beckers & Blair, 2002). This is attributed to the inability of ARIMA models to capture nonlinear patterns in the data. For this reason, much of the research on ARIMA models is concerned with a hybrid approach, combining the ARIMA model with other models, such as support vector machines (Pai & Lin, 2005; Dong et al., 2020). Other work focuses on analyzing the environment in which ARIMA models perform best. For example, Dong et al. (2020) analyze the optimal window length for ARIMA models and the corresponding predictions.

They find that a five-year running window is optimal for predicting daily U.S. equity returns. Also, they conclude that the absence of nonlinearity in the ARIMA model is not a major issue, as long as the data windows are chosen carefully.

Most times-series models, including ARIMA, assume constant forecast variance. However, financial data typically has periods when returns are either very high or low, and these periods tend to follow each other, resulting in clusters of high or low volatility (Bollerslev et al., 1994; De Gooijer & Hyndman, 2006). To account for this fact, Engle (1982) introduced ARCH models, which can model the change in volatility over time. Unlike the times-series models described earlier, ARCH models are used primarily to model and predict the volatility of financial returns rather than to predict the returns themselves. The idea behind these models is that by capturing the time-varying volatility, one can gain insight into the level of risk or uncertainty associated with returns. Thus, ARCH models can indirectly provide information about expected returns by providing insight into the level of risk or uncertainty in the market (Bollerslev et al., 1992).

Bollerslev et al. (1992) provide a comprehensive literature review of the application of ARCH models with stock return data. It is shown that the models exhibit significant results for predicting the volatility of stock returns, as well as the volatility of stock indices and futures markets.

Regression-Based Models

Traditional times-series forecasting focuses on predicting the return of a stock only based on past performance. Unexpected changes in market conditions or external factors can make this approach challenging. This is where time-series and cross-sectional regression-based models come in. In regression-based models, the predictability of stock returns is examined using a predictive regression model in which the excess return is predicted using one or more pre-defined predictors. One key advantage of those models is their ability to incorporate both time-series and cross-sectional data, providing a more comprehensive and accurate analysis of stock performance.

In the past, there were found several predictors that have predictive power in the aggregate U.S. stock market. The starting point of the corresponding research is the study of Sharpe (1964) and Lintner (1965) on the effects of risk and uncertainty on asset prices. They found that given a range of assumptions, a stock's expected return is linearly related to its beta in the cross-section. The beta is a measure of a stock's systematic

risk, measuring the volatility of a stock's returns relative to the overall market. In other words, the beta is the regression coefficient of a stock's excess return on the market portfolio's excess return. The resulting asset pricing model is the Capital Asset Pricing Model (CAPM). The idea behind the CAPM is that a security that carries high risk should expect a high compensation for this risk. Early research by Black et al. (1972) showed that the beta factor is useful for predicting security returns. This is confirmed by many other studies. For example, Polk et al. (2006) report a statistically significant cross-sectional beta premium when forecasting the Center for Research in Security Prices (CRSP) value-weighted index between 1927 and 2002.

However, Black et al. (1972) find that the expected excess return on an asset is not strictly proportional to its beta. This led to the development of several adaptions of the CAPM as well as to the search for other predictors of aggregate stock returns besides their betas (Rapach & Zhou, 2013).

To fit the empirical evidence on size and value effects, Fama and French (1993) introduced the three-factor model. This model takes into account the excess returns on a broad stock index, the excess returns of small stocks over large ones, and the excess returns of high book-to-market stocks over low book-to-market stocks. Fama and French (1993) show that the chosen three factors are useful for explaining the cross-section of returns. Later, Carhart (1997) made use of momentum effects and expanded the three-factor model to include high-return stocks from recent months as a fourth factor. Although these models work well, Fama and French (1993) critically address the fact that the factors chosen are based on empirical evidence rather than underlying theory.

In addition to the factors already mentioned, several studies provide evidence of other factors that predict aggregate U.S. stock returns. Green et al. (2013) find that more than 330 firm-specific predictive signals were reported in accounting and finance journals between 1970 and 2010. The most popular ones include the dividend-price ratio (Campbell & Shiller, 1988b; Fama & French, 1988; Hodrick, 1992), the earnings-price ratio (Campbell & Shiller, 1988a), the book-to-market ratio (Kothari & Shanken, 1997; Lewellen, 1999), and the dividend payout ratio (Lamont, 1998). Also, a range of macroeconomic factors was studied, including inflation (Nelson, 1976; Fama, 1981; Campbell & Vuolteenaho, 2004), default spread (Keim & Stambaugh, 1986), term spread (Campbell, 1987), and the net equity expansion (Boudoukh et al., 2007).

Instead of addressing each of the studies individually, one thing should be highlighted.

This is the fact that most of these studies provide evidence of U.S. aggregate stock return predictability in-sample, while the out-of-sample performance is rather poor (Campbell, 2000). In this context, Welch and Goyal (2008) question the usefulness of previous findings on the predictability of stock returns. They review many of the variables that the literature suggested in simple time-series regressions. Their results show weak predictive performance, both in-sample and out-of-sample. They conclude that the studied variables would not have been useful for an investor who only had access to publicly available information. Ultimately, the findings of Welch and Goyal (2008) suggest that predictive regressions are unstable and that the chosen models are unable to outperform the historical average.

Fortunately, several later studies have shown that newer approaches produce better results than traditional predictive regression models using the same predictor variables. Two of these approaches are presented below.

First, predictive accuracy can be improved by incorporating economically-driven restrictions on predictive regression forecasts (Rapach & Zhou, 2013). Campbell and Thompson (2008) predict monthly and annual stock returns on the Standard & Poor's 500 (S&P 500) index using simple regressions of excess returns on many of the above-mentioned predictor variables. They compare the performance of a model with sign restrictions on slope coefficients and forecasts as well as quantitative coefficient restrictions for valuation ratios to the baseline model. Imposing restrictions improved the out-of-sample R^2 from a range of -0.66% to 0.32% in the base case to a range of 0.24% to 0.97%, indicating a clear enhancement in predictive power. Campbell and Thompson (2008) conclude that when subjected to certain restrictions, the majority of the well-known predictor variables outperform the historical average return forecast in out-of-sample tests. Although the R^2s still appear to be low, Campbell and Thompson (2008) show that even small predictive R^2s are economically meaningful to investors.

As an alternative to imposing model restrictions, Rapach et al. (2010) suggest a forecast combination approach. They average the return predictions of individual regression models and calculate the predictive R^2 using a weighted prediction. For their forecasts, they use quarterly data of the 15 variables from Welch and Goyal (2008). For weighing the forecast, Rapach et al. (2010) use different approaches ranging from simple averaging schemes to building weights based on the historical forecasting performance of the individual models. They find that various combinations of the individual forecast outperform the historical average in terms of predictive power and conclude that their approach is very useful for predicting the equity premium out-of-sample.

2.3. Machine Learning in Asset Pricing

In recent years, the use of machine learning techniques has had a significant impact on the field of asset pricing, providing new tools and methods to improve predictive accuracy. The popularity of machine learning can be attributed to its ability to process huge amounts of data, advancements in computing power, lower data storage costs, and flexibility in capturing nonlinear relationships between variables (Weigand, 2019). After a brief introduction to the concept of machine learning, we present the most influential research in the field of asset pricing and forecasting that incorporates these techniques.

In general, machine learning enables machines to process data more efficiently. This is achieved by using algorithms that can recognize patterns and relationships in large and complex datasets (Dey, 2016). Since there are various applications for machine learning techniques, it is important to tailor their definition to the specific domain in which they are used. To this end, Gu et al. (2020) define machine learning in the context of empirical asset pricing. According to them, machine learning refers to

> "(1) a diverse collection of high-dimensional models for statistical prediction, combined with (2) so-called "regularization" methods for model selection and mitigation of overfit, and (3) efficient algorithms for searching among a vast number of potential model specifications" (Gu et al., 2020, p. 2225).

In other words, machine learning methods are extremely flexible due to their ability to process high-dimensional data. This allows them to better approximate the complex data-generating process underlying equity returns. However, the high flexibility increases the risk of overfitting the data. To counteract this, machine learning models are regularized to ensure stable performance and prevent overfitting. In addition, they optimize model specification with manageable computational costs (Gu et al., 2020).

Machine learning has found application in several areas of financial research. One major area is derivatives pricing. In early research, Hutchinson et al. (1994) and Yao et al. (2000) used neural networks to price options. Among others, Yao et al. (2000) show that the option pricing model with neural networks performs better than the traditional Black-Scholes model on highly unstable markets. Another application is the prediction of consumer credit risk. Both Khandani et al. (2010) and Butaru et al. (2016) test different machine learning methods for consumer credit risk prediction, with decision trees and random forests outperforming other models in terms of out-of-sample R^2. Other applications include real estate valuations or measuring bond risk premiums (Weigand, 2019).

With this in mind, the remainder of this section focuses on the application of machine learning in the area of equity pricing and return forecasting. The starting point is a vast amount of literature that combines machine learning with traditional econometric models. White (1988) attempted to predict daily IBM stock returns using feed-forward neural networks. The setup is inspired by AR models in that it uses past returns as the only explanatory variable. Unfortunately, White (1988) was not very successful in extracting nonlinear regularities from the time series of returns using a network with only one hidden layer. Among others, he argued that neural networks are prone to overfitting even with a very basic configuration (White, 1988). Another attempt to combine econometric models and machine learning is the study of Wang and Leu (1996), who predict the Taiwan weighted stock index using ARIMA-based neural networks. According to their empirical findings, networks trained with four years of weekly data can forecast the market trend for up to six weeks with an acceptable level of accuracy.

Building upon the earlier research, numerous recent studies have demonstrated economically significant performance of machine learning in return prediction. Freyberger et al. (2020) and Kozak et al. (2020) use shrinkage and selection methods to estimate a nonlinear function for expected returns. Freyberger et al. (2020) test different lasso methods to determine which firm characteristics explain the cross-section of expected returns without making strong functional form assumptions. They estimated different models using 62 characteristics and monthly return data of a set of U.S. stocks from July 1965 to June 2014. In their studies, the nonlinear adaptive group lasso dominates the linear lasso and the simple linear model. They also find that only 9 to 16 characteristics have explanatory power for expected returns, depending on the sample period and stock universe chosen. Moreover, it is noteworthy that they demonstrate time-dependent changes in the predictive ability of characteristics. Compared to the linear model, the nonparametric model chooses a smaller set of predictors and has an out-of-sample Sharpe ratio that is 2.5 times higher. Hence, Freyberger et al. (2020) show that their nonlinear models outperform linear models in predicting returns out-of-sample.

Kozak et al. (2020) instead use shrinkage and selection methods in constructing a robust stochastic discount factor that is supposed to summarize the explanatory power of a large number of cross-sectional stock return predictors. They use daily returns of the CRSP value-weighted index from 1926 to 2017 and examine several factors, including 50 well-known anomaly portfolios, portfolios based on lagged returns and financial ratios, as well as interactions of these characteristics. Moreover, Kozak et al. (2020) construct high-variance

principal components and use ridge and lasso penalties. Based on their results, a sparse factor model that contains only three, four, or five factors, is essentially meaningless. They argue that such a sparse model is unable to effectively capture cross-sectional price trends because of the large number of cross-sectional return predictors and the lack of redundancy among them. Instead, they find that a stochastic discount factor formed from a small number of principal components and coefficients shrunk with a combination of ridge and lasso penalties perform best (Kozak et al., 2020).

Despite the growing popularity of machine learning in asset pricing, determining the most effective models for predicting stock returns out-of-sample is a challenge. Gu et al. (2020) address this problem by comparing the performance of different machine learning methods in predicting U.S. stock returns. In their study, they use more than 900 predictors to predict monthly individual U.S. excess stock returns with data from 1957 to 2016. The predictor set includes firm-level characteristics, industry sector dummy variables, and interaction terms with macroeconomic factors. On average, their panel consists of more than 6,200 companies per month. In their approach, the models maintain the same form over time and across stocks to leverage information from the entire panel. As a benchmark, Gu et al. (2020) choose a panel regression of individual excess stock returns on size, book-to-market, and momentum characteristics. Their results are extensive. First, they show that neural networks and regression trees are the best-performing methods out-of-sample. The superior predictive performance is attributed to their ability to account for nonlinear interactions that are missed by alternative methods. Second, their study identified a limited number of dominant predictor signals that are consistent across methods. The most important predictors are associated with price trends, such as momentum, followed by indicators of stock liquidity, volatility, and valuation ratios. Third, Gu et al. (2020) conclude that the economic benefits of machine learning predictions are large. By testing the models in various investment strategies, they show, among other things, that timing the S&P 500 using a neural network is significantly superior to a buy-and-hold strategy. They also show that long-short strategies using forecasts from the benchmark model are outperformed by most other models.

The approach proposed by Gu et al. (2020) was subsequently tested in other markets. Drobetz and Otto (2021) apply the same methodology to the European stock market. They use data from all companies listed in one of the nineteen Eurozone countries between 1990 and 2020. As predictors, they use 22 firm characteristics and interactions between firm characteristics and their second- and third-order polynomials to capture nonlinearity.

Aside from having different data, the approach from Drobetz and Otto (2021) differs from the analysis of Gu et al. (2020) in the following ways. As benchmark model, Drobetz and Otto (2021) use a multiple linear model with 22 predictors instead of 3. Also, they include transaction costs in their analysis and challenge the performance of traditional expected return-based portfolio construction with a classification-based approach based on support vector machines. In analyzing long-only investment strategies, Drobetz and Otto (2021) find that all models outperform the linear benchmark model. As in Gu et al. (2020), the neural network stands out as the best model, even after accounting for transaction costs. Interestingly, Drobetz and Otto (2021) find that a classification approach that builds portfolios using support vector machines outperforms all strategies including neural networks. Their analysis leads them to conclude that a classification-based approach avoids some of the noise in excess returns while preserving the signal necessary for a successful trading strategy (Drobetz & Otto, 2021).

Leippold et al. (2022) conduct a similar study for the Chinese stock market. In their analysis, they use more than 1,000 predictors and monthly return data of A-shares listed on the Shanghai and Shenzhen stock exchanges from 2000 to 2020. In addition to the methods used by Gu et al. (2020), Leippold et al. (2022) also employ the variable subsample aggregation (VASA) approach. Their results confirm regression trees and neural networks as the best methods in terms of the out-of-sample R^2. The out-of-sample R^2 for VASA is similar to that of regularized linear models. Leippold et al. (2022) report the highest R^2 of 2.71% for their regression tree-based model, which is almost seven times higher than the highest R^2 of 0.40% obtained by Gu et al. (2020) with a neural network. They explain this result by the large differences between the Chinese and U.S. markets, which lead to higher predictability of the Chinese markets. The Sharpe ratios for the long-short investment strategies are also significantly higher than those obtained by (Gu et al., 2020). However, they may not be easy to implement given the shorting constraints in Chinese markets. Finally, Leippold et al. (2022) find that predictors based on liquidity are the most important factors, while predictors based on price dynamics have a relatively insignificant impact. This is in contrast to the results reported by Gu et al. (2020).

3. Data

As highlighted in the literature review, machine learning methods have shown promising results in predicting stock returns, especially when using a large set of predictors. Building on this, our study focuses on analyzing the predictive performance of machine learning techniques using monthly U.S. equity returns from 1957 to 2021. The dataset is derived from multiple sources, including financial databases and publicly available data libraries. To ensure reproducibility and transparency, a comprehensive overview of each data source and the various steps taken in data preparation is provided. Subsequently, descriptive statistics of the data are presented. The section concludes by explaining the construction of the final dataset that is used throughout the analysis. This includes details on the set of predictors as well as the normalization procedure.

3.1. Data Sources and Preparation

In total, four different data sources are used. First, the monthly return data for U.S. equities is downloaded from the Center for Research in Security Prices (CRSP), which is accessed via Wharton Research Data Services (WRDS). Similar to Gu et al. (2020), all companies listed on the New York Stock Exchange (NYSE), American Stock Exchange (AMEX), and National Association of Securities Dealers Automated Quotations Stock Market (NASDAQ) between 1957 and 2021 are selected. Instead of selecting only firms with a specific share code or price, we follow Gu et al. (2020) and use the largest possible number of firms in the analysis to avoid a sample selection or data snooping bias. After downloading the data, all duplicates and observations with missing return data are removed. This is necessary because predictions and observed returns cannot be compared without having information on the actual realization.

As our main predictor variables, we use a set of 94 firm characteristics and 10 industry variables representing the 10 Standard Industrial Classification (SIC) divisions. Table 1 lists the firm characteristics used in the following analysis. The data on firm characteristics and SIC codes for each firm are downloaded from the Dacheng Xiu Data Library (n.d.). Table 9 in Appendix A provides a detailed overview of the 94 firm characteristics based on the work of Green et al. (2013) including their abbreviations used in later tables, figures, and descriptive statistics. The characteristics are updated either monthly, quarterly, or annually. Thus, they are only available to the public with a delay. To avoid a forward-

Table 1: Firm Characteristics

No.	Firm Characteristic	No.	Firm Characteristic
1	Absolute accruals	48	36-month momentum
2	Working capital accruals	49	6-month momentum
3	Abnormal earnings announcement volume	50	Financial statement score
4	# years since first Compustat coverage	51	Size
5	Asset growth	52	Industry-adjusted size
6	Bid-ask spread	53	Number of earnings increases
7	Beta	54	Operating profitability
8	Beta squared	55	Organizational capital
9	Book-to-market	56	Industry adjusted % change in CAPEX
10	Industry-adjusted book to market	57	% change in current ratio
11	Cash holdings	58	% change in depreciation
12	Cash flow to debt	59	% change in gross margin - % change in sales
13	Cash productivity	60	% change in quick ratio
14	Cash flow to price ratio	61	% change in sales - % change in inventory
15	Industry-adjusted cash flow to price ratio	62	% change in sales - % change in A/R
16	Industry-adjusted change in asset turnover	63	% change in sales - % change in SG&A
17	Change in shares outstanding	64	% change sales-to-inventory
18	Industry-adjusted change in employees	65	Percent accruals
19	Change in inventory	66	Price delay
20	Change in 6-month momentum	67	Financial statements score
21	Industry-adjusted change in profit margin	68	Quick ratio
22	Change in tax expense	69	R&D increase
23	Corporate investment	70	R&D to market capitalization
24	Convertible debt indicator	71	R&D to sales
25	Current ratio	72	Real estate holdings
26	Depreciation / PP&E	73	Return volatility
27	Dividend initiation	74	Return on assets
28	Dividend omission	75	Earnings volatility
29	Dollar trading volume	76	Return on equity
30	Dividend to price	77	Return on invested capital
31	Earnings announcement return	78	Revenue surprise
32	Growth in common shareholder equity	79	Sales to cash
33	Earnings to price	80	Sales to inventory
34	Gross profitability	81	Sales to receivables
35	Growth in capital expenditures	82	Secured debt
36	Growth in long term net operating assets	83	Secured debt indicator
37	Industry sales concentration	84	Sales growth
38	Employee growth rate	85	Sin stocks
39	Idiosyncratic return volatility	86	Sales to price
40	Illiquidity	87	Volatility of liquidity (dollar trading volume)
41	Industry momentum	88	Volatility of liquidity (share turnover)
42	Capital expenditures and inventory	89	Accrual volatility
43	Leverage	90	Cash flow volatility
44	Growth in long-term debt	91	Debt capacity/firm tangibility
45	Maximum daily return	92	Tax income to book income
46	12-month momentum	93	Share turnover
47	1-month momentum	94	Zero trading days

Notes: This table presents the 94 firm characteristics used in our analysis. Data for the characteristics is obtained from Dacheng Xiu Data Library (n.d.).

looking bias in our predictions, we follow Gu et al. (2020) and incorporate a delay of up to one month for monthly data, at least four months for quarterly data, and at least six months for annual data. Consequently, when making predictions for returns in month $t+1$, we rely on the most recent monthly data available at the end of month t, the most recent

quarterly data available at the end of $t-4$, and the most recent annual data available at the end of $t-6$. This ensures that our predictions are based on information available at the time of the prediction. If the information on firm characteristics is missing, the missing value is replaced by the cross-sectional median for that month. This step is required given that the predictors must be normalized, which is not possible with missing values.

Since SIC codes cannot be used as predictors without transformation, dummy variables are formed from each SIC division. An overview of the 10 SIC divisions can be found in Table 2. To avoid multicollinearity between the dummy variables, we drop one of them and end up with a total of nine dummy variables. Any missing values are set to 99, which corresponds to the category for unclassifiable establishments (United States Department of Labor, n.d.).

Table 2: SIC Divisions

Division	Major Group	Industry Title
A	01 – 09	Agriculture, Forestry, and Fishing
B	10 – 14	Mining
C	15 – 17	Construction
D	20 – 39	Manufacturing
E	40 – 49	Transportation, Communications, Electric, Gas, and Sanitary Services
F	50 – 51	Wholesale Trade
G	52 – 59	Retail Trade
H	60 – 67	Finance, Insurance, and Real Estate
I	70 – 89	Services
J	91 – 99	Public Administration

Notes: This table shows the structure of SIC divisions based on the United States Department of Labor (n.d.).

Next, data on four macroeconomic factors are obtained from the Amit Goyal Data Library (n.d.). The definitions of these factors align with the ones provided by Welch and Goyal (2008). First, to have a measure of the corporate issuing activity, the net equity expansion (ntis) is used. The dollar amount of net equity issuing activity is calculated as

$$\text{Net Issue}_t = \text{Mcap}_t - \text{Mcap}_{t-1} * (1 + \text{vwretx}_t), \tag{3.1}$$

where, $Mcap$ is the total market capitalization, and $vwretx$ is the value-weighted return (excluding dividends) on the NYSE index. Second, we use the treasury bill (tbl) rate as a measure of short-term interest rates. In our analysis, the tbl rate corresponds to the three-month secondary market rate from the Federal Reserve Economic Data (FRED)

database. Third, we use the term spread (tms), which is calculated as the difference between the long-term government bond yield and the tbl rate. The reason for including tms is that it is frequently considered a good predictor of the market's expectations for future economic growth and inflation (Estrella & Mishkin, 1996). Lastly, the default yield spread (dfy) is used, which gives an understanding of corporate bond returns. It is calculated by subtracting AAA-rated corporate bond yields from BAA-rated corporate bond yields.

The tbl rate from the Amit Goyal Data Library (n.d.) is not only used as a macroeconomic factor but also as a proxy for the risk-free rate. Since the rate is stated on an annual basis, we convert it to a monthly rate according to the information provided by the U.S. Department of the Treasury (n.d.).

Since we are limited in computational resources, we only use a subset of the features used by Gu et al. (2020). In contrast to their study, we do not use the dividend-price ratio (dp), earnings-price ratio (ep), book-to-market ratio (bm), and stock variance (svar) in our analysis. This is because information on these valuation ratios and the volatility of the aggregate market is already included in the firm characteristics data listed in Table 1. Thus, there is no intuitive reason for interacting these macroeconomic factors with the individual firm characteristics as well.

Finally, the data on the five-factor model and the momentum factor are retrieved from the Kenneth French Data Library (n.d.). The five-factor model comprises information about the small minus big (SMB), high minus low (HML), robust minus weak (RMW), and conservative minus aggressive (CMA) factors, as well as the market excess return. In the factor analysis of the portfolios, we add the momentum factor to the five-factor model (FF5 + MOM) and therefore also obtain data on momentum. In addition, the Kenneth French Data Library (n.d.) provides us with the NYSE market equity breakpoints, which are used as the benchmark for market capitalization when constructing portfolios.

3.2. Descriptive Statistics

To gain a better understanding of the characteristics of the data, descriptive statistics are presented. The data includes a total of 32,756 firms, with a monthly mean of 5,252 firms. Thus, the data sample provides a comprehensive view of the U.S. equity market. Figure 1 displays the monthly number of firms over time. The figure shows a clear upward trend, indicating that it has become more common for firms to be listed in recent years.

This trend reflects a growing interest in public offerings and improved access to financial markets.

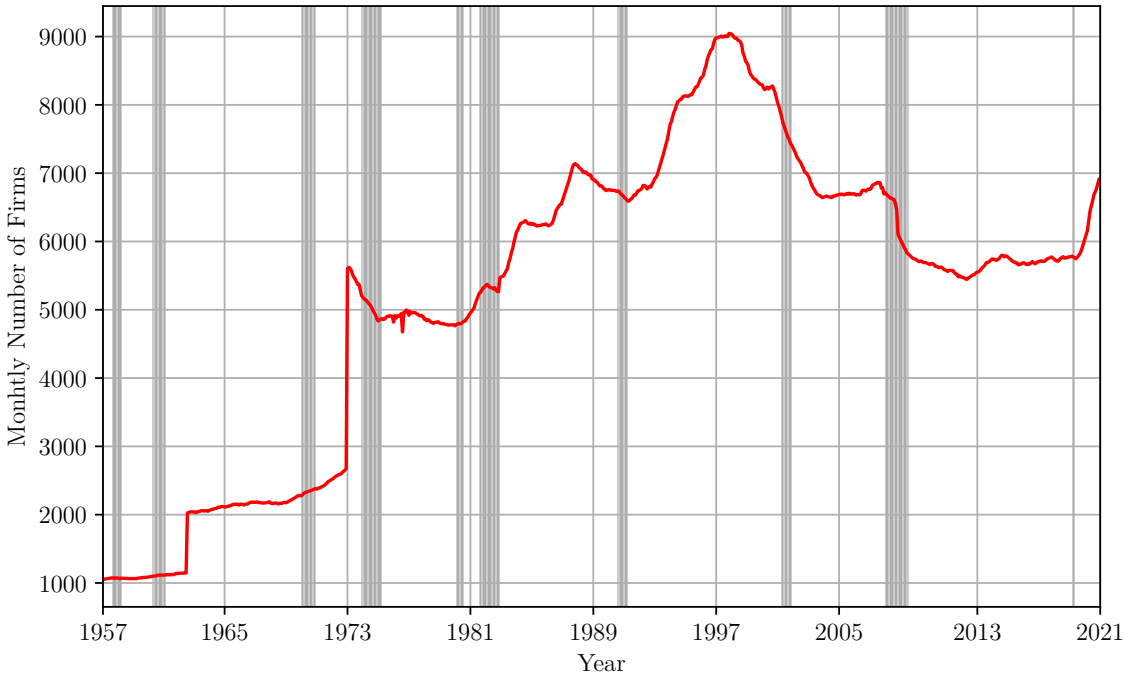


Figure 1: Monthly Number of Firms over Time
This figure shows the monthly number of firms in the data sample from 1957 to 2021. The grey-shaded areas mark NBER recession periods in the U.S. (Source: data from own analysis, recession indicators from FRED (n.d.)).

Next, we focus on the descriptive statistics of the four macroeconomic factors. By analyzing these factors, it is possible to gain insights into the broader trends that may have affected firm performance and provide a more comprehensive understanding of the economic landscape. The mean and standard deviation of the factors are reported in Table 3. However, mean and standard deviation alone are not particularly meaningful because of the variability of the factors over time. Figure 2 provides a better understanding by plotting the factors between 1957 and 2021. The grey-shaded areas indicate NBER recessions. It can be seen that macroeconomic factors respond differently to economic trends. During economic booms, the net equity expansion ratio tends to increase, while during recessions it tends to decrease. For example, the net equity expansion ratio decreased during the global financial crisis in 2008, followed by an increase afterward. The treasury bill rate also tends to decrease during recessions as central banks lower interest rates to stimulate economic growth but can be influenced by other factors such as high

inflation. In contrast, the default yield spread typically increases in recessions, while the term spread tends to narrow as investors become more risk-averse and demand for short-term securities increases.

Table 3: Descriptive Statistics of Macroeconomic Factors

Factor	Mean	Std
Net equity expansion	1.05%	1.96%
Treasury bill rate	4.33%	3.14%
Term spread	1.71%	1.42%
Default yield spread	1.00%	0.43%

Notes: This table reports the mean and standard deviation of the macroeconomic factors between 1957 and 2021.

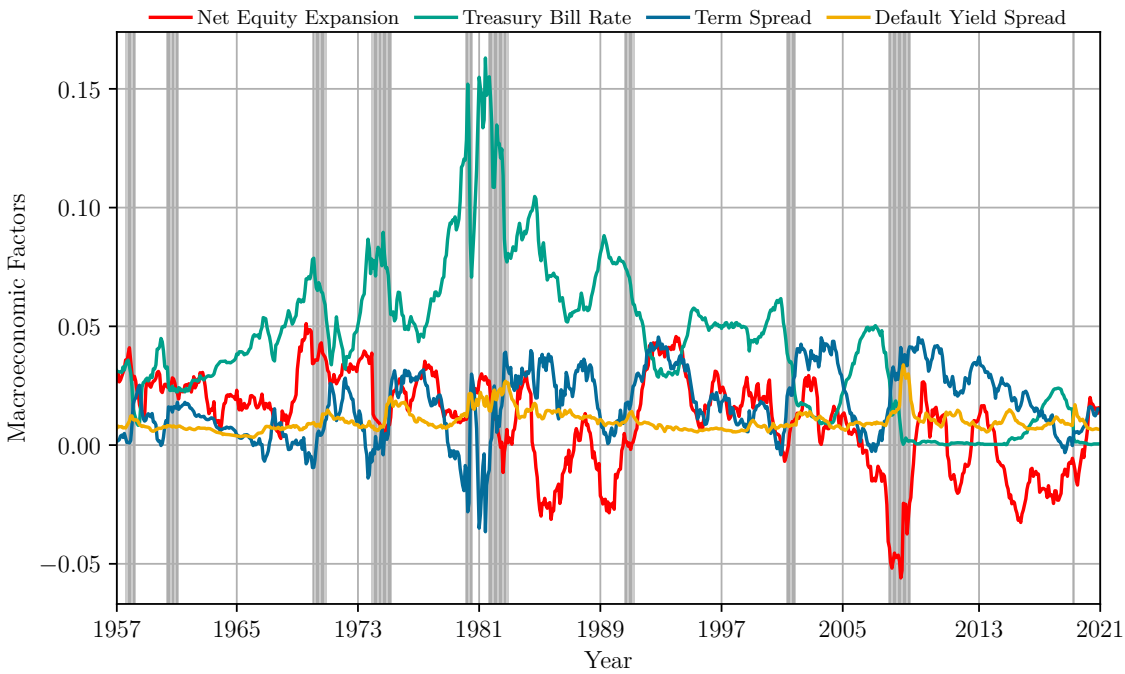


Figure 2: Macroeconomic Factors over Time
This figure shows the macroeconomic factors described in Section 3.1 from 1957 to 2021. The grey-shaded areas mark NBER recession periods in the U.S. (Source: macroeconomic factors from Amit Goyal Data Library (n.d.), recession indicators from FRED (n.d.)).

It is important to mention that the objective of this analysis is not to evaluate the effectiveness of the macroeconomic factors in predicting recessions, but rather to demonstrate their ability to capture the broader economic trends and provide insights for predicting

stock returns. Also, it is worth noting that the behavior of these factors is complex, which makes it difficult to comprehensively explain their fluctuations over time.

Finally, the mean and standard deviation of the firm characteristics are provided in Table 1 in Appendix A.

3.3. Construction of Final Dataset

The final dataset used to analyze the models is generated by merging the return data with the firm characteristics and macroeconomic factors. The excess returns are calculated by subtracting the risk-free rate from the returns and are used as the dependent variable. Next, interactions between all firm characteristics and macroeconomic factors are calculated:

$$x_{i,t} = m_t \otimes c_{i,t}, \tag{3.2}$$

where $c_{i,t}$ is a $P_c \times 1$ matrix of firm characteristics for each stock i, and m_t is a $P_m \times 1$ vector of macroeconomic factors common to all stocks (including a constant) (Gu et al., 2020). With a total of 94 firm characteristics and four macroeconomic factors, this results in 376 interaction variables that serve as predictors in addition to the 94 individual firm characteristics.

To prepare the predictor set, the 94 firm characteristics as well as the 376 interaction variables are normalized. This is important because some machine learning methods are sensitive to the scale of the input data. For example, principal component regression (PCR) and partial least squares (PLS) regression are both not scale-invariant, which means that the results of these methods can be affected by the scale of the input data (James et al., 2021). The normalization process follows the approach used by Kelly et al. (2019), Freyberger et al. (2020), and Gu et al. (2020). First, the predictors are cross-sectionally ranked for each month. Then, the ranks are mapped into the interval $[-1, 1]$, using

$$z_{ij,t} = \frac{rank(x_{ij,t})}{N_t} * 2 - 1, \tag{3.3}$$

where $rank(x_{ij,t})$ is the rank of stock i and predictor j in month t, and N_t is the total number of firms in month t.

Finally, the nine dummy variables for the SIC divisions are also included as predictors, bringing the total number of predictors to $94 + 376 + 9 = 479$. Throughout the analysis,

this predictor set of 479 features is referred to as $z_{i,t}$, where $z_{i,t}$ is a $P \times 1$ vector of features (with $P = 479$) to predict the individual stock returns. $z_{i,t}$ consists of the firm characteristics, the interactions between the firm characteristics and the macroeconomic factors, and the industry dummies. More formally, $z_{i,t}$ can be represented as:

$$z_{i,t} = \begin{pmatrix} c_{i,t} \\ m_t \otimes c_{i,t} \\ d_{i,t} \end{pmatrix}, \tag{3.4}$$

where $c_{i,t}$ is a 94×1 vector of firm characteristics, $m_t \otimes c_{i,t}$ is the Kronecker product of the 5×1 vector of macroeconomic factors m_t (including a constant), and $d_{i,t}$ is a 9×1 vector of industry dummies.

4. Methodology

With a comprehensive understanding of the final dataset, we proceed to the methodology of our analysis. The methodology section begins with an overview of our overall approach, followed by a detailed description of how we optimize the models and our method for splitting the sample. Next, we provide a comprehensive description of the machine learning models used in our analysis, including their regularization. Having introduced the different models, the section continues with details on the comparative performance evaluation. Subsequently, we introduce SHAP values, a powerful tool to determine which predictors drive the models' performance. Finally, the backtesting approach as well as the corresponding performance measures and factor analysis are presented.

4.1. Overall Approach

To examine the ability of machine learning models in forecasting U.S. stock returns, we follow the framework of Gu et al. (2020). First, we construct several models to predict the excess returns of a large U.S. panel of individual stocks. Based on the predicted excess returns we sort the stocks into decile portfolios. We provide more details on how we define our portfolio investment strategies in Section 4.7. Finally, we evaluate both the individual model forecasts as well as the performance of the investment strategies. To understand the

drivers of the forecasts we also shed light on the variable importance of our 479 predictors. Specifically, we want to find out which variables are most important in forecasting stock returns. In contrast to Gu et al. (2020), we use a longer time horizon as more recent data is available. Since we are restricted in computational resources, we limit our set of predictor variables and only use a subset of the explanatory variables used by Gu et al. (2020) as described in Section 3.

We use the set of 479 input variables $z_{i,t}$ of stock i in month t to predict the excess return of the following month $t+1$. Generally, following Gu et al. (2020), we define an asset's excess return as an additive prediction error model as follows

$$r_{i,t+1} = E_t(r_{i,t+1}) + \epsilon_{i,t+1}, \tag{4.1}$$

with

$$E_t(r_{i,t+1}) = g^*(z_{i,t}). \tag{4.2}$$

In both equations the index i indicates stocks $i = 1, \ldots, N_t$ and t marks the months $t = 1, \ldots, T$. $z_{i,t}$ is the P-dimensional vector of predictors. For ease of presentation, we assume a balanced panel of stocks in Equation (4.2) but treat missing values in the further analysis as described in Section 3 (Gu et al., 2020). The objective is to find the predicted excess return as a flexible function of the predictor variables $g^*(z_{i,t})$ that minimizes the mean squared error (MSE). Note that $g^*(\cdot)$ does neither depend on i nor t. Hence, the model maintains the same form over time and across stocks and can thereby utilize information from the entire panel. This is different from traditional asset pricing models that estimate a cross-sectional model each time period t or independent time-series models for each stock i. Additionally, $g^*(z_{i,t})$ only depends on $z_{i,t}$ which means that predictions are not based on any information prior to t or from individual stocks apart from the ith (Gu et al., 2020).

Since some of our models are computationally very intensive, we do not re-estimate models every month but instead re-estimate $g^*(z_{i,t})$ once a year. The following two sections expand more on the specific design of the model training process and the construction of training and out-of-sample test sets.

4.2. Optimization of Machine Learning Models

When estimating and training the general model in Equation (4.2), machine learning models as well as traditional models optimize an objective function. In our case, this is the mean squared error (MSE) between the predicted excess returns and the realized excess returns. However, minimizing the MSE alone is not sufficient. The goal is to train a model that is also able to make accurate predictions on new, unseen data from the test set. This means the model should be able to generalize as well (Müller & Guido, 2016).

If a model can only make accurate predictions on the training set, it is too complex and overfits the data. This is known as the bias-variance trade-off. A model with high bias will underfit the data, while a model with high variance will overfit the data (Müller & Guido, 2016). Overfitting occurs when a model is too complex and captures not only the underlying patterns in the data but also the noise. This will result in poor performance on the test set as the model overfits and has high variance and low generalizability.

Many machine learning models, especially neural networks, are prone to overfitting due to their complex nature and the many parameters they contain (Hastie et al., 2009; Müller & Guido, 2016; Gu et al., 2020). To avoid this, regularization techniques are used which introduce a penalty term that discourages the model from fitting the noise. Regularization of the models is especially important in our application of predicting stock returns. The reason for this is that financial data is characterized by having low signal-to-noise ratios which makes it more difficult to capture reliable patterns in the data as it becomes challenging to differentiate meaningful signals from random noise (Israel et al., 2020). We will elaborate more on this in Section 6. The regularization techniques rely on a selection of hyperparameters that control model complexity and are therefore critical to achieving a good performance. Examples of hyperparameters are the penalization terms in lasso and ridge, the number of trees in random forests, or the depth of these trees (Gu et al., 2020). Hyperparameters have to be determined adaptively from the data and cannot be known in advance. For this, we create a validation sample containing observations that the trained model did not observe yet. The validation sample is used to find the optimal combinations of the tuning parameters in an iterative process that trains and tests various combinations of hyperparameters. The combination of parameters that leads to the lowest validation loss (MSE) is selected for the final model which is then used to make predictions on the test set. All models have different hyperparameters and we therefore explain the detailed choice of tuning ranges in the corresponding model sections

in Section 4.4. Generally, there is not much guidance on how to tune the hyperparameters (Müller & Guido, 2016; Gu et al., 2020). When selecting the specific values or intervals for a specific model we primarily rely on the findings of Gu et al. (2020) as the finance literature on machine learning for stock return prediction is rather limited. In addition, we also refer to the general machine learning literature and experiment with different intervals with trial-and-error as the hyperparameter choice is highly dependent on the dataset and the specific problem or application (Géron, 2019; Gu et al., 2020; James et al., 2021).

In the following, we focus on the tuning process, while we explain the process of splitting the data into training, validation, and testing samples in the next section. There are different methods, also called tuning algorithms, to find the optimal combination of tuning parameters. One of the most common methods is grid search. Grid search takes a "grid" of parameters with specified values and trains a model for each combination of tuning parameters. The set of tuning parameters is chosen so that the validation loss is minimized and is then used to train the final model for making predictions (Müller & Guido, 2016; James et al., 2021). Grid search is a simple and effective way to tune hyperparameters, but it can be computationally expensive because each combination of tuning parameters needs to be trained and tested (Müller & Guido, 2016). Often a large range of hyperparameters needs to be specified to find the minimum loss and avoid running into a local minimum. To mitigate this, techniques such as random search, which randomly samples hyperparameters from a predefined distribution, or Bayesian optimization, which uses a probabilistic model to select hyperparameters that are likely to improve performance, can be used. One major disadvantage of grid search and random search is that they do not use the information from previous trials of parameters. Instead, they simply try a certain set of combinations and choose the one with the lowest validation loss (James et al., 2021). In contrast, Bayesian optimization leverages the information from past steps when deciding on which values of the tuning parameters to try next (Brochu et al., 2010). Bayesian optimization is a sequential model-based approach for optimizing a function that is expensive to evaluate, where the goal is to find the minimum or maximum of the function with as few evaluations as possible.

While Gu et al. (2020) use grid search for all models that need parameter tuning, we only use grid search for computationally less intensive models. This is more reasonable as we can then test a large range of tuning parameters, such as the number of components in PCR or PLS, to make sure we find the optimum. However, this would be unfeasible for computationally expensive models like XGBoost, random forests, and neural networks.

Therefore, we apply Bayesian optimization (Bayesian search) for these models. Instead of specifying specific values of the parameters like in grid search, with Bayesian search we only have to specify an interval with upper and lower bounds for each tuning parameter. This allows us to still test a wide range of hyperparameter combinations and lets us find solutions which we might not have found with grid search.

Bayesian optimization uses *Bayes' theorem* which states that the *posterior* probability of a model M given some observed evidence E is proportional to the product of the likelihood of E given M and the *prior* probability of M (Brochu et al., 2010):

$$P(M|E) \propto P(E|M)P(M), \tag{4.3}$$

where in our setting, the belief about the space of possible objective functions is the *prior*.

The main idea behind Bayesian optimization is to approximate an objective function f, that is costly to evaluate, with a surrogate function that is less expensive to optimize (Brochu et al., 2010). Bayesian optimization incorporates a prior belief about the objective function f and updates the prior with samples drawn from f to get a posterior that better approximates f (Brochu et al., 2010). As surrogate function, Bayesian optimization uses a Gaussian process model which represents a distribution over possible functions that are consistent with the observed data (Brochu et al., 2010). In addition, Bayesian optimization uses an acquisition function to efficiently direct the search to areas where an improvement over the current best observation is likely (Brochu et al., 2010). By maximizing the acquisition function, which balances exploration and exploitation, the next point (tuning parameter) to evaluate is selected. Here, exploitation refers to the process of using the current knowledge of f and sample where the surrogate model predicts a high objective, i.e. areas that are likely to improve the current best observation. Exploration, on the other hand, means sampling at locations where the prediction uncertainty is high and which have not been explored yet (Brochu et al., 2010). The acquisition function is high when both exploitation and exploration are high. In other words, with Bayesian optimization, the true objective function f will at each iteration only be evaluated at the point x^* that maximizes the surrogate function. In mathematical notation, the objective function f will be sampled at

$$x_t = \underset{x}{\operatorname{argmax}}\, u(x|\mathcal{D}_{1:t-1}), \tag{4.4}$$

where u represents the acquisition function and $\mathcal{D}_{1:t-1} = (x_1, y_1), \ldots, (x_{t-1}, y_{t-1})$ are the previous $t-1$ samples drawn from f (Brochu et al., 2010). As the observations are accumulated in $\mathcal{D}_{1:t} = \{x_{1:t}, y_{1:t}\}$, a prior distribution function $P(f)$ is combined with the likelihood function $P(\mathcal{D}_{1:t}|f)$ to generate the posterior distribution which reflects the updated beliefs about the unknown objective function and is given as

$$P(f|\mathcal{D}_{1:t}) \propto P(\mathcal{D}_{1:t}|f)P(f), \tag{4.5}$$

and equals the approximation of the objective with a surrogate function (Brochu et al., 2010).

Figure 3 illustrates an example of a Bayesian optimization for a simple 1-dimensional problem. In this example, we try to optimize the objective function that is represented by the dashed blue line. At each iteration, the acquisition function is maximized to find the next best location (x_t) to draw from the objective function (dashed green line). As explained earlier, the acquisition function is high when the Gaussian process predicts a high objective (exploitation) and when the prediction uncertainty is high (exploration) (Brochu et al., 2010). Areas where both are given are sampled first from the objective function ($f(x_t)$). Then, the Gaussian process is updated and the process is repeated iteratively until the objective function is maximized which leads to the optimal hyperparameters (Brochu et al., 2010).

We implement Bayesian optimization with the *BayesianSearch* algorithm from the *scikit-optimize* library. We choose the expected improvement (EI) as acquisition function, which is commonly used (Brochu et al., 2010). It measures how much the optimization of the objective function is expected to improve by evaluating the difference between the current best value of the function and the expected value of the function at the given point, relative to the uncertainty in the model's predictions (Brochu et al., 2010).

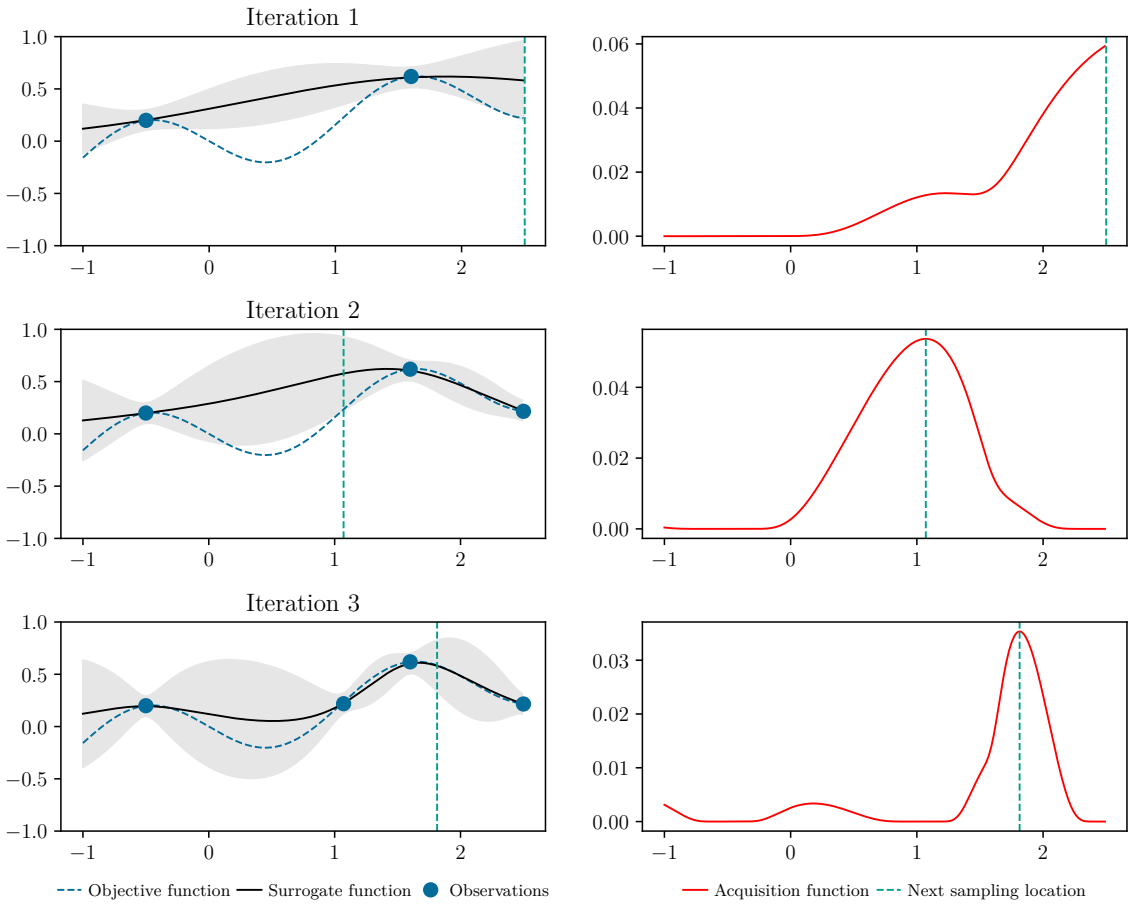


Figure 3: Illustration of Bayesian Optimization
This figure shows a Gaussian process approximation of the objective function f over three iterations. The dashed blue line represents the true unknown objective function f. The black line is the Gaussian process surrogate posterior mean and the grey-shaded area is the posterior uncertainty $\mu(\cdot) \pm \sigma(\cdot)$. The red line represents the acquisition function and the dashed green line is the location of the next sampling. (Source: own figure with simulated data, example inspired by Brochu et al. (2010) and Krasser (2018)).

4.3. Sample Splitting

In the machine learning literature, there are several best practices of how to split the data into training, validation, and test samples. For instance, the data is often split into 80% training and 20% testing, and the hyperparameters are tuned using cross-validation (Géron, 2019). In cross-validation, the data is split into multiple subsets where one of them is withheld for testing while the others are used for training the model. This is repeated for each subset, with each subset taking a turn as the test set (Géron, 2019; James et al., 2021). However, since we are working with time-series data we cannot apply

cross-validation as this would change the temporal order of the data (Arnott et al., 2019; Gu et al., 2020). Instead, we divide the data into a training, a validation, and a testing sample for each out-of-sample year. The tuning algorithms described in Section 4.2 are applied to find the best parameters that lead to the lowest validation loss (MSE in the validation sample) by training the models on the training sample and using them to make predictions on the validation sample. Once the hyperparameters are found, the model can be fit on the whole training and validation sample to use as much data as available. Finally, out-of-sample predictions are made on the testing set. This ensures that the model performance is evaluated truly out-of-sample (Gu et al., 2020). The reason for using a validation sample is to simulate an out-of-sample test to evaluate how the model performs with a certain parameter. Since the validation data is used for tuning, this is of course not completely out-of-sample. Therefore, we also need a testing sample to make the final out-of-sample predictions for our model evaluations and backtesting approach.

Next, it has to be decided how the different windows move over time and how long each of the three windows should be. West (2006) compares different data splitting methods such as *fixed*, *rolling*, and *recursive* splits. The simplest approach of the three is the *fixed* split which divides the data once into a training-, a validation-, and a test set and then forecasts all stocks in the whole test window using one fixed model (West, 2006). From a computational standpoint, this may be the simplest approach. However, in the case of predicting stock returns, the inclusion of more recent data in the training process could have significantly improved the forecast performance of the models. Hence, we will not use the *fixed* splitting scheme.

In contrast to the *fixed* split, incorporating a *rolling* window approach means keeping the length of the three windows constant. However, instead of having one fixed time frame, the training and validation periods gradually shift forward by one period to include more recent data. The disadvantage of this approach is that not all the available data is used for model training as the window size is fixed (West, 2006).

The *recursive* or *expanding* window approach fixes this disadvantage by always including the entire history of available data while still gradually including the most recent data. Hence, the window size of the *recursive* splitting scheme gradually increases every year (West, 2006). The downside of the last two approaches is that they are computationally very expensive (Géron, 2019; Gu et al., 2020). However, for our objective using as much data as available seems to be the best way as some of the models such as neural networks need a large amount of data to make precise forecasts (Géron, 2019; Gu et al., 2020; James

et al., 2021).

For our analysis, we follow the sample splitting approach of Gu et al. (2020) and use a hybrid version of the above approaches. We choose an expanding training window and recursively increase the training window by one year each time we refit the models. The length of the validation window remains constant, but it is shifted forward each time the models are refit, using a rolling window approach. Hence, we use the same model to make forecasts for all months in the year following the validation period. Following Gu et al. (2020), we set the initial size of the training window to 18 years and keep the length of the validation window at 12 years.

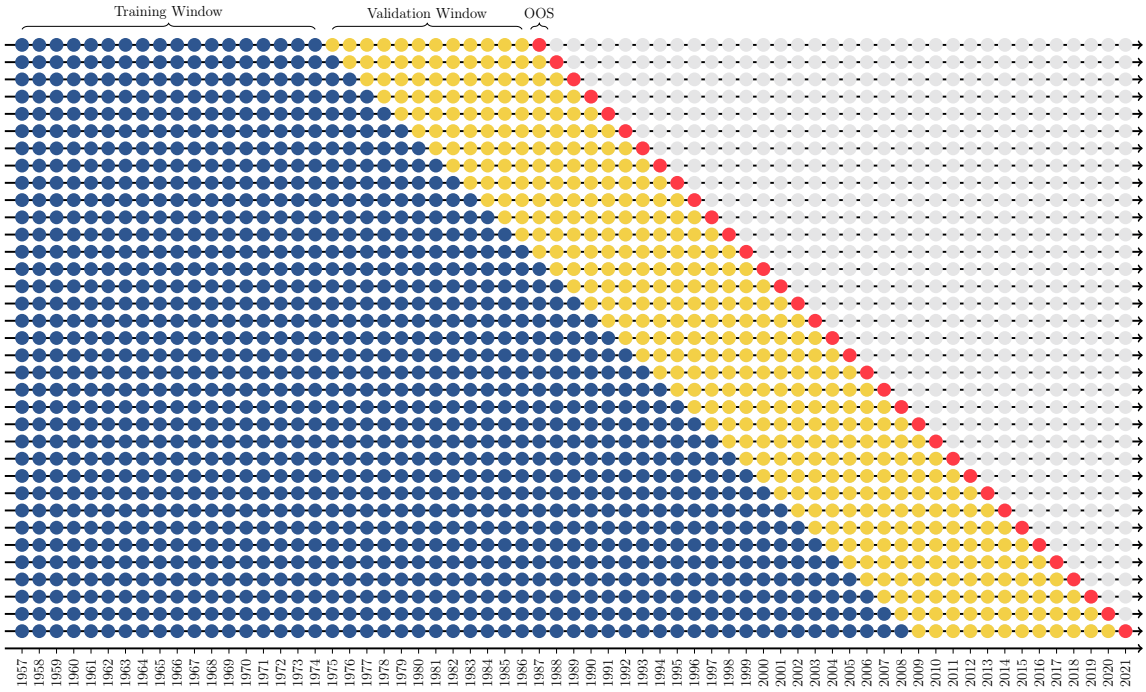


Figure 4: Sample Splitting

This figure shows our sample splitting process. Each dot represents data for one year. Blue indicates that data belongs to the training sample, yellow indicates the validation window, and red is the out-of-sample test year. The light grey data is unknown data for that specific period. In total, we have 35 estimation iterations with out-of-sample (OOS) test years from 1987 to 2021. (Source: own creation based on Gu et al. (2020)).

Figure 4 illustrates our sample splitting approach. Each colored dot marks one year in our time horizon from 1957 to 2021 and the rows of dots represent the forecast iterations. Models are re-estimated once a year and one-month-ahead out-of-sample forecasts are made for the subsequent out-of-sample year (illustrated with red dots in the graph). We do not refit the models every month because machine learning algorithms are computationally

intensive and because most of our predictor variables are only updated once a year (see Table 1 in Appendix A). After each estimation, we expand the training window by one year and keep the validation window as a rolling window at 12 years. This means that we have a total of 35 years in the out-of-sample testing period for our backtesting approach. For instance, for the first out-of-sample year 1987 (first row in Figure 4), the training period is 18 years (1957 – 1974) and the validation period for hyperparameter tuning is 12 years (1975 – 1986). For the next out-of-sample year 1988, the training period is then 19 years and the validation window is rolled forward by one year and kept at a length of 12 years. This is continued until the last out-of-sample year 2021 is reached with a total training window of 52 years.

4.4. Models

In this section, we describe the specific models and algorithms that we apply in our analysis. Models are trained and optimized as described in the previous sections. Each of the following sections first gives an introduction to the methodology of the corresponding model and then continues with the specifics on how we implement and construct the models. This includes the choice of regularization techniques and the interval ranges of the hyperparameters.

4.4.1. Multiple Linear Regression Model

The multiple linear regression model is a commonly used asset pricing model that is both simple and easy to interpret. In some cases, it can provide better predictions than more complex nonlinear models. This is particularly true when dealing with limited training data, low signal-to-noise ratios, or sparse data (Hastie et al., 2009). The multiple linear model expresses conditional expectations of future returns $g^*(\cdot)$ as a linear function of predictor variables and a parameter vector θ,

$$g(z_{i,t};\theta) = z'_{i,t}\theta, \tag{4.6}$$

where $z'_{i,t}$ is the P-dimensional vector of predictors, and θ the vector of model parameters.

As can be seen in Equation (4.6), all predictors enter the model linearly. However, the model can also be used to analyze nonlinear relationships between the predictors and

the dependent variable. This is achieved, for example, by transforming the predictors nonlinearly before they enter the model (James et al., 2021).

The most popular method to solve the multiple linear model is ordinary least squares (OLS) estimation. In OLS, the coefficients of the model are determined by minimizing the sum of squared differences between the observed and predicted values of the dependent variable. Put differently, OLS finds the set of coefficients that minimize the sum of squared residuals. Since we use panel data, the pooled OLS estimator is obtained by minimizing

$$\mathcal{L}(\theta) = \frac{1}{NT} \sum_{i=1}^{N} \sum_{t=1}^{T} \left(r_{i,t+1} - g(z_{i,t}; \theta) \right)^2 , \tag{4.7}$$

where $r_{i,t+1}$ is the observed return of stock i in month $t+1$, while $g(z_{i,t}; \theta)$ is the return prediction for stock i in month $t+1$, determined in month t (Gu et al., 2020).

As discussed in Section 2, previous studies have mostly used the multiple linear model as a parsimonious model with a limited number of predictors. Therefore, it is questionable whether the model still works well when we use 479 predictors, where some of them may be highly correlated. To make it easier to understand, we answer this using the example of a basic multiple regression model instead of the more complex panel data model that we have. In such a model, the statistical properties of the OLS estimator depend on whether the Gauss-Markov assumptions are satisfied. The five Gauss-Markov assumptions for the multiple linear model are listed in Appendix B. Under MLR.1 through MLR.5, OLS estimators are the best linear unbiased estimators (Woolridge, 2020). With this in mind, we now turn to the implications of including many predictor variables in the model.

On the one hand, some aspects are in favor of including many predictors. Intuitively, one can account for more variation in the dependent variable if more independent variables are included in the model. Also, while predictors in multiple regression analysis cannot be perfectly correlated, assumption MLR.3 does allow for some correlation. If this allowance did not exist, the usefulness of multiple regression in econometric analysis would be severely restricted. Finally and most importantly, as long as assumptions MLR.1 through MLR.4 hold, including irrelevant variables or overspecifying the multiple regression model will not impact the unbiasedness of the OLS estimators (Woolridge, 2020).

However, using too many correlated predictors in a multiple regression model increases the variance of the estimators. To understand this, note that under MLR.1 to MLR.5, the

variance of the OLS estimator for predictor j is given as

$$\mathrm{Var}(\hat{\theta}_j) = \frac{\sigma^2}{SST_j(1 - R_j^2)}, \tag{4.8}$$

where σ^2 is the residual variance, $\mathrm{SST}_j = \sum_{i=1}^{N}(x_{ij} - \bar{x}_j)^2$ gives the total sample variation in z_j, and R_j^2 measures the strength of the linear relationship among the predictors (Woolridge, 2020).

Instead of providing a detailed explanation of each component of Equation (4.8), the focus is on R_j^2. R_j^2 can be determined by regressing z_j on the other predictors. When R_j^2 is close to one, the other predictors explain much of the variation in z_j. In other words, z_j is highly correlated with the other independent variables. This problem, known as multicollinearity, affects the variance of OLS estimators. As can be seen in Equation (4.8), the closer R_j^2 is to one, the higher the variance. In the case where R_j^2 approaches one, $\mathrm{Var}(\hat{\theta}_j)$ approaches infinity (Woolridge, 2020).

Two things can be inferred from this. First, having a lower degree of correlation between the independent variables is typically preferable. Second, adding irrelevant variables to the multiple linear model usually leads to an increase in the variance of the OLS estimator, resulting in a less precise estimator (Woolridge, 2020).

When predicting returns, the size of the variance has important practical implications. If OLS estimators have a high variance, they are less precise, resulting in less accurate predictions. Thus, the question of whether the multiple linear model can be successful with many correlated predictors is easily answered. Since we assume that it is not practical to include all 479 predictors in the model, we follow Gu et al. (2020) and also use a second multiple linear model that uses only size, book-to-market, and momentum as preselected predictors. The two models are referred to as *OLS-all* and *OLS-3*, respectively, where we use the latter as the benchmark model for our analysis.

The models are implemented with *LinearRegression* from the *scikit-learn* library. Since there are no hyperparameters to be tuned in linear regression, the initial training window is set to 30 years instead of 18 years. Note that we only reduce the number of predictors for the OLS-3 model. All subsequent models are estimated with the full set of predictors.

4.4.2. Penalized Linear Model: Elastic Net

Often, the results of the multiple linear model are not satisfactory. As discussed above, when many predictors are included in the model, the large variance of the OLS estimators can cause them to be inefficient (Tibshirani, 1996; Woolridge, 2020). In particular, if the data is noisy, the model tends to overfit the data rather than extract the signal (Murphy, 2012; Gu et al., 2020). To mitigate overfitting and improve the prediction accuracy of the model, various penalty techniques can be used. These techniques aim to create a more parsimonious model by incorporating mechanisms of shrinkage and variable selection. In our analysis, we use the elastic net, which is a combination of the two well-known techniques ridge and lasso.

In a ridge regression, all regression coefficients are shrunk toward zero by penalizing the size of the coefficients (Hastie et al., 2009). This can significantly reduce the variance of the coefficients, resulting in a more stable model (Tibshirani, 1996; Hastie et al., 2009). Like any penalized linear model, ridge regression modifies the multiple linear model by adding a penalty term to Equation (4.7). Following James et al. (2021), the modified loss function has the form

$$\mathcal{L}(\theta; \cdot) = L(\theta) + \phi(\theta; \cdot). \tag{4.9}$$

For ridge regression, $\phi(\theta; \cdot)$ is the sum of squared coefficients given as

$$\phi(\theta; \lambda) = \lambda \sum_{j=1}^{P} \theta_j^2, \tag{4.10}$$

where λ is a constant that multiplies the penalty term. Thus, λ controls how much the coefficients are shrunk toward zero. When $\lambda = 0$, the loss function is not penalized at all. For $\lambda \to \infty$, the coefficient estimates approach zero. Minimizing the sum of squared coefficients is also referred to as l_2 regularization (James et al., 2021).

In general, ridge regression is most beneficial when the OLS coefficient estimates have high variance. This may be the case, for example, when the relationship between the predictors and the target is approximately linear. Then, small variations in the training data can lead to substantial variations in the coefficients obtained using least squares (James et al., 2021).

The disadvantage of ridge regression is that it always keeps all predictors in the model. Therefore, it sometimes fails to construct a model that is sparse and easy to interpret

(Tibshirani, 1996; Zou & Hastie, 2005). This is where lasso regression comes in. In lasso regression, the sum of the absolute values of the coefficients is minimized, which is known as l_1 regularization. Put differently, lasso regression requires the sum of the absolute values of the coefficient estimates to be smaller than a constant. In this way, some coefficients are set to exactly zero, while others are shrunk toward zero (Tibshirani, 1996). The lasso penalty $\phi(\theta; \cdot)$ is calculated as

$$\phi(\theta; \lambda) = \lambda \sum_{j=1}^{P} |\theta_j|, \tag{4.11}$$

where again λ controls the amount of shrinkage. Comparing Equation (4.10) and Equation (4.11), the only difference to ridge is that lasso replaces θ_j^2 by $|\theta_j|$.

By setting some of the coefficients to zero, lasso penalization results in a model with fewer predictors. Therefore, lasso regression is preferred over ridge regression when a simpler, more interpretable model is desired.

Combining ridge and lasso, the elastic net penalty balances sparsity and regularization (Murphy, 2012). Since neither model dominates the other in all scenarios, the degree of the lasso and ridge components in the elastic net is kept variable (James et al., 2021). Following Gu et al. (2020), the elastic net penalty added to the loss function from Equation (4.9) is given by

$$\phi(\theta; \lambda, \rho) = \lambda(1-\rho) \sum_{j=1}^{P} |\theta_j| + \frac{1}{2}\lambda\rho \sum_{j=1}^{P} \theta_j^2, \tag{4.12}$$

where λ is a constant that multiplies the penalty terms and ρ modifies the strength of the ridge and lasso components. It can be seen that the model corresponds to ridge when $\rho = 1$, while it corresponds to lasso when $\rho = 0$.

The model is implemented using the *ElasticNet* class from *skicit-learn*. Both λ and ρ are tuning parameters obtained from the validation sample using grid search. We use $\lambda \in \{10^{-4}, 10^{-3}, \ldots, 10^{-0}\}$ and $\rho \in \{0.25, 0.5, 0.75\}$ as pre-defined choices for the model.

4.4.3. Dimension Reduction: PCR and PLS

When dealing with numerous predictors, a high correlation is often observed between them. This poses a challenge for penalized linear models. Although they can effectively manage the complexity of highly parameterized models, they may not be able to fully account for the presence of strongly correlated independent variables (Drobetz & Otto,

2021). In this case, dimension reduction can be a superior approach. The basic idea of dimension reduction is to form a small number of linear combinations from the original predictors and then use them as independent variables in a regression (Hastie et al., 2009). This reduces noise and helps to better capture the signal of the predictors. In forming these combinations, the structure and correlation between the independent variables are used to obtain a more compact version of the data, while minimizing the loss of information (Deisenroth et al., 2020). In the following, we present principal component regression (PCR) and partial least squares (PLS) regression. While both methods share the same fundamental idea, they differ in how the linear combinations are constructed.

PCR regression involves two steps. First, a principal component analysis (PCA) is performed to find a dimension-reduced predictor set of the original predictor set. This dimension-reduced predictor set consists of K principal components that are intended to represent the information contained in the original predictor set as well as possible (Deisenroth et al., 2020). Principal components are constructed so that the first principal component has the largest variance considering all linear combinations of the original predictors. Thus, it captures as much variation in the data as possible. In other words, the first principal component is the dimension along which the data vary the most. Graphically, this is the line that best fits the data (James et al., 2021). Subsequent principal components have the maximum variance provided they are orthogonal to previous ones (Hastie et al., 2009). This implies that all constructed components are uncorrelated.

After performing the PCA, the principal components are used as predictors in a linear regression. Since the principal components are orthogonal, this regression can technically be considered a sum of univariate regressions (Hastie et al., 2009). The more principal components included in the regression model, the lower the bias but the higher the variance of the coefficients (James et al., 2021).

PLS regression also follows a two-step procedure. However, unlike PCR, PLS uses information from both the response vector of returns $r_{i,t}$ and the original predictor set $z_{i,t}$ to form predictive components. This can be advantageous because it cannot be ensured that the components that best explain the predictors are also effective in predicting returns. In the first step of PLS, the returns are regressed sequentially on each independent variable j (Hastie et al., 2009). The resulting univariate coefficient φ_j represents the partial sensitivity of the returns to each predictor j (Gu et al., 2020). Next, the first PLS component is constructed by weighting each predictor j according to its corresponding weight φ_j, resulting in $k_1 = \sum_j \widehat{\varphi}_{1j} z_j$ (Hastie et al., 2009). This shows that PLS assigns

the highest weight to the predictor with the highest univariate coefficient and vice versa (Gu et al., 2020). Since the univariate coefficients are proportional to the correlation between $r_{i,t}$ and z_j, PLS assigns the highest weights to those predictors that have the strongest correlation with the returns (James et al., 2021).

To form the second component, each predictor z_j is sequentially regressed on the first component k_1. The residuals of these regressions can be considered as the information not yet explained by the first component. Then the procedure described above is repeated, using these residuals in place of the original predictors. By construction, the resulting component k_2 is orthogonal to k_1 (James et al., 2021). After the desired number of components is constructed, the components are used to predict the returns in a linear regression.

Next, PCR and PLS are expressed in mathematical terms following the notation in Gu et al. (2020). The starting point of both models is the vectorized version of the linear model, given as

$$R = Z\theta + E, \tag{4.13}$$

where R is the $NT \times 1$ vector of returns $r_{i,t+1}$, Z is the $NT \times P$ matrix of predictors $z_{i,t}$, and E is the $NT \times 1$ vector of residuals $\epsilon_{i,t+1}$.

The models reduce the dimensions of P by constructing K components that are linear combinations of the predictors. Derived from this, the predictive model is given as

$$R = (Z\Omega_K)\theta_K + \tilde{E}, \tag{4.14}$$

where Ω_K is a $P \times K$ matrix with columns $\omega_1, \omega_2, ..., \omega_K$. Each ω_j represents the weights to get the jth predictive component. Hence, $Z\Omega_K$ gives the dimension-reduced predictor set. The coefficient θ_K is now a $K \times 1$ vector instead of a $P \times 1$ vector.

As explained before, the difference between PCR and PLS lies in the way they construct their components. PCR focuses on identifying components that capture the largest amount of variation in the original predictor set. Put differently, the objective of PCR is to find the K linear combinations of Z that represent the full predictor set as accurately as possible (Gu et al., 2020). It must be ensured that the weights chosen to form the components sum to one and that the components are orthogonal to each other. Thus, the weights ω_j should solve

$$\omega_j = \underset{\omega}{\operatorname{argmax}} \operatorname{Var}(Z\omega), \tag{4.15}$$

$$\text{s.t.} \qquad \omega'\omega = 1, \qquad \text{Cov}(Z\omega, Z\omega_l) = 0, \quad l = 1, 2, ..., j-1.$$

Instead, PLS chooses weights that maximize the squared covariance between the components and the vector of returns. In other words, the goal is to find K linear combinations of Z that have the highest predictive relationship with the forecast target (Gu et al., 2020). Having the same constraints as in PCR, the weights ω_j must solve

$$\omega_j = \underset{\omega}{\text{argmax}}\, \text{Cov}^2(R, Z\omega), \tag{4.16}$$

$$\text{s.t.} \qquad \omega'\omega = 1, \qquad \text{Cov}(Z\omega, Z\omega_l) = 0, \quad l = 1, 2, ..., j-1.$$

In both methods, the number of components K is a tuning parameter, which is determined from the validation sample with grid search. We implement the PCR regression with *PCA* and *LinearRegression* from the *scikit-learn* library. The number of components K is chosen from the interval (1, 60). The PLS regression is implemented using *PLSRegression* from *scikit-learn*, where the possible number of components K is chosen among (1, 25).

4.4.4. Tree-Based Methods: Random Forests and XGBoost

In contrast to the linear models and models applying dimension reduction techniques described earlier, tree-based models do not make any assumptions about the distribution of the data. Instead, decision trees are used to recursively split the data into smaller subsets (regions) based on the values of the predictor variables. With respect to our application, decision trees group stocks based on the various firm characteristics such that observations within a group are similar in terms of future returns. Tree-based models are conceptionally simple and can be visualized and interpreted due to the resulting tree-like structure. This makes them particularly useful for generating insights into the underlying relationships between the response variable and the predictors (Hastie et al., 2009). An example of how the structure of a tree-based model could look is given in Figure 5. The top of the tree is called the *root node* which is split into two nodes in a binary tree. These nodes can be further divided into *intermediary nodes* or *leaf nodes/leaves* (the regions).

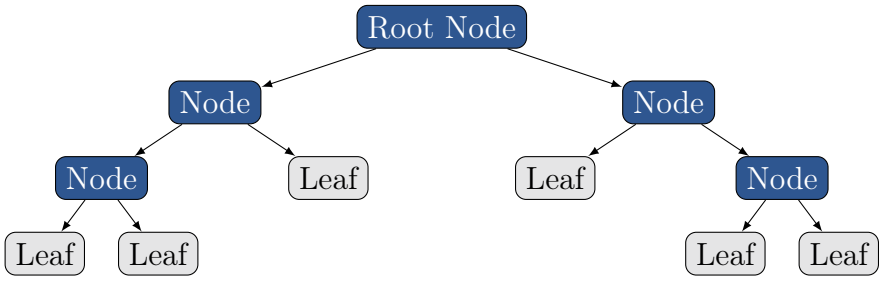


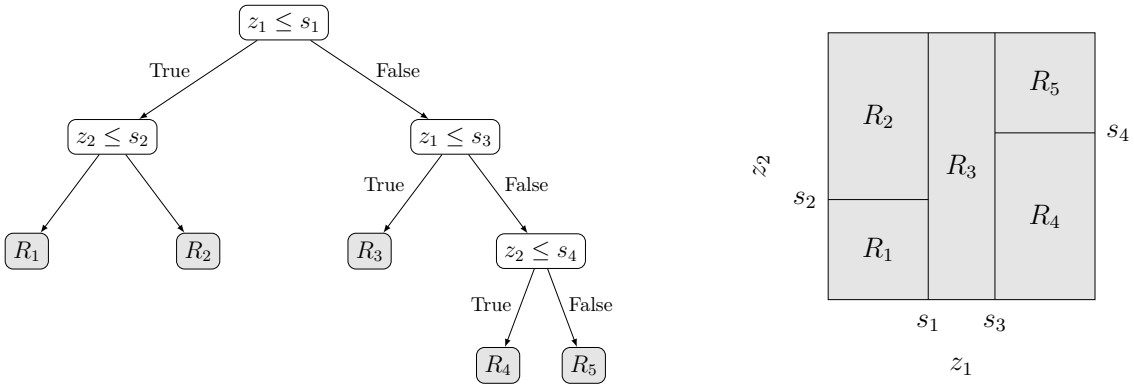
Figure 5: Structure of a Tree-Based Model
This figure shows an example tree structure. (Source: based on Hastie et al. (2009)).

There are several types of tree-based methods for both classification and regression tasks, which are all based on the concept of decision trees. Since our objective is to predict stock returns, we will only focus on regression trees. In our analysis, we use random forests and XGBoost. To understand these methods, we start by introducing the concept of decision trees.

Decision Trees

The basic idea behind tree-based models is to find groups of observations that are similar to each other. Following Gu et al. (2020), we focus on the well-known and one of the most used decision tree algorithms called *CART* which was introduced by Breiman et al. (1984) (Hastie et al., 2009). The tree algorithm partitions the data into a set of regions based on if statements involving values of the independent variables. This happens recursively and the tree grows in steps. First, the algorithm selects the best feature to split the data at the root node. The best feature represents the one that can best differentiate the data and produce the most homogeneous subsets. Then, the best feature is used to split the data into two subsets represented by nodes. This is repeated recursively until a stopping criterion is met. Each node can also be seen as a rectangular partition and approximates the function $g^*(\cdot)$ with the average value of the outcome variable within each partition (Hastie et al., 2009; Gu et al., 2020).

Figure 6 shows an example of how the tree algorithm and its splitting process work. The left panel shows the tree and how each observation gets assigned to a region based on the values of the input variables in the right panel. In this simplified example there is one response variable and two input variables z_1 and z_2, for instance, momentum and size in our application. First, the data is split on z_1. The region $z_1 \leq s_1$ is split at $z_2 = s_2$, and observations that are smaller or equal to s_2 are assigned to region 1, otherwise to region 2.



(a) CART **(b)** Partitions

Figure 6: Splitting Process and Partitions of CART
This figure shows an example of the splitting process of the CART algorithm. (Source: based on Hastie et al. (2009)).

Next, the region $z_1 > s_1$ is split at $z_1 = s_3$, and all observations that are smaller than s_3 are assigned to region 3. Finally, the region $z_1 > s_3$ is split at $z_2 = s_4$ and observations smaller or equal to s_4 go to region 4, while those above s_4 are assigned to region 5. Predictions for each partition are then defined as the average of the response variable's values among the observations in a partition. The prediction of a tree T can be formally written as

$$g(z_{i,t}; \theta, K, L) = \sum_{k=1}^{K} \theta_k \mathbf{1}_{\{z_{i,t} \in R_k(L)\}}, \tag{4.17}$$

where K is the number of leaves (terminal nodes), L is the depth of the tree and $R_k(L)$ is one of the K partitions of the data. It can be seen that each leaf (partition) is represented by a product of up to L indicator functions. The constant θ_k is defined to be the average of the response variable's outcomes within a partition k (Breiman et al., 1984; Hastie et al., 2009). In other words, stock i with characteristics $z_{i,t}$ is grouped into the k-th leaf and will be predicted to have a return of θ_k. The prediction equation for the example in Figure 6 is given as

$$g(z_{i,t}; \theta, 5, 3) = \theta_1 \mathbf{1}_{\{z_1 \leq s_1\}} \mathbf{1}_{\{z_2 \leq s_2\}} + \theta_2 \mathbf{1}_{\{z_1 \leq s_1\}} \mathbf{1}_{\{z_2 > s_2\}} + \cdots + \theta_5 \mathbf{1}_{\{z_1 > s_1\}} \mathbf{1}_{\{z_1 > s_3\}} \mathbf{1}_{\{z_2 > s_4\}}.$$

The main objective of decision trees is to find optimal splits of the data that minimize the forecast error. Since considering every possible partition of the feature space is computationally infeasible, CART employs a greedy approach and starts at the top node

and successively splits the data. The algorithm is said to be greedy because at each step of the tree-building process, the best split is made at that particular step (node), instead of considering future steps and choosing a split that will result in a better tree in some future step (Hastie et al., 2009; James et al., 2021). When growing the tree, several aspects have to be decided by the algorithm. At each node, the algorithm needs to select a variable among the 479 predictor variables available that serves as splitting variable and the value where the splitting variable should be split (split point). Furthermore, the algorithm needs to decide on the shape of the tree that determines the complexity. These aspects are chosen such that the forecast error is minimized. For the loss function, also called impurity measure in tree-based models, the CART algorithm chooses the l_2 impurity for each branch of the tree which is defined as

$$H(\theta, R) = \frac{1}{|R|} \sum_{z_{i,t} \in R} (r_{i,t+1} - \theta)^2, \tag{4.18}$$

where $|R|$ is the number of observations in region R (Hastie et al., 2009; Gu et al., 2020; James et al., 2021). Hastie et al. (2009) show that given R, the optimal choice of θ is $\theta = \frac{1}{|R|} \sum_{z_{i,t} \in R} r_{i,t+1}$.

Explaining the above more formally, the selection of the splitting variable and its split point works as follows. The greedy approach starts with the entire dataset at the root node and calculates the impurity for each feature $j = 1, 2, \ldots, P$ and all possible values of the cutpoint s for each of the predictors. Finally, the predictor and cutpoint are chosen that produce the largest reduction in impurity measure. Hence, for each candidate split (j, s) consisting of splitting variable j and cutpoint s the data in the parent node is partitioned into the regions R_{left} and R_{right}:

$$R_{left}(j, s) = \{z | z_j \leq s\} \qquad \text{and} \qquad R_{right}(j, s) = \{z | z_j > s\}, \tag{4.19}$$

where z_j denotes the jth feature. Then the performance of each split is evaluated using the loss function $H(R)$ defined in Equation (4.18) with $\theta = \frac{1}{|R|} \sum_{z_{i,t} \in R} r_{i,t+1}$, i.e.

$$\mathcal{L}(R, R_{left}, R_{right}) = \frac{|R_{left}|}{|R|} H(R_{left}) + \frac{|R_{right}|}{|R|} H(R_{right}). \tag{4.20}$$

The optimal split consisting of the optimal values for j and s is found by minimizing the

above equation:

$$(j^*, s^*) \leftarrow \underset{(j,s)}{\operatorname{argmin}}(\mathcal{L}(R, R_{left}, R_{right})).$$

After having found the best split at the current node, the above procedure is repeated at each of the two new regions R_{left} and R_{right} to split the data further (Hastie et al., 2009). This continues recursively until a stopping criterion is met, for example when the number of leaves or the depth of the tree reach a specified threshold (Gu et al., 2020). These thresholds are among the hyperparameters that need to be tuned in the parameter tuning process using a validation sample to avoid overfitting the data. Tree size represents model complexity in tree-based models and there is a similar trade-off between model complexity and the ability to generalize well to new data as explained in Section 4.2 (Hastie et al., 2009). Large trees might capture specific relationships in the data better but might be prone to overfitting.

Decision trees, i.e. models using only one single tree, have several major disadvantages. They tend to be very unstable and can often produce very different results even for small changes in the data (Hastie et al., 2009). Since decision trees are so flexible in capturing all sorts of interactions and nonlinearities, they are one of the methods that are most vulnerable to overfitting (Breiman et al., 1984; Hastie et al., 2009; Gu et al., 2020). Because of these two reasons, we employ ensemble methods that address these two issues by combining the forecasts of multiple decision trees to improve overall performance and robustness.

Random Forests

The combination of multiple decision trees can be done in several ways. The two most popular methods are bootstrap aggregation (also called bagging) and boosting. Following Gu et al. (2020), we use random forests as a slight variation of a more general procedure called bagging and one boosting method to predict stock returns. Both of our ensemble methods use the regression tree model described in the previous section as base learner.

As mentioned in the previous section, a single tree suffers from high variance due to its high sensitivity to the training data. Therefore, a single instance of an ensemble model is often called a weak learner due to its mediocre predictions. Bagging is a method to reduce the variance of a statistical model. It is based on the concept that variance can be reduced by averaging a set of observations. If we consider a set of n independent observations $z_1, \ldots, z_n$, each with variance σ^2, the mean $\overline{z}$ of the observations has variance σ^2/n. This

means that the variance is reduced by taking the average of the observations. Thus, if we had many training samples, we could train independent models on them and average their predictions. However, we usually do not have many training samples.

Hence, bagging works in three steps: bootstrapping, parallel training, and aggregation. First, the bagging algorithm draws B bootstrap samples from the data. Bootstrapping is a sampling technique to create diverse samples, each drawn at random and with replacement (Hastie et al., 2009). Second, a separate regression tree model is trained on each of the B bootstrap samples. When making predictions on new data, the bagging ensemble averages the predictions of all B regression trees to obtain a final prediction. This averaging of predictions from multiple trees helps to reduce the variance of the overall prediction and improve the generalization performance of the model (Gu et al., 2020; James et al., 2021).

Random forests are an extension of the bagging algorithm designed to decrease the correlation among the trees. They were introduced by Breiman (2001). Each time a potential split in a tree is considered, only a randomly selected subset m of all the P features is considered as splitting variables (Hastie et al., 2009). If we suppose that one variable is a very strong predictor in the data, then it is very likely that most of the B decision trees of a bagging algorithm will have this variable at the root node of the tree. This can lead to high correlation among the trees as they will look very similar and an average of highly correlated forecasts will not lead to a big decrease in variance (Hastie et al., 2009). Hence, when only using a subset of all predictors as splitting candidates, the correlation among trees and their bias can be reduced which leads to a better performance. Figure 7 shows the process of how a random forest model works.

Expressed mathematically, each tree in a random forest gives the forecast

$$\hat{g}_b(z_{i,t}; \hat{\theta}_b, L) = \sum_{k=1}^{2^L} \theta_b^{(k)} \mathbf{1}_{\{z_{i,t} \in R_k(L)\}}. \tag{4.21}$$

To receive the final prediction of the random forest, we just have to take the average of the outputs of all B trees:

$$\hat{g}(z_{i,t}; L, B) = \frac{1}{B} \sum_{b=1}^{B} \hat{g}_b(z_{i,t}; \hat{\theta}_b, L). \tag{4.22}$$

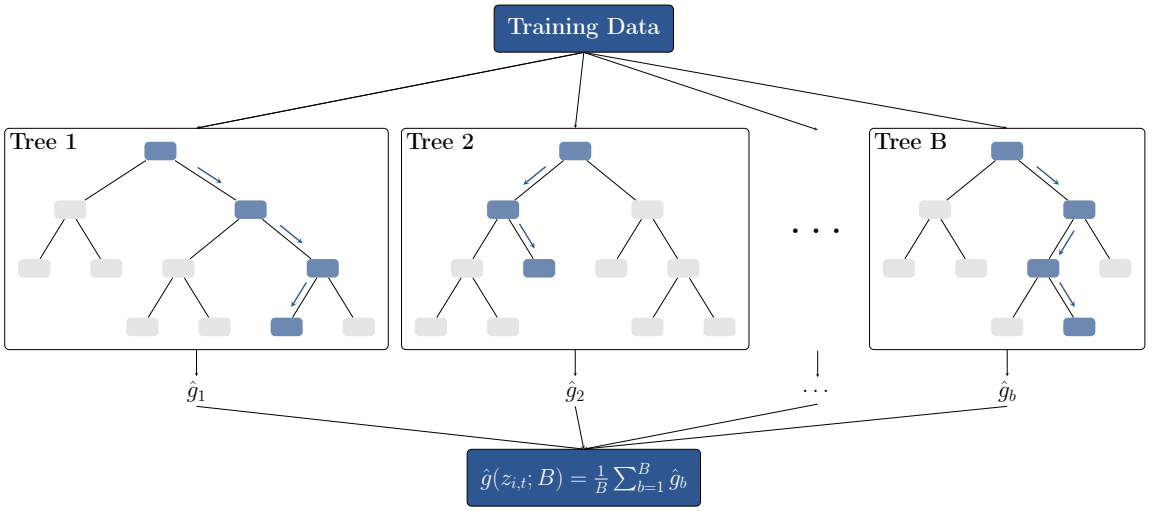


Figure 7: Random Forest Model
This figure shows the process of how a random forest works. The final prediction of a random forest is the average of the predictions of all the individual trees. (Source: own figure).

We implement the random forest algorithm using the *skicit-learn* library. The depth of each tree L, the number of predictors to be considered at each split, and the number of bootstrap samples B, i.e. the number of resulting trees, are tuned using Bayesian search on the validation sample. Following Gu et al. (2020), we choose the tree depth $L \in (1, 6)$, the number of trees $B \in (100, 300)$, and the number of features used as splitting candidates from the interval $(3, 479)$. Since the results of the random forests are associated with randomness, we use ensembles of 5 models computed with different random seeds as the final predictions made by the random forest. This makes the results more robust.

XGBoost

Similar to random forests, boosting methods also combine the forecasts of many trees but the process works slightly differently. There are several algorithms available. Gu et al. (2020) use gradient boosted regression trees (GBRT) as boosting algorithm, while we employ the eXtreme Gradient Boosting (XGBoost) algorithm. XGBoost is a specific implementation of the GBRT algorithm from Friedman (2001) that is a more optimized and efficient approach for building regression trees (Chen & Guestrin, 2016). Introduced by Chen and Guestrin (2016), XGBoost is more scalable for larger datasets and faster since it can leverage parallel computing. Moreover, it provides additional regularization techniques to prevent overfitting. This makes it more suitable for our purpose than GBRT.

While each tree in a random forest can be trained in parallel, the boosting algorithm

adds trees sequentially to the ensemble (James et al., 2021). Thereby, information from previous trees is used to improve the model where it currently performs poorly by training new trees on the residuals of the previous tree (Chen & Guestrin, 2016). The idea behind boosting is that later trees learn slowly from the errors previous trees have made. The final prediction is the sum of the predictions of all the trees in the ensemble and can be written as

$$\hat{r}_{i,t+1} = \hat{g}(z_{i,t}; B) = \sum_{b=1}^{B} \nu g_b(z_{i,t}), \qquad g_b \in \mathcal{G}, \tag{4.23}$$

where B is the number of trees, $\mathcal{G}$ is the set of all possible regression trees, and ν is the learning rate (Chen & Guestrin, 2016). The learning rate can be seen as a shrinkage factor that reduces the influence of each newly added tree to prevent overfitting the residuals (Chen & Guestrin, 2016; Gu et al., 2020). A smaller learning rate will require more boosting trees but can significantly improve generalization power (Friedman, 2001). For the following, it is important to understand that a single tree as described by Equation (4.17) can also be expressed as

$$g_b(z_{i,t}) = w_{t(z_{i,t})}, \quad w \in \mathbb{R}^K, \quad t : \mathbb{R}^P \to \{1, 2, \ldots, K\}, \tag{4.24}$$

where t represents the structure of a tree that maps each observation to the corresponding leaf and K is the number of leaves. w is the vector of scores on each leaf. Then, the final prediction is given by summing up the scores of the corresponding leaves (Chen & Guestrin, 2016). To learn the set of trees, we minimize the objective function

$$\mathcal{L} = \sum_{i=1}^{N} \ell(\hat{r}_{i,t+1}, r_{i,t+1}) + \sum_{b=1}^{B} \Omega(g_b), \tag{4.25}$$

where $\Omega(g_b)$ is a regularization term that penalizes the complexity of the model and is defined as

$$\Omega(g_b) = \gamma K + \frac{1}{2}\lambda \sum_{k=1}^{K} w_k^2. \tag{4.26}$$

Since Equation (4.25) includes functions as parameters, this equation cannot be optimized in Euclidean space. However, it can be trained in an additive manner. We can rewrite the

s-th step from the summation in Equation (4.23) as

$$\hat{r}_{i,t+1}^{(s)} = \sum_{b=1}^{s} \nu g_b(z_{i,t}) = \hat{r}_{i,t+1}^{(s-1)} + \nu g_b(z_{i,t}), \tag{4.27}$$

which we can use to write Equation (4.25) at step s as

$$\mathcal{L}^{(s)} = \sum_{i=1}^{N} \ell(r_{i,t+1}, \hat{r}_{i,t+1}^{(s-1)} + \nu g_b(z_{i,t})) + \sum_{b=1}^{B} \Omega(g_b). \tag{4.28}$$

In other words, this means that at each step we greedily add the tree g_b that leads to the highest improvement of our model. This objective function is then optimized using a second-order Taylor approximation to find the tree g_b that minimizes the objective:

$$\mathcal{L}^{(s)} \simeq \sum_{i=1}^{N} \left[\ell(r_{i,t+1}, \hat{r}_{i,t+1}^{(s-1)}) + p_i \nu g_b(z_{i,t})) + \frac{1}{2} q_i \nu g_b^2(z_{i,t})) \right] + \sum_{b=1}^{B} \Omega(g_b) + constant, \tag{4.29}$$

where $p_i = \partial_{\hat{r}_i^{(s-1)}} \ell(r_{i,t+1}, \hat{r}_{i,t+1}^{(s-1)})$ and $q_i = \partial^2_{\hat{r}_i^{(s-1)}} \ell(r_{i,t+1}, \hat{r}_{i,t+1}^{(s-1)})$ denote the first and second-order gradient statistics. Without constants, the objective function at step s to be minimized is

$$\tilde{\mathcal{L}}^{(s)} \simeq \sum_{i=1}^{N} \left[p_i \nu g_b(z_{i,t})) + \frac{1}{2} q_i \nu g_b^2(z_{i,t})) \right] + \sum_{b=1}^{B} \Omega(g_b). \tag{4.30}$$

Using the reformulation of a single tree from Equation (4.24), we can further simplify our objective function to

$$\begin{aligned} \tilde{\mathcal{L}}^{(s)} &\simeq \sum_{i=1}^{N} \left[p_i \nu g_b(z_{i,t})) + \frac{1}{2} q_i \nu g_b^2(z_{i,t})) \right] + \gamma K + \frac{1}{2} \lambda \sum_{k=1}^{K} w_k^2 \\ &= \sum_{i=1}^{K} \left[P_k w_k + \frac{1}{2} (Q_k + \lambda) w_k^2 \right] + \gamma K, \end{aligned} \tag{4.31}$$

where $P_k = \sum_{i \in I_k} p_i$, $Q_k = \sum_{i \in I_k} q_i$, and $I_j = \{i : t(z_{i,t}) = k\}$ is the set of indices of observations assigned to the k-th leaf (Chen & Guestrin, 2016). The optimal weight w_k^* and the corresponding best objective reduction w_k^* for a fixed structure $t(z_{i,t})$ are then

given by

$$
\begin{aligned}
w_k^* &= -\frac{P_k}{Q_k + \lambda}, \\
\mathcal{L}^* &= -\frac{1}{2} \sum_{k=1}^{K} \frac{P_k^2}{Q_k + \lambda} + \gamma K,
\end{aligned}
\tag{4.32}
$$

where $\mathcal{L}^*$ is a measure of how good a tree structure t is. It is similar to the impurity measure we introduced for the decision trees with the difference that it takes model complexity into account. XGBoost uses this measure to optimize the tree structure and find the best splits by iteratively adding branches to the tree like the CART algorithm explained earlier. The splitting candidates are evaluated using the following expression, where the variable and its corresponding splitting threshold are the ones that lead to the largest loss reduction, also called gain. The indices $Left$, $Right$, and $Prev$ indicate the *left*, *right*, and *previous* node after a split.

$$
\mathcal{L}_{split} = \frac{1}{2} \left[\frac{P_{Left}^2}{Q_{Left} + \lambda} + \frac{P_{Right}^2}{Q_{Right} + \lambda} - \frac{P_{Prev}^2}{Q_{Prev} + \lambda} \right] - \gamma \tag{4.33}
$$

Equation (4.33) consists of four parts: the loss reduction score on the left leaf, the score on the right leaf, the score on the previous leaf, and the regularization term at the end. If the loss reduction is smaller than γ, it is better to not add a branch and hence the regularization term γ ensures that the model does not overfit the data.

To summarize the steps above, the XGBoost algorithm starts by fitting a single tree on the training data. It then adds subsequent trees to correct the errors made by previous trees. Each tree grows greedily as it is impossible to iterate over all possible tree structures to find the best one. Instead, the algorithm starts at the top node and iteratively adds branches to the tree by evaluating the loss for each split with Equation (4.33). Here the predictor and the corresponding split point are chosen that lead to the largest reduction of the loss function $\mathcal{L}_{split}$, also referred to as gain. If the gain is smaller than the regularization term, the split into two more branches is not performed (Chen & Guestrin, 2016).

We implement the XGBoost algorithm using the *XGBoost* library. The depth of each tree L, the number of trees B, the learning rate ν, as well as the regularization term γ are tuned using Bayesian optimization on the validation sample. Following Gu et al. (2020), we choose the tree depth $L \in \{1, 2\}$, the number of trees $B \in (1, 1000)$, the learning rate $\nu \in \{0.01, 0.1\}$, and the regularization term $\gamma \in (0, 10)$. Similar to random forests, we

also use ensembles of 5 XGBoost models calculated with different seeds, to reduce the dependence on the random seed value.

4.4.5. Neural Networks

Decision trees and random forests learn from data by recursively partitioning the predictor space. In contrast, neural networks use a fundamentally different approach to learn complex relationships between predictors and the target. In their initial form, neural networks were developed as a model for the human brain (Hastie et al., 2009). The human brain processes information across multiple levels of abstraction. Similarly, neural networks process and transform information through many layers of interconnected nodes. In this context, nodes are the basic unit of computation in neural networks, also referred to as neurons or units. Because neural networks better understand the data at each layer, they are commonly referred to as deep learning methods (James et al., 2021).

There are many different forms of neural networks. The most common ones are feed-forward, convolutional, and recurrent networks. Feed-forward networks are the simplest form. They usually consist of an input layer, one or more hidden layers, and an output layer. The hidden layers are fed with information from the input layer and forward the processed information to the output layer. The information flows only in one direction, from the input layer to the output layer (James et al., 2021).

In convolutional neural networks, the interaction of layers differs from the idea in feed-forward networks. Convolutional networks are specialized in image classification tasks. To recognize and combine graphical features, a specific network architecture is required. Most commonly, convolutional networks consist of convolutional and pooling layers. In convolutional layers, many convolutional filters try to detect small patterns in the image, such as edges or color patches. The resulting information is combined in pooling layers. By stringing together many convolutional and pooling layers, images can be classified (James et al., 2021).

Just as convolutional networks are best suited for detecting dimensional structures in images, recurrent networks perform best when the input data is sequential. This is true, for example, for language processing problems where the order of words has semantic meaning. Another example is time-series data, such as daily temperature or stock prices. Recurrent networks process information by always taking a sequence of the predictor set as input. At each step in the sequence, they consider information from the previous

step by incorporating feedback loops. In this way, they can extract information from the sequential structure of the data. Unfortunately, fitting recurrent neural networks requires an enormous amount of computation. Also, the regularization of these networks is very difficult. Therefore, when dealing with time-series data, using lagged predictors as input matrix in an ordinary feed-forward network often leads to a simpler and more flexible model (James et al., 2021).

Generally speaking, two things should be kept in mind when using neural networks. First, the use of neural networks is most effective in problems where the ratio of information to noise in the data is high. This is because they tend to overfit when the data is too noisy (Hastie et al., 2009). As our application in stock return prediction is characterized by low signal-to-noise ratios, regularization is of great importance. Second, neural networks are black-box models that process data in a very complex way. This makes it more difficult to draw conclusions about the influence of certain predictors on the result (Baesens, 2014).

In our analysis, we use different feed-forward networks to predict returns. In the following, we first explain the architecture and functioning of a feed-forward network before describing its regularization.

Architecture and Functioning of Neural Networks

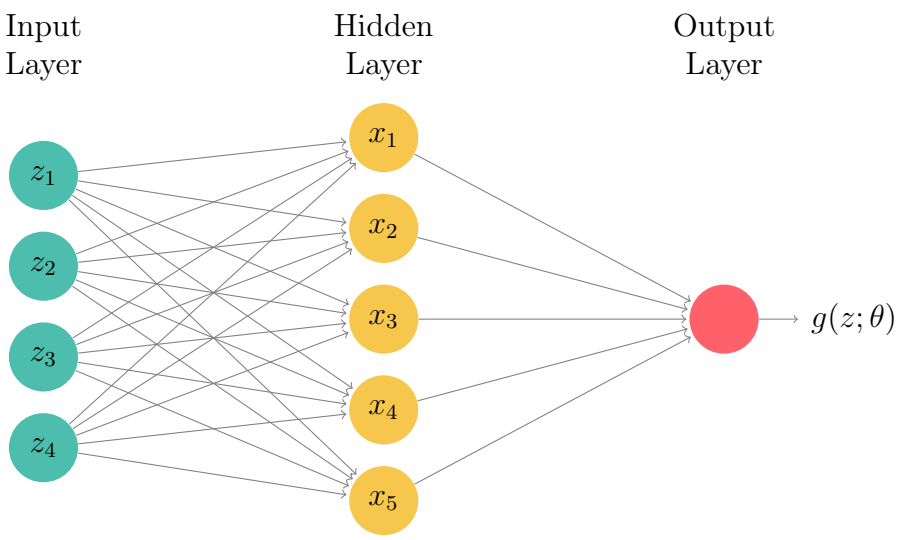


Figure 8: Architecture of a Single Hidden Layer Neural Network
This figure illustrates the architecture of a simple neural network with one hidden layer with 5 neurons $x_1, \ldots, x_5$. There are 4 input variables $z_1, \ldots, z_4$ to the model. (Source: based on James et al. (2021)).

Like any other model described, feed-forward neural networks build a function that uses a set of predictors $z_{i,t}$ to predict the response vector $r_{i,t}$. The simplest form of such a network is the so-called "vanilla" network shown in Figure 8. It consists of an input layer, one single hidden layer, and an output layer (Hastie et al., 2009). In our example, the model has four input nodes represented by the green circles z_1 to z_4. Each input node represents one predictor. Also, the model consists of one hidden layer with five neurons represented by the yellow circles x_1 to x_5. The neurons perform two tasks. In the first step, they determine five new features by computing five different linear combinations of the four predictors including an intercept. This step is represented by the grey arrows connecting the input and hidden layer. Next, the neurons calculate activations of these features. Activations are nonlinear transformations. Since these activations cannot be directly observed, the corresponding layer is called hidden layer (James et al., 2021). For any nonlinear activation function f, the activation $x(z)$ at each neuron in the hidden layer has the form

$$x_k(z) = f\left(\theta_{k,0}^{(0)} + \sum_{j=1}^{4} z_j \theta_{k,j}^{0}\right), \tag{4.34}$$

where $\theta_{k,0}^{(0)}$ is the intercept and $\theta_{k,j}^{(0)}$ the vector of weights for the linear combination of the predictors z_j at node x_k. The (0) in the power of θ indicates that $\theta^{(0)}$ represents the intercept and weights for the first transformation. The weights that $\theta^{(0)}$ chooses are determined by the solution algorithm (Gu et al., 2020).

When it comes to choosing the activation function, there are several options. In the past, the sigmoid function was the most common choice for nonlinear activation (Hastie et al., 2009). However, in modern neural networks, the rectified linear unit (ReLU) activation function is usually preferred. This is because the ReLU function can be computed and stored more efficiently than the sigmoid function (James et al., 2021). Therefore, in our analysis, we use the ReLU function, which has the form

$$\mathrm{ReLU}(x) = \begin{cases} 0 & \text{if } x < 0 \\ x & \text{otherwise} \end{cases}. \tag{4.35}$$

The final prediction target $g(z;\theta)$ is a linear model of the outcomes of the activations. Therefore, the neural network is technically a nonlinear extension of the linear model (Hastie et al., 2009). It follows that the final result that enters the output layer of the

neural network from Figure 8 is given as

$$g(z;\theta) = \theta_0^{(1)} + \sum_{k=1}^{5} x_k \theta_k^{(1)}, \tag{4.36}$$

where $\theta^{(1)}$ is the vector of weights used for the final prediction. Again, those weights are determined by the solution algorithm and among others depend on the chosen loss function that is minimized (Hastie et al., 2009).

Summarizing the steps described above, the single-hidden-layer model shown in Figure 8 first obtains new features by computing different linear combinations of the predictors. Then, each of these features undergoes an activation function that transforms them. The resulting model is linear with respect to the outcomes of the activations.

Theoretically, a model with a single hidden layer with many neurons is able to approximate most functions. However, the learning process can be significantly improved if a deeper model consisting of several hidden layers with fewer neurons in each layer is used instead (Eldan & Shamir, 2016). The so-called multi-layer network has a very similar architecture to the model displayed in Figure 8. The only difference is that it contains multiple hidden layers. When the predictors are transformed by the hidden layers, the second hidden layer takes the activations of the first hidden layer as input. This continues until the last hidden layer is reached. Thus, the chain of transformations of the predictors $z_{i,t}$ is longer than in a single-layer network (James et al., 2021).

In the generalized multi-layer neural network, the number of neurons in each layer $l = 1, ..., L$ is denoted by $K^{(l)}$, while the output of neuron k in layer l is denoted by $x_k^{(l)}$. Hence, the output of layer l, including a constant, can be written as $x^{(l)} = (1, x_1^{(l)}, ..., x_{K^{(l)}}^{(l)})'$. It follows that the input layer is given by $x^{(0)} = (1, z_1, ..., z_N)'$, where z_1 to z_N are the predictors. Combining these elements, Gu et al. (2020) define that for each neuron in layer $l > 0$, the output has the form

$$x_k^{(l)} = \text{ReLU}\left(x^{(l-1)'}\theta_k^{(l-1)}\right), \tag{4.37}$$

which leads to the final output of the network given as

$$g(z;\theta) = x^{(L-1)'}\theta^{(L-1)}. \tag{4.38}$$

Choosing the number of hidden layers and neurons in each layer is a challenging task. In

general, it is advisable to have a rather high number of neurons than too few in a model. Having too few neurons may limit the model's ability to capture the nonlinear complexities within the data. On the other hand, having too many neurons can be compensated for by applying appropriate regularization techniques to shrink the excess weights toward zero (Hastie et al., 2009). When choosing the number of hidden layers, the complexity of the problem plays a major role. Adding more layers increases the computation time, and can also lead to overfitting if the amount of data is small. Ultimately, the optimal number of hidden layers is often determined by experimentation. (Hastie et al., 2009).

Gu et al. (2020) find that a neural network with 3 hidden layers performs best with their sample. Thus, we implement neural networks with up to 4 hidden layers, in case a network architecture with more layers performs better in our sample. We call the neural networks NN1 to NN4, depending on the number of hidden layers. The selection of the number of neurons for each layer is based on the geometric pyramid approach introduced by Masters (1993). NN1 has a single hidden layer with 32 neurons. For NN2, there are two hidden layers with 32 and 16 neurons, respectively. Next, NN3 has three hidden layers with 32, 16, and 8 neurons, while NN4 has a total of four hidden layers with 32, 16, 8, and 4 neurons, respectively. We keep the set of different network architectures rather small due to the limited sample size compared to typical applications of neural networks (Gu et al., 2020).

The most common method for solving neural networks is the stochastic gradient descent (SGD) algorithm. Unlike gradient descent, SGD only considers a random subsample of the data to compute the gradient at each iteration and is therefore much faster (James et al., 2021). In our analysis, we use the Adam optimizer. The Adam optimizer is a variant of SGD that converges quickly, requires little memory, and is easy to implement (Kingma & Ba, 2014).

The main drawback of SGD optimization is that it is not convex in the parameters. This means that the error function may have several local minima in addition to the global minimum. As a result, if the randomly chosen starting weights are unfavorable, the algorithm may get stuck in a local minimum, leading to suboptimal results (Baesens, 2014).

Finally, it is important to keep in mind that the power of neural networks combined with SGD optimization comes at risk. In particular, these techniques have the potential to easily model the noise in the training data. This can significantly worsen prediction

performance and should therefore be avoided by applying several regularization methods described in the next section (Baesens, 2014).

Regularization of Neural Networks

To address the above issues and achieve optimal performance, neural networks need to be highly regularized. In our analysis, we apply five regularization techniques. Since neural networks are computationally very expensive, we use Bayesian optimization instead of grid search for all parameters that need to be tuned. First, the l_1-penalty is used to penalize the MSE loss function of the weight parameters. As described in Section 4.4.2, this reduces the complexity of the model by encouraging sparsity. We use $\lambda_1 \in \{10^{-5}, 10^{-4}, 10^{-3}\}$ to tune the penalization.

Next, the learning rate is regularized. As one of the most critical parameters that need to be tuned, the learning rate controls the step size of the descent (Gu et al., 2020). It is important to shrink the learning rate towards zero when the gradient approaches zero. Otherwise, noise could determine the direction of the signal when computing the gradient (Gu et al., 2020). We take care of this by using the Adam optimizer, which already takes shrinking the learning rate into account. In addition, we determine the learning rate from the validation sample and tune it among $LR \in \{0.001, 0.01\}$.

Third, overfitting is prevented by incorporating an early stopping rule. As explained earlier, neural networks are powerful and therefore tend to overfit the data at the global minimum. This should be avoided as it degrades the predictive performance on new, unseen data. During the optimization, the neural network updates its weights to minimize the loss function. The basic idea behind early stopping is to monitor the validation loss during training and stop the training process when the validation loss stops improving (Géron, 2019). Hence, if the validation loss has not improved after a certain number of epochs, training is stopped and the model with the best validation loss is selected (Gu et al., 2020). In this context, the number of epochs refers to the number of times the training dataset is passed through the neural network during training (Hastie et al., 2009). During each epoch, the dataset is divided into smaller subsets called batches, each containing the same number of observations. This number of observations in one batch is referred to as *batch size*. The batches are used to update the weights based on the error calculated on the outputs of each batch (Géron, 2019). Following Gu et al. (2020), we set the patience to 5. This means that the training is terminated if the validation loss has not improved after 5 consecutive epochs. In addition, we set the batch size to 10,000, while

the number of epochs is set to 100.

The fourth regularization technique is Batch Normalization, which aims to reduce the internal covariate shift when training the neural networks. Internal covariate shift refers to the change in the distribution of the inputs of each layer when the weight parameters of the previous layers change (Ioffe & Szegedy, 2015). Unfortunately, covariate shift slows down the training of the networks significantly. This is because neural networks converge much faster when the inputs are normalized. Also, covariate shift reduces the ability of the network to account for nonlinearities in the predictors (Ioffe & Szegedy, 2015). Batch normalization is a technique used to address these problems by normalizing the inputs to each layer of a neural network. In this way, the training process is stabilized. In our implementation, we add Batch Normalization layers between all hidden layers by using the *BatchNormalization* function from *TensorFlow*.

Finally, the dependence of the solution on randomly chosen starting weights is addressed. For this, several neural networks are first trained with different random seeds. The predictions of these individual networks are then averaged, reducing the impact of the starting weights on the final result. This technique is usually referred to as an *ensemble approach* (Hastie et al., 2009). Following the literature, we choose an ensemble of 10 models for our analysis (Gu et al., 2020). It can be said that the more ensemble models, the more robust the results, but due to limited computation resources we have to make a trade-off at 10 models.

The neural networks are implemented using the *TensorFlow* and *Keras* libraries in Python.

4.5. Comparative Performance Evaluation

This section deals with the assessment of the previous modeling approach. To find out which model makes the best predictions on an out-of-sample test set, we first present a measure that evaluates the individual predictive ability to forecast excess returns. Thereafter, we present a second measure to make pairwise comparisons of forecasts obtained from different models.

4.5.1. Predictive Out-Of-Sample R^2

To assess the predictive power of individual forecasts, we use the out-of-sample R^2 measure which is widely used in the return prediction literature (Campbell & Thompson, 2008; Welch & Goyal, 2008; Feng et al., 2018; Kelly et al., 2019; Gu et al., 2020). The out-of-sample R^2 evaluates the predicted excess returns relative to the realized excess returns. Following Gu et al. (2020), we calculate the out-of-sample R^2 as

$$R^2_{OOS} = 1 - \frac{\sum_{(i,t)\in\mathcal{T}_{Test}} (r_{i,t+1} - \hat{r}_{i,t+1})^2}{\sum_{(i,t)\in\mathcal{T}_{Test}} r^2_{i,t+1}}, \tag{4.39}$$

where $\hat{r}_{i,t+1}$ is the predicted excess return and $r_{i,t+1}$ is the realized or target excess return for stock i for the next month $t+1$. $\mathcal{T}_{Test}$ indicates that the observations used to calculate the out-of-sample R^2 are part of the testing window. The result is one out-of-sample R^2 measure for each model. Thus, we can make a grand panel-level assessment of each model as the out-of-sample R^2 pools prediction errors across stocks and over time (Gu et al., 2020). For ease of readability, we sometimes refer to the out-of-sample R^2 as "R^2" since we only report out-of-sample R^2 values and do not perform any in-sample tests.

It has to be noted that the out-of-sample R^2 as presented by Gu et al. (2020) and which we use is slightly different than the one used in many out-of-sample forecasting applications. Often the predicted returns are compared against historical mean returns while the measure of Gu et al. (2020) does not *demean* the excess returns. Gu et al. (2020) argue that it is more reasonable to assume a mean of zero when predicting individual excess returns instead of e.g. portfolio returns as the historical average is too noisy. Therefore, using *demeaned* excess returns would lower the bar for "good" forecast performance artificially (Gu et al., 2020). Hence, our out-of-sample R^2 is more conservative as we compare it against a forecast of zero.

4.5.2. Diebold-Mariano Test

Besides looking at the out-of-sample R^2 of the individual models, we also test whether one forecasting model performs significantly better than another. For this, we use the Diebold and Mariano (1995) test for differences in predictive accuracy of two competing forecasts. It has to be noted that the Diebold-Mariano test is not intended to compare econometric models but instead should be seen as a comparison of forecasts obtained from

different models due to its model-free nature (Diebold, 2015).

The Diebold-Mariano test requires forecast errors to have weak dependence (Diebold & Mariano, 1995). Gu et al. (2020) argue that this condition of weak error dependence may not hold for predicting individual stock returns because of potentially strong cross-sectional dependence. As proposed by Gu et al. (2020), we therefore adjust the test to a cross-sectional setting by comparing the cross-sectional average of the prediction errors from each model rather than comparing the errors among individual stock returns.

The Diebold-Mariano test statistic to evaluate the forecast of model (1) against the forecast of model (2) is defined as

$$DM_{12} = \frac{\bar{d}_{12}}{\hat{\sigma}_{\bar{d}_{12}}}, \tag{4.40}$$

where

$$d_{12,t+1} = \frac{1}{n_{Test,t+1}} \sum_{i=1}^{n_{Test,t+1}} ((\hat{e}^{(1)}_{i,t+1})^2 + (\hat{e}^{(2)}_{i,t+1})^2). \tag{4.41}$$

Here $\hat{e}^{(1)}_{i,t+1}$ and $\hat{e}^{(2)}_{i,t+1}$ represent the prediction error for stock i at time t obtained from models (1) and (2). We denote the number of stocks in the testing sample as $n_{Test,t+1}$. The mean and Newey-West standard error of $d_{12,t}$ over the testing sample are represented by $\bar{d}_{12}$ and $\hat{\sigma}_{\bar{d}_{12}}$, respectively. We use Newey-West standard errors to account for potential autocorrelation and heteroscedasticity in the errors (Newey & West, 1987).

Under the null hypothesis that there is no difference in the forecast accuracy between model (1) and (2), the Diebold-Mariano test statistic is normally distributed ($N(0,1)$) (Diebold & Mariano, 1995; Gu et al., 2020).

Evaluating the performance of different model forecasts with the Diebold-Mariano test and the out-of-sample R^2 measure provides us insights into how well a model is fitting the data and predicting stock returns. However, they do not tell us which variables were the most important for making those predictions. Thus, we will introduce a measure for the variable importance in the next section.

4.6. Variable Importance: SHAP Values

Understanding what happens inside a black-box model such as a neural network is not easy. Hence, it would be useful for many stakeholders to know what leads a model to

a certain return prediction. For investors but also for the developers of the models it is important to understand what the underlying drivers of a return forecast are, that is knowing which predictor performs well in predicting stock returns and which predictor does not. In the early days of machine learning, the focus was more on improving the predictive accuracy of the models without much consideration for how the model makes its predictions. But as machine learning and artificial intelligence become more and more integrated in many applications, there is a growing need for more transparency and interpretability of machine learning models. Hence, in recent years there has been a growing trend called *Explainable AI* that aims to address this. There have been major improvements and developments of new and powerful techniques to make complex models more understandable (Molnar, 2022).

Tree-based models such as XGBoost and random forest have the benefit of providing a variable importance measure directly. This is because they evaluate almost all the predictor variables when choosing the splitting variables in the training process. Relative variable importance is measured by the mean decrease in impurity, i.e. the improvement in the split-criterion accumulated over all trees (Hastie et al., 2009). However, since we want to compare the variable importance between all the different models we use, we must apply a measure that exists for all models.

Gu et al. (2020) base their relative importance measure on the reduction in the out-of-sample R^2 from setting all values of a variable j to zero while holding the other features constant. This is computationally unfeasible for us as this requires re-estimating all models J times, each with a different variable set to zero. Instead, we use SHapley Additive exPlanations (SHAP) values to explain our models. SHAP values were introduced by Lundberg and Lee (2017) and the idea behind them is to use the concept of Shapley values from cooperative game theory to allocate the contribution of each feature to the prediction in a fair and meaningful way (Lundberg & Lee, 2017). SHAP values are such a powerful tool because they combine six previous methods[1], come with improved computational performance, and tend to be more accurate and consistent than other methods (Lundberg & Lee, 2017).

For each feature j of a model's input, the SHAP method assigns a value ϕ_j indicating its contribution to the output (prediction) of the model. This is done by measuring how much the model's prediction changes when a specific feature is included in the input, compared

[1]SHAP values include other algorithms such as LIME, DeepLIFT, Layer-Wise Relevance Propagation, Shapley Regression Values, Shapley Sampling Values, and Quantitative Input Influence.

to when the feature is not included (Lundberg & Lee, 2017). Figure 9 shows an example of a prediction of one observation made by a model containing four input variables. Without having any features, we would simply predict the base value $E[f(z)]$. Conditioning only on variable x_1 leads to the prediction $E[f(z)|z_1 = x_1]$ and the corresponding contribution of ϕ_1 of variable x_1. Then, the next variable x_2 is incorporated into the model and shifts the prediction to $E[f(z)|z_{1,2} = x_{1,2}]$ with a SHAP value of ϕ_2. This is continued until all M variables are considered in the model which leads to the final prediction of the model denoted by $f(x)$. In this example variable x_1, x_2, and x_3 increase the prediction whereas variable x_4 decreases the prediction. It can also be seen that for this prediction, variables x_2 and x_4 have the highest importance. It has to be noted that when the model is nonlinear or the predictors are not independent, it matters in which order the features are added to the expectation (here only one possible order is displayed) (Lundberg & Lee, 2017). Therefore, the SHAP values are calculated by averaging the ϕ_j values across all possible orders.

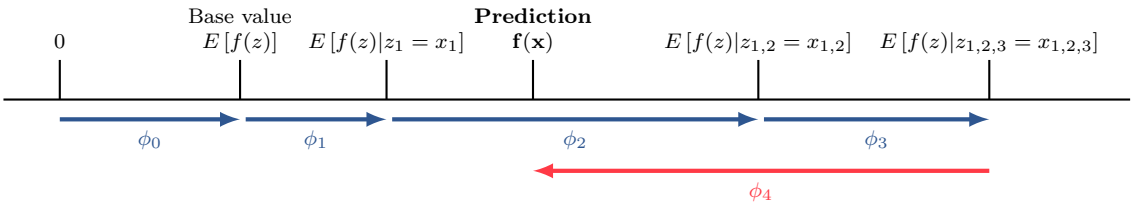


Figure 9: Illustration of SHAP Values
This figure shows an example of the features' SHAP values of a model. The SHAP values explain how to get from the base value $E[f(z)]$ to the model's prediction when a specific feature is considered. (Source: own creation based on Lundberg and Lee (2017)).

More formally, SHAP values use an explanation model g to specify the explanation of a machine learning model:

$$g(z') = \phi_0 + \sum_{j=1}^{M} \phi_j z'_j, \tag{4.42}$$

where $z' \in \{0, 1\}^M$ is the vector of simplified features and M is the number of simplified features. z' is 1 if a corresponding feature value is "present" and 0 if it is "absent". This conversion to a linear model of simplified features makes the computation of ϕ_j much more efficient (Lundberg & Lee, 2017; Molnar, 2022). Here it can also be seen that SHAP values are local methods that can explain a single prediction $f(x)$. If we define $\phi_0 = E[f(x)]$

and set all $x'_j = 1$, Equation (4.42) becomes

$$f(x) = g(x') = E\left[f(x)\right] + \sum_{j=1}^{M} \phi_j. \tag{4.43}$$

The SHAP values ϕ_j are calculated as

$$\phi_j(f, x) = \sum_{z' \subseteq x'} \frac{|z'|!(M - |z'| - 1)!}{M!} \left[f_x(z') - f_x(z' \setminus j)\right], \tag{4.44}$$

where $|z'|$ is the number of non-zero entries in z', and the notation $z' \subseteq x'$ denotes all vectors z' where the non-zero entries form a subset of the non-zero entries in x' (Lundberg & Lee, 2017).

We implement the calculation of the SHAP values using the Python library *SHAP* from Lundberg and Lee (2017). Since we receive one SHAP value for each observation (i.e. for each stock per month), we report a feature's importance as the mean SHAP value across N predictions:

$$\overline{\phi} = \frac{1}{N} \sum_{j=1}^{N} \phi_j(f, x). \tag{4.45}$$

Furthermore, we construct the *relative importance* of each variable by normalizing the mean SHAP values within a model to sum to one following Gu et al. (2020). Since the importance of the predictor variables might change over time and reporting SHAP values from a single year only would be an arbitrary choice, we report SHAP values as averages over the whole out-of-sample testing period from 1987 to 2021 for all models. Moreover, for neural networks, the mean SHAP values do not contain all predictions but only a subset of 1,000 observations as it is computationally infeasible to use all observations. However, Lundberg and Lee (2017) show that already a sample of 1,000 observations leads to very good approximations of a feature's contribution. For linear models and tree-based models, the SHAP library uses faster approximation methods that make use of the specific model structure, and hence, we can use all observations for these models (Lundberg et al., 2020). We do not calculate SHAP values for the PLS and PCR models as they do not use the original variables as regressors but a set of constructed components as regressors.

4.7. Investment Strategy

After obtaining the monthly return prediction for all the stocks in our sample, we make use of these predictions and build zero-net investment machine learning portfolios. The objective is to test whether investors can potentially generate higher returns when incorporating machine learning techniques into their investment process. This section is divided into two parts and starts by introducing our backtesting approach. That is how we build the portfolios and invest in the specific stocks. Next, we give an overview of the performance metrics we use to evaluate the machine learning portfolios.

4.7.1. Backtesting Approach

After obtaining the predicted returns from the various machine learning models described in Section 4.4, the next step is to build portfolios using these predictions. The investment strategy employed in this study follows the one from Gu et al. (2020) and involves sorting the predicted returns each month and constructing long-short portfolios based on the sorted results.

Specifically, for each model, the predicted returns are sorted into ten deciles, with the decile containing the highest predicted returns representing the long position (decile 10), and the decile with the lowest predicted returns representing the short position (decile 1). Put differently, we go long the stocks with the highest predicted excess returns and short the stocks with the lowest predicted excess returns. The zero net-investment portfolios are rebalanced monthly, thus we hold each position for one month.

Regarding the weighting scheme of the investment strategy, we generate both equal- and value-weighted portfolios. For value-weighted portfolios, we weight the corresponding stocks by their relative market capitalization of the previous month to avoid having a look-ahead bias. Value-weighted portfolios are less sensitive to trading costs and illiquidity as the weights of smaller stocks are lower. Thus, our main focus is on value-weighted portfolios. However, the statistical objective functions of our models minimize equal-weighted forecast errors which might make it more natural to study equal-weighted portfolios (Gu et al., 2020). To receive more robust results, we perform the analysis of machine learning portfolios with both weighting schemes.

4.7.2. Performance Metrics and Factor Analysis

To assess our long-short portfolios we report several performance measures such as average return and standard deviation, Sharpe ratio, and maximum drawdown. Furthermore, we run factor regressions to determine whether the machine learning portfolios can achieve a positive alpha and whether the excess returns of the portfolios are due to their exposure to systematic risk factors or their own unique characteristics (Pedersen, 2015).

Following Gu et al. (2020), we use cumulative log returns in our portfolio plots and for the calculation of drawdowns. This is because log returns provide benefits for visualization and are more efficient when computing return metrics due to their additive nature. Since they are more linear it makes it easier to compare the performance of the different portfolios over time.

The Sharpe ratio is a measure of risk-adjusted returns and is a useful tool for evaluating the performance of investment strategies because it provides a measure of the excess return generated per unit of risk taken (Pedersen, 2015). The Sharpe ratio is calculated as

$$SR_p = \frac{E\left[r_p\right] - r_f}{\sigma_p}, \tag{4.46}$$

where $E(r_p)$ is the expected return and σ_p is the standard deviation of a portfolio.

Another important risk measure of a strategy is its drawdown. The drawdown measures how much a strategy has lost since its peak. Large drawdowns are risky and costly not only in terms of economic loss but also in terms of redemptions and concerns from investors. In addition, large drawdowns can lead to higher margin requirements (Pedersen, 2015). We calculate the drawdown DD_t in period t as

$$DD_t = HWM_t - Y_t, \tag{4.47}$$

where Y_t is the cumulative log return from month 0 to month t and HWM_t is the high water mark, i.e. the highest cumulative log return the portfolio has achieved in the past (Pedersen, 2015). The high water mark is calculated as $HWM_t = \max_{0 \leq s \leq t} Y_s$ (Pedersen, 2015). Often it is interesting to report the largest loss an investor would have incurred if they had invested at the worst possible time and held the investment until recovery. Thus, we report the maximum drawdown $MaxDD_T$ of a strategy over the given investment

horizon from 1987 to 2021 which is calculated as

$$MaxDD_T = \max_{0 \leq t \leq T} DD_t. \tag{4.48}$$

To extend the performance evaluation with another downside risk measure, we also report the most extreme negative return that the portfolios incur in a month and refer to it as the maximum one-month loss $Max1MLoss_T$. The maximum one-month loss is given as

$$Max1MLoss_T = -\min_{0 \leq t \leq T} r_{p,t}. \tag{4.49}$$

When reading about portfolios based on machine learning models, one often hears that these portfolios require a lot of trades which leads to high transaction costs (Drobetz & Otto, 2021). Hence, to get a sense of how many trades are made, we also report a measure for the turnover of a strategy. Following the method of Koijen et al. (2018), we compute the turnover for month t as

$$Turnover_t = \frac{1}{2} \sum_i |w_{i,t-1}(1 + r_{i,t}) - w_{i,t}|, \tag{4.50}$$

where $w_{i,t}$ is the weight of stock i in the portfolio in month t. We divide by 2 to avoid double-counting for buying and selling the same stock. We report the average monthly turnover over the whole out-of-sample testing period for each strategy.

Next, we perform factor regressions to examine whether the long-short strategies yield an excess return (alpha) above and beyond different risk exposures measured by common risk factors described below. In addition, this sheds light on the sources of risk and return of the long-short strategies. We do this for each of the long-short portfolios obtained from the different machine learning models. Here, the monthly excess return of a portfolio is the dependent variable and the excess returns on the risk factors are the independent variables. Following Gu et al. (2020), we use the Fama and French (2015) five-factor model and extend it with the momentum factor from Carhart (1997). More formally, we run the following OLS regression

$$\begin{aligned} r_{p,t} = \alpha + \beta_{MKT} MKT_t + \beta_{SMB} SMB_t + \beta_{HML} HML_t + \\ \beta_{RMW} RMW_t + \beta_{CMA} CMA_t + \beta_{MOM} MOM_t + \epsilon_t, \end{aligned} \tag{4.51}$$

where $r_{p,t}$ is the excess return of portfolio p at time t, MKT_t are the excess returns of the

market, SMB_t are the returns of a portfolio of small stocks minus the returns of a portfolio of big stocks, HML_t is the difference between the returns on a portfolio of high book-to-market (value) stocks and the returns on a portfolio of low book-to-market (growth) stocks, RMW_t is the difference between the returns on portfolios of stocks with robust (high) and weak (low) profitability, CMA_t are the returns of a portfolio of low-investment (conservative) stocks minus the returns of a portfolio of high-investment (aggressive) stocks, MOM_t is the difference of returns on portfolios of stocks with high and low momentum, and ϵ_t is the zero-mean residual (Carhart, 1997; Fama & French, 2015). As is common in the asset pricing literature, we adjust the standard errors for potential autocorrelation and heteroskedasticity by using Newey and West (1987) standard errors (Scheuch et al., 2023).

5. Empirical Results

Having established the methodology, this section will present the empirical results of our analysis. We will highlight the main findings and offer solutions to the research questions posed in Section 1. To provide a comprehensive overview of the analysis, we divide the results section into three subsections. The first subsection analyzes the performance of the models, assessing their ability to make useful out-of-sample predictions. The second subsection examines the SHAP values, investigating the importance of each predictor variable for the models' predictions. The final subsection evaluates the performance of the machine learning portfolios by comparing different performance metrics and conducting spanning regressions with common risk factors.

5.1. Performance of Model Forecasts

To assess the predictive performance of each forecast, we report the out-of-sample R^2 for all eleven models in Table 4. The results provide several important insights into the predictive power of the models. First, it can be seen that the out-of-sample R^2 for the multiple linear regression model using all predictors is strongly negative, consistent with our expectations. As explained in Section 4.4.1, including too many variables in the multiple linear model is not beneficial because the variance of the OLS estimators consequently increases, making them less precise. This is also noted by Gu et al. (2020),

who state that a simple naive prediction of zero for all stocks and months performs better than the multiple linear model with all predictors.

Table 4: Monthly Out-Of-Sample Predictive R^2 Performance

	OLS-all	OLS-3	ENet	PCR	PLS	RF	XGB	NN1	NN2	NN3	NN4
R^2_{OOS} (%)	< -100	0.17	0.16	0.24	0.18	0.15	0.19	0.21	0.21	0.25	0.26

Notes: This table reports the out-of-sample predictive R^2 for OLS using all variables (OLS-all), OLS using only three variables: size, b/m, and momentum (OLS-3), elastic net (ENet), principal component regression (PCR), partial least squares (PLS), random forest (RF), XGBoost (XGB), and the neural networks with 1 to 4 hidden layers (NN1–NN4). All values are reported in percent. (Source: based on own analysis).

The R^2 values for the other models range from 0.15% to 0.26%, with the random forest having the lowest and the neural network with four hidden layers (NN4) having the highest predictive R^2. The R^2 of 0.17% for the benchmark model (OLS-3) is at the lower bound of the range. While this value improves only slightly to 0.18% for the PLS regression, the PCR model shows an R^2 of 0.24%, indicating that it is the better dimension reduction method. The higher R^2 of the PCR model suggests that predictors are partially redundant and noisy signals. Reducing the dimension of the data to a smaller set of components filters out noise and helps to better capture the signals (Gu et al., 2020). The results for tree-based methods are not consistent. On the one hand, the random forest model with an R^2 of 0.15% performs worse than the benchmark model. On the other hand, the XGBoost model beats the OLS-3 model with an R^2 of 0.19%. This is not what we expected, given that other studies find that both tree-based methods perform much better than the linear benchmark model. The highest R^2s are observed for the neural networks with three and four hidden layers, which is also observed in the studies of Gu et al. (2020), Drobetz and Otto (2021), and Leippold et al. (2022). This suggests that incorporating complex nonlinear interactions between features helps to improve the predictive accuracy of the models.

A notable feature of the out-of-sample R^2 values reported in Table 4 is that the R^2s of the best-performing models are lower than those reported in previous studies. While the R^2 for our NN4 model is 0.26%, Gu et al. (2020) report an R^2 of 0.40% for their best-performing model, which is a neural network with three hidden layers. On the one hand, low predictive R^2 values are not a problem, as they can still be economically meaningful (Campbell & Thompson, 2008). On the other hand, it is questionable why the

R^2s of tree-based methods and neural networks are significantly higher in the study by Gu et al. (2020) than in ours, since the results for the benchmark model are almost the same. We defer this discussion to Section 6 for now.

Comparing the performance of the four neural networks, we can see that the R^2 increases with the number of hidden layers. With this in mind, it would have been interesting to investigate the optimal number of hidden layers. However, the required computational capacity increases significantly with additional layers, making this unfeasible for us. Also, as discussed in Section 4.4.5 and supported by other studies, increasing the number of layers does not always increase performance. This is especially true when the amount of data is limited. For example, Gu et al. (2020) report a decrease in R^2 after adding more than three hidden layers, suggesting that the inclusion of more than four layers in our study may not have led to better performance.

In addition to evaluating the out-of-sample R^2 of each model, we assess whether the predictions of one model show a statistically significant improvement over the predictions of another model. Table 5 reports the Diebold-Mariano test statistic for each pair of predictions. Significance at the 5% confidence level is denoted by bold numbers. As the test statistic follows a normal distribution, the critical value is 1.96. If the test statistic is positive, the forecasts of the column model outperform the forecasts of the row model. If the test statistic is negative, the opposite is given.

Table 5: Diebold-Mariano Test Results - Comparison of Predictive Performance

	OLS-3	ENet	PCR	PLS	RF	XGB	NN1	NN2	NN3	NN4
OLS-all	**2.53**	**2.43**	**2.73**	**2.69**	**2.46**	**2.63**	**2.67**	**2.66**	**2.75**	**2.76**
OLS-3		-0.16	**1.96**	0.01	-0.03	0.69	0.69	0.76	**1.98**	**2.01**
ENet			1.16	0.30	0.03	0.53	0.56	0.62	1.90	1.93
PCR				-1.85	-0.33	-0.09	-0.48	-0.49	0.09	0.12
PLS					-0.03	0.39	0.38	1.04	1.13	1.03
RF						0.83	0.76	0.80	1.09	1.12
XGB							-0.06	-0.05	0.12	0.13
NN1								0.05	0.35	0.33
NN2									0.54	0.43
NN3										0.10

Notes: This table reports the results of the Diebold-Mariano test. The Diebold-Mariano test compares the forecasts of two models pairwise. A positive number indicates that the column model outperforms the row model. Significance at the 5%-level is indicated with bold font. (Source: based on own analysis).

The numbers in the first row of Table 5 show that all models perform significantly better than the OLS-all model. This was expected since the predictive R^2 of the OLS-all model is highly negative. Next, we can see that the PCR model and the NN3 and NN4 models significantly outperform the OLS-3 benchmark model with test statistics of 1.96, 1.98, and 2.01, respectively. Besides that, there are no other significant results. The fact that the random forest and the elastic net are not significantly outperformed by any of the other models is surprising since the predictions of these models have the lowest predictive R^2.

A final observation is that the neural network with four hidden layers tends to perform better than all other models, as indicated by the fact that the test statistic in the NN4 column is positive for all rows. For example, the test statistic of 1.93 indicates that the NN4 model performs better than the elastic net, although this result is not significant at the 5%-level.

Other studies have also used the Diebold-Mariano test to compare the predictive accuracy of different forecasting models and report results that are largely consistent with ours. Gu et al. (2020) find that their best models significantly outperform the OLS-all, OLS-3, and elastic net, while their test statistics are slightly higher than ours. This could be because the differences in R^2s between their models are larger than between our models. In contrast, the study by Drobetz and Otto (2021) finds that only the OLS-all and the simpler OLS-benchmark model are significantly outperformed by other models. This is in line with what we find in our analysis.

To answer the question of which machine learning models perform best in predicting stock returns out-of-sample, the results of this section can be divided into two parts. The first part of the analysis has shown that all models except the random forest and elastic net outperform the linear benchmark model in terms of predictive out-of-sample R^2. In this context, the best methods are PCR and neural networks. The success of PCR can most likely be attributed to its ability to construct a simpler model that focuses on the most important predictors and reduces the risk of overfitting. On the other hand, the results show that there is value in including complex predictor interactions that are captured in neural network models but ignored by other methods. This is also concluded by Gu et al. (2020), Drobetz and Otto (2021), and Leippold et al. (2022).

However, this section also finds that the outperformance of the benchmark model is not statistically significant, except for the PCR, NN3, and NN4 models. This raises the question of whether the results of the other models can still be useful. We address this

question when analyzing the performance of the machine learning portfolios in Section 5.3 and in the discussion in Section 6.

In the following sections, we will no longer consider the multiple linear model that uses all predictors (OLS-all), as it is presented only to demonstrate its limited practical value. Also, to give a more concise overview, we will focus on the best-performing neural network (NN4) instead of listing the results for all four networks.

5.2. Variable Importance

Having found the model with the best predictive power, we move on to research question two and analyze the SHAP values to gain a better understanding of the relative importance of the independent variables for the performance of each model. Figure 10 shows the SHAP values for the firm characteristics of each model. Each chart shows the 20 most important firm characteristics in descending order, with the most important characteristic first. As explained in Section 4.6, the average SHAP values within a given model are normalized so that they sum to one. This allows for better comparability of the predictors within a model as well as the effect of a predictor in different models. As mentioned previously, the SHAP values are not reported for the PCR and PLS models because both use a set of constructed components instead of the original variables as inputs to the models.

When analyzing Figure 10, several dominant groups of variables can be identified across the models. First, it can be seen that price trends have a large impact on the predictions and are among the best-performing characteristics in all models. Important price trend variables include the 1-month momentum (mom1m), the 12-month momentum (mom12m), and the industry momentum (indmom). Liquidity variables, including stock turnover (turn) and volatility of stock turnover (std_hturn), log market equity (mvel1), dollar trading volume (dolvol), illiquidity (ill), and the number of zero trading days (zerotrade) are almost equally important factors as price trends. A third dominant group is investment and growth variables, including the increase in research and development (R&D) expenses (rd), the ratio of R&D expenses to market capitalization (rd_mve), and asset growth (agr). These features are particularly dominant in the tree-based models. Leippold et al. (2022) argue that the reason for the dissimilarity of results between tree models and alternative models may lie in the general properties of decision trees, as they randomly select a subset of firm characteristics when developing the model. As a result, certain predictors may be important in certain trees, while their relevance is lower in other models.

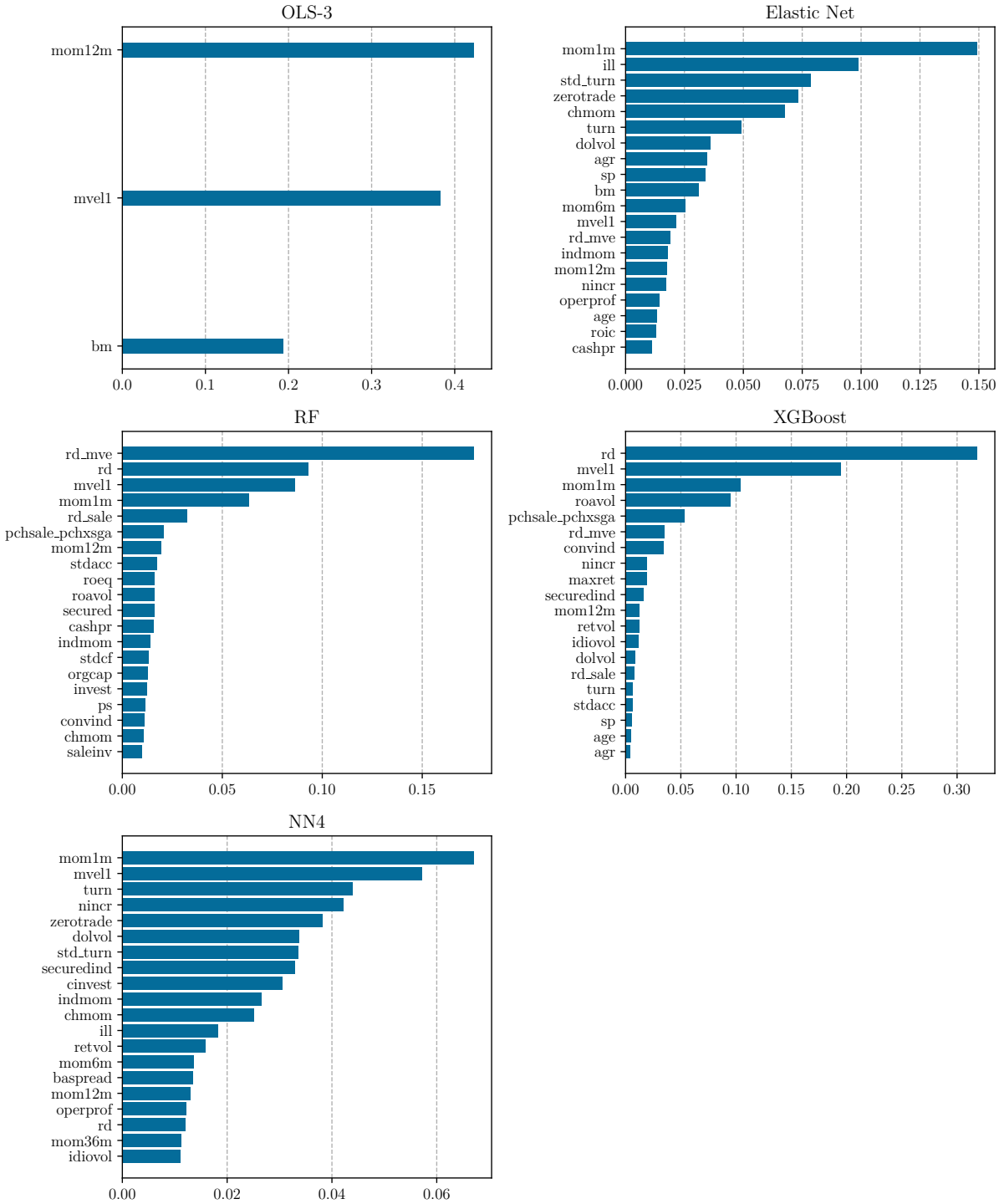


Figure 10: Variable Importance (SHAP) for Firm Characteristics (Top 20) by Model
This figure shows the variable importance in terms of mean SHAP values for the 20 most important variables of each model. For the interpretation of relative variable importance, SHAP values within each model are normalized to sum to one. (Source: own figure based on analysis).

Finally, there are three other groups of important characteristics that are less dominant than those previously mentioned. The first group is financial ratios, including the book-to-market ratio (bm) and the sales-to-price ratio (sp). The second group is profitability measures, with return on equity (roe) and return on invested capital (roic) being the most prominent. Last but not least, risk measures such as return volatility (retvol) or idiosyncratic volatility (idiovol) play an important role in the models' forecasts. Surprisingly, the industry dummies do not seem to play a major role in the forecasts.

When comparing the plots of the five different models, it is noticeable that for the elastic net, the importance of the predictors is skewed towards a small group of characteristics. This is not surprising, since the elastic net reduces the number of variables needed to explain the data by its regularization features (variable selection and shrinkage). A similar skewness is observed for the random forest and the XGBoost model. Instead, the neural network appears to draw its predictive power from a wider set of independent variables.

Most of our results are consistent with findings from other studies. Gu et al. (2020) also find that price trends and liquidity variables are the most important characteristics, followed by risk measures and valuation ratios. The results of their elastic net and tree-based methods are even more skewed than ours. For the return prediction of the European stock market, Drobetz and Otto (2021) find that valuation ratios such as the earnings-price ratio as well as price trends have the most influence. In addition, their analysis shows that tree-based models concentrate their attention on only a few variables, but dimension-reduction methods incorporate a more extensive set of predictors into their analyses. Finally, Leippold et al. (2022) find that measures of market liquidity are most important for predicting the Chinese stock market. This is followed by fundamental signals and valuation ratios, as well as risk measures. In contrast to our results, those of Gu et al. (2020), and Drobetz and Otto (2021), price trends do not turn out to be important predictors for the Chinese stock market.

To obtain an even more comprehensive overview of the importance of the variables, we proceed with analyzing the impact of the four macroeconomic factors. For this purpose, the SHAP values of the 94 interaction terms with each of the four macroeconomic factors are averaged. As with the firm characteristics, the average SHAP values within a given model are then normalized to sum to one. The results for all models can be seen in Figure 11. As the bar chart shows, the importance of the macroeconomic factors is mostly evenly distributed across the factors, with slight variations for each model. While the default spread (dfy) is most important for the random forest, the term spread (tms) has

the greatest influence in the XGBoost model. Instead, the neural network shows a more uneven distribution of factors, with the treasury bill rate (tbl) being the most important macroeconomic signal. Finally, the effects of the factors in the elastic net are almost uniformly distributed. Thus, we conclude that none of the four macroeconomic factors is redundant.

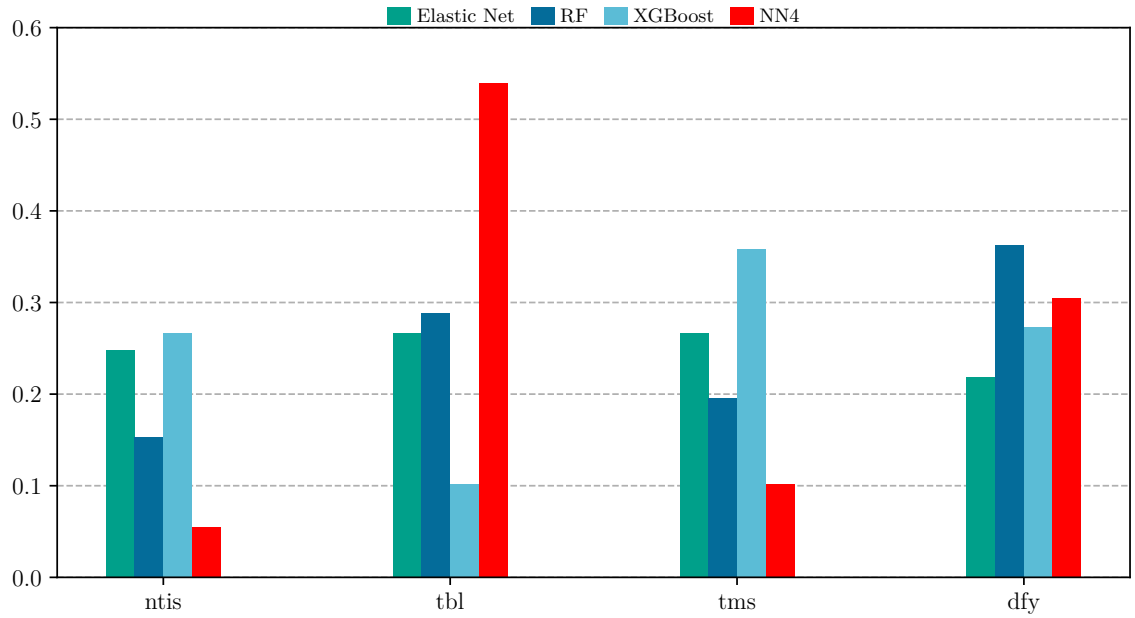


Figure 11: Variable Importance (SHAP) for Macroeconomic Factors
This figure shows the variable importance in terms of mean SHAP values for the four macroeconomic factors of each model. For the interpretation of relative variable importance, SHAP values within each model are normalized to sum to one. The SHAP value for a macroeconomic factor is calculated as the average of all variable interactions with a specific macroeconomic factor. (Source: own figure based on analysis).

Gu et al. (2020) also report on the importance of macroeconomic factors for their predictions. Since they included four more factors than we did, it is difficult to compare the results. However, we can say that none of the factors we included seems to be unimportant in their study.

To sum up, our examination of the variable importance helps us to identify the firm characteristics and macroeconomic factors that drive the models' performance and yields two major findings. First, the most important firm characteristics are broadly consistent across models and include price trends and liquidity variables. Moreover, investment and growth indicators dominate, especially in tree-based methods. When looking at macroeconomic factors, importance is evenly distributed across factors, with slight variations across

the models.

5.3. Performance of Machine Learning Portfolios

Finally, in this section, we present the results of the investment strategy that uses machine learning portfolios to answer our last research question. As described in Section 4.7.1, our goal is to evaluate whether investors can improve their investment performance by incorporating machine learning techniques. For this, we sort the next month's return predictions and construct zero-net-investment portfolios with long and short positions based on the ranked predictions. We often refer to these machine learning-based portfolios as *machine learning portfolios*. Table 6 shows the results of this strategy for the out-of-sample period from 1987 to 2021, where Panel A presents the results for value-weighted and Panel B for equal-weighted portfolios.

In Panel A, the annualized average realized returns (Mean) range from 9.8% to 12.5%. It stands out that the strategy that uses the neural network predictions to form the portfolios has the highest mean return. The annualized standard deviation (Std) ranges from 16.3% to 18.6%, with the most volatile results for the elastic net and the random forest. Looking at the risk-adjusted performance in terms of Sharpe ratio (SR), it can be seen that the annualized Sharpe ratio of the benchmark model is 0.64, which is exceeded by all models except for PCR and PLS. The highest Sharpe ratio of 0.73 is achieved by the neural network-based strategy which is more than 15% higher than the Sharpe ratio of the benchmark model.

To get an understanding of the downside risk of the different strategies, the maximum drawdown (Max DD) and the most extreme negative monthly return (Max 1M Loss) are analyzed. The maximum drawdown varies quite a lot, with a minimum of 34.6% for the XGBoost model and a maximum of 73.5% for the PCR model. Compared to most of the other models, OLS-3 experiences a relatively high maximum drawdown of 60.4% together with a worst one-month performance of −15.4%. In comparison, the strategy using the NN4 model has a maximum drawdown of 48.4% and a maximum one-month loss of 12.1%. Thus, the NN4 model outperforms the benchmark model in both risk measures. Another model that performs well on these two measures is the XGBoost model, while the performance of the PCR and PLS models is comparatively poor.

The strong performance of the NN4 model comes along with an average monthly turnover of 110.0%. Other machine learning models also exhibit high average monthly turnovers,

Table 6: Long-Short Portfolio Performance

	OLS-3	Elastic Net	PCR	PLS	RF	XGBoost	NN4
Panel A: Value-Weighted Portfolios							
Mean (%)	11.07	11.89	10.48	9.82	11.90	11.47	12.45
Std (%)	17.42	18.57	17.48	16.88	18.57	16.25	17.04
SR	0.64	0.64	0.60	0.58	0.64	0.71	0.73
Max DD (%)	60.37	46.09	73.45	68.60	43.93	34.58	48.36
Max 1M Loss (%)	15.39	29.61	23.78	23.92	18.85	17.26	12.09
Turnover (%)	55.92	156.55	115.07	119.90	122.35	140.67	110.02
Panel B: Equal-Weighted Portfolios							
Mean (%)	14.59	26.96	24.31	25.55	30.55	29.70	27.26
Std (%)	17.55	19.64	17.52	18.63	19.86	19.71	18.95
SR	0.83	1.37	1.39	1.37	1.54	1.51	1.44
Max DD (%)	59.10	101.79	54.35	122.86	23.60	35.30	25.04
Max 1M Loss (%)	26.53	55.48	16.57	60.19	10.94	12.72	10.51
Turnover (%)	52.04	144.37	105.64	109.64	119.27	151.41	137.87

Notes: This table reports the performance of the machine learning-based long-short strategy over the 35-year out-of-sample testing period from 1987 to 2021. The predictions of each model for the next month are sorted into deciles and we go long the stocks with the highest excess returns and short the stocks with the lowest excess returns. We report the annualized average realized returns (Mean), annualized standard deviation (Std), annualized Sharpe ratio (SR), maximum drawdown (Max DD), the most extreme negative monthly return (Max 1M Loss), and the average monthly turnover for all value-weighted (Panel A) and equal-weighted (Panel B) portfolios. (Source: based on own analysis).

with the turnover being larger than 100% for all models. The only model that does not have such a high turnover is the OLS-3 model with a turnover of 55.9%. However, this is also stimulated by the fact that there are only three factors in the OLS-3 model. Thus, by construction, there are not many factors from which the OLS-3 model can learn. In contrast, machine learning algorithms have the ability to identify and thus respond to subtler signals in the data. This could be one reason why changes in the holdings of a neural network-based portfolio are more frequent and significant.

Panel B shows slightly different results for the equal-weighted portfolios. While the qualitative results are in most cases consistent with those from panel A, they differ in terms of their magnitude. This is best seen by looking at annualized mean returns, which have a maximum of 30.6% compared to 12.5% in panel A. Since the standard deviations do not change too much, the Sharpe ratios obtained in Panel B are also considerably higher than in Panel A. The benchmark model has a Sharpe ratio of 0.83, while the ratios of the other

models range from 1.37 to 1.54. The highest Sharpe ratios are observed for the tree-based methods, followed by the neural network. Regarding the downside risk of equal-weighted portfolios, it can be seen that the maximum drawdowns increase substantially for the elastic net and PLS models. The same is true for the most extreme negative monthly returns. For the other models, these measures do not differ too much from the numbers in Panel A, except that the maximum drawdown for the neural network has almost halved, indicating lower downside risk. Similarly, as in Panel A, the monthly turnover is above 100% for all models except the benchmark model, which has a much lower turnover.

The higher Sharpe ratios for equal-weighted portfolios indicate that the higher performance of equal-weighted portfolios is driven by very small stocks that have historically had higher average returns than larger stocks. To analyze the influence of micro-cap stocks on our findings, we repeat the performance analysis but exclude stocks that are below the 20th percentile of the NYSE breakpoints for market equity. We report the results excluding micro-cap stocks in Table 10 in Appendix C. For value-weighted portfolios, the metrics for the OLS-3, elastic net, PCR, and PLS models do not change significantly. However, the portfolios based on random forests, XGBoost, and NN4 earn larger Sharpe ratios which are mainly due to the lower volatility of the portfolios. In addition, the turnover rates can be lowered substantially. For instance, excluding micro-cap stocks leads to a Sharpe ratio of 0.85 and a turnover of 99.5% for the NN4-based long-short strategy. The exclusion of micro-cap stocks has a larger effect on the results of equal-weighted portfolios. In this case, the Sharpe ratio of the NN4-based strategy decreases to 1.13 together with a turnover of 131.8%.

Relating these findings to the results in Section 5.1, we see that models that have a high predictive R^2 in Table 4 do not necessarily perform well in our chosen investment strategy. For example, the PCR model has a comparatively high R^2, while it does not have a higher Sharpe ratio than the benchmark model in the long-short strategy. This might be because our investment strategies evaluate the performance on a portfolio level while the forecasts are made on the individual stock level. Thus, there is no guarantee that a model which performs well in predicting individual stock returns also produces accurate portfolio-level forecasts as the distribution of portfolio returns is sensitive to the dependence among stock returns (Gu et al., 2020). Still, the neural network is the best model in terms of predictive out-of-sample R^2, while it is also the best model in the investment strategy.

Finally, we illustrate the results of Table 6 in Figure 12. Each of the lines represents the cumulative log returns of the value-weighted long-short strategies from 1987 to 2021.

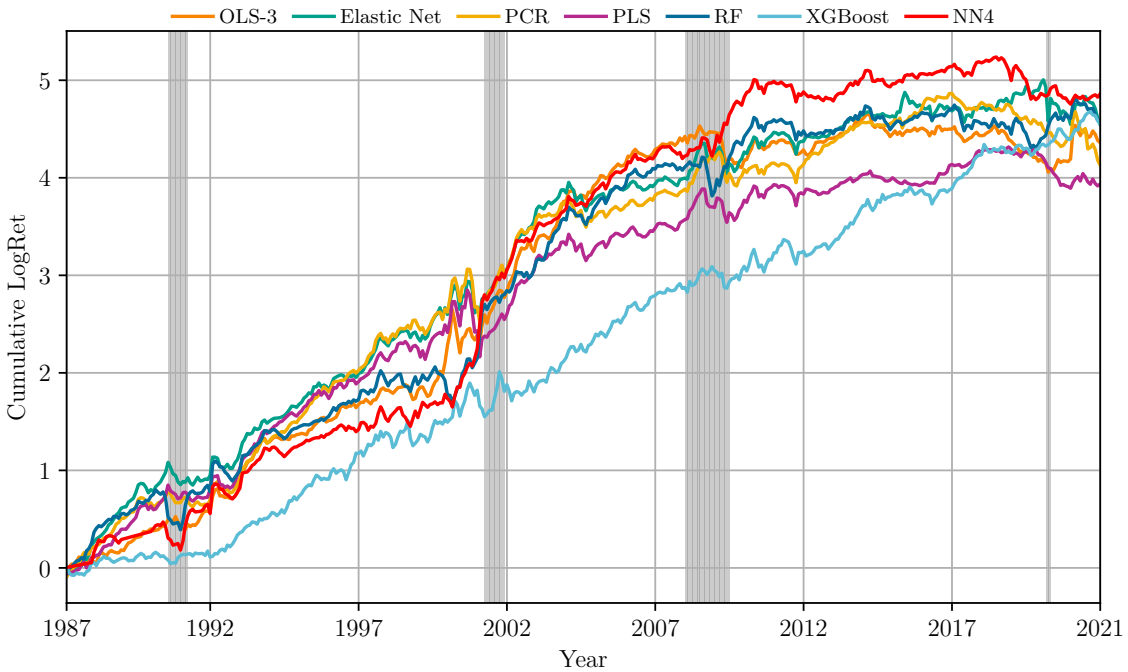


Figure 12: Cumulative Log Returns of Machine Learning Portfolios
This figure shows the cumulative log returns of the long-short portfolios based on the predictions of each model. The portfolios are long the stocks with the highest out-of-sample return forecasts and short the stocks with the lowest return forecasts. All portfolios are value-weighted. The grey-shaded areas mark NBER recession periods in the U.S. (Source: data from own analysis, recession indicators from FRED (n.d.)).

When analyzing the graph, two things stand out. First, the neural network has the highest cumulative log return over the entire period, while the PLS model has the lowest. This is consistent with the results from Panel A in Table 6, where the neural network has the highest mean and Sharpe ratio, while the PLS model performs worst on both measures. Second, the cumulative log returns of the different models vary widely over time, while the lines become closer again toward the end of the period. The clearest example of this is the plot of the cumulative log return of the XGBoost model. The line is below all other lines for most of the time, while between 2017 and 2021 it rises sharply relative to the other models. Surprisingly, the graph indicates that the performance of the machine learning portfolios during the most recent years has slightly decreased. This is in line with a newer version of the study by Gu et al. (2020). Gu (2021) finds that when extending the initial analysis with updated data until 2020, the margin between machine learning-based strategies becomes substantially weaker after 2016.

Recapitulating the results in this section, we conclude that machine learning models are

able to outperform a linear benchmark model in a long-short investment strategy. In this context, the neural network-based strategy has the highest Sharpe ratio when investing based on value-weighted portfolios. However, building portfolios based on neutral networks requires almost twice the monthly trading effort compared to a strategy that uses the benchmark model. For equal-weighted portfolios, the tree-based models and the neural network have the highest Sharpe ratios, with mean and Sharpe ratios considerably higher than for the value-weighted portfolios.

Ensemble Strategy

The previous sections showed that the excess return predictions of the different machine learning models can vary quite a lot among the models. As we have seen with the tree-based models and neural networks, creating an ensemble of different models can improve the accuracy and the robustness of the predictions tremendously compared to using a single model (Hastie et al., 2009; Géron, 2019; Gu et al., 2020). Also, as seen in Figure 12 in the previous section, the cumulative log returns of the individual strategies seem to vary considerably among the models. By using an ensemble approach we can leverage the individual strengths and mitigate the weaknesses of the different models. The same concept can be applied to the long-short portfolio formation as well. We construct a long-short portfolio using the average of the models' predictions, similar to the approach used for the individual models. This involves taking a long position in stocks that are predicted to have high excess returns and a short position in stocks that are predicted to have low excess returns. By relying on the ensemble predictions of all models we can reduce the risk of relying on a single model's predictions and improve the overall performance. Thereby, greater fluctuations in the returns are smoothed out and a more consistent and stable performance over time can be achieved. Our ensemble comprises the full set of models that were included in our comparison, which consists of OLS-3, elastic net, PCR, PLS, random forest, XGBoost, and NN4.

The ensemble strategy achieves a predictive out-of-sample R^2 of 0.44% per month over the whole testing period which is significantly higher than the out-of-sample R^2 of the best individual model of 0.26% that the NN4 model achieves. Thus, we can almost double the out-of-sample predictive R^2 by combining the forecasts of the individual models.

Apart from the predictive R^2, the performance of the ensemble strategy is also superior to the portfolios based on the predictions of the individual models. The value-weighted

long-short ensemble strategy generates an annualized Sharpe ratio of 0.81 with an annualized mean return of 14.8% and annualized standard deviation of 18.4%. The superior performance in terms of the Sharpe ratio comes with a slightly higher downside risk. The maximum drawdown of 52.1% and the maximum one-month loss of 19.6% are slightly higher than the metrics of the neural networks. However, they are still below the benchmark model OLS-3. Furthermore, the ensemble strategy also has a higher turnover of 135.7% than the neural network which will decrease the performance of the strategy when evaluating the performance net of transaction costs. The results of the equal-weighted portfolio are similar. With an annualized Sharpe ratio of 1.91, the ensemble portfolio also beats the best model in an equal-weighted setting. The results are robust to excluding micro-cap stocks from the investment universe (see Table 10 in Appendix C).

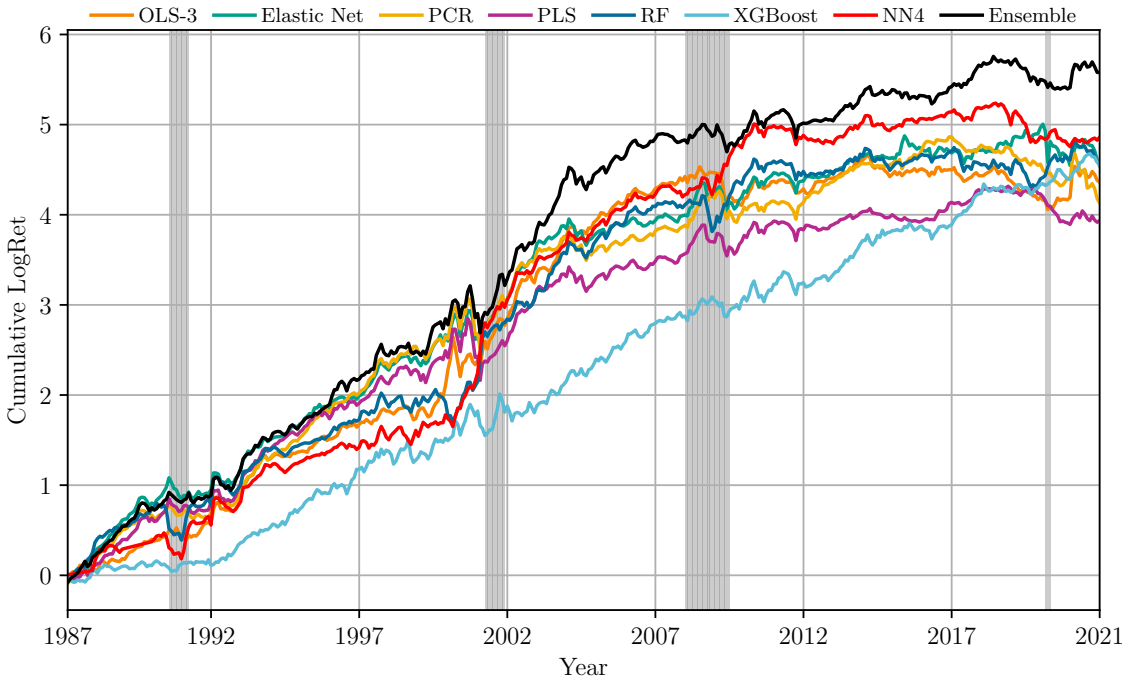


Figure 13: Cumulative Log Returns of Ensemble Strategy

This figure shows the cumulative log returns of the ensemble strategy as well as the machine learning portfolios described earlier. The ensemble strategy combines the forecasts of all models and forms a long-short portfolio based on the average predictions of the individual model forecasts. All portfolios are value-weighted. The grey-shaded areas mark NBER recession periods in the U.S. (Source: data from own analysis, recession indicators from FRED (n.d.)).

The graph of the cumulative log returns of the ensemble strategy indicated by the black line in Figure 13 supports the superior performance already described by the performance metrics above. The ensemble strategy dominates the individual models over nearly the whole testing period. Moreover, it can be observed that the strategy is more stable over the whole out-of-sample horizon and especially in times of recession. Specifically, in the most recent years since 2017, the ensemble strategy has been able to outperform the individual model strategies.

In summary, we have seen that using an ensemble of machine learning models and combining their return forecasts can significantly improve the predictive power and profitability of the strategy. In our case, the ensemble strategy outperforms individual models in terms of predictive R^2, Sharpe ratio, and cumulative log returns, with only slightly higher downside risk and turnover.

Factor Analysis

Having examined the performance of machine learning-based investment strategies, we finally want to learn more about what drives the superior performance and whether our strategy is superior to other long-short strategies. To shed light on the sources of risk and return and to find out whether the different long-short portfolios can still achieve a positive alpha when controlling for other common risk factors, we perform a factor analysis. For this, we regress the excess returns of each portfolio, including the ensemble strategy, against the returns of the Fama and French (2015) five factors – MKT (market excess returns), SMB (small minus big), HML (high book-to-market ratio minus low book-to-market ratio), RMW (robust minus weak), CMA (conservative minus aggressive) – and the Carhart (1997) momentum factor MOM.

Table 7 reports the results of the factor analysis for value-weighted portfolios in Panel A and equal-weighted portfolios in Panel B. For the statistical inference, we use Newey and West (1987) standard errors to account for potential heteroscedasticity and autocorrelation in the residuals. A Durbin-Watson test statistic of close to 2 for all strategies confirms that there is no significant autocorrelation in the residuals of the factor regressions (Woolridge, 2020). It can be seen that the machine learning portfolios earn positive and significant alphas in the range of 5.5% up to 10.4% for value-weighted portfolios. The only alpha which is not significant is the one obtained from the PLS model.

Table 7: Factor Analysis of Machine Learning Portfolios

	OLS-3	Elastic Net	PCR	PLS	RF	XGBoost	NN4	Ensemble
Panel A: Value-Weighted Portfolios								
Alpha (%)	6.46	6.24	5.45	3.88	10.40	6.21	9.58	7.60
	(3.96)	(2.13)	(2.37)	(1.57)	(2.97)	(2.58)	(3.11)	(3.06)
MKT	−0.02	0.26	0.14	0.24	0.12	0.32	0.12	0.29
	−(0.45)	(2.43)	(2.19)	(3.88)	(1.31)	(3.25)	(1.64)	(4.48)
SMB	0.83	0.54	0.42	0.46	0.61	0.02	0.73	0.61
	(11.74)	(4.11)	(3.65)	(4.90)	(3.73)	(0.21)	(4.55)	(5.17)
HML	0.31	0.16	0.01	0.04	0.23	0.22	0.26	0.22
	(4.55)	(0.85)	(0.12)	(0.33)	(1.32)	(1.91)	(1.83)	(1.82)
RMW	−0.32	−0.06	−0.24	−0.20	−0.06	−0.02	0.21	−0.17
	−(4.47)	−(0.29)	−(1.23)	−(1.20)	−(0.30)	−(0.14)	(1.24)	−(0.83)
CMA	0.24	0.30	0.56	0.53	0.20	−0.05	0.06	0.59
	(2.10)	(1.40)	(3.05)	(2.93)	(1.10)	−(0.23)	(0.42)	(3.70)
MOM	0.74	0.36	0.51	0.48	−0.14	0.42	−0.09	0.51
	(19.37)	(4.45)	(7.82)	(5.52)	−(1.49)	(6.07)	−(1.15)	(6.31)
R^2_{adj} (%)	73.53	21.58	32.53	34.57	19.45	17.60	24.48	38.96
DW	1.87	2.09	1.84	1.88	1.78	1.99	1.73	1.93
Panel B: Equal-Weighted Portfolios								
Alpha (%)	11.15	20.20	20.46	20.43	32.75	26.74	28.35	33.55
	(6.54)	(4.69)	(7.57)	(4.65)	(6.91)	(7.07)	(6.02)	(8.46)
MKT	−0.28	0.27	0.05	0.14	−0.04	0.27	−0.03	0.11
	−(9.75)	(2.44)	(0.94)	(1.81)	−(0.41)	(2.58)	−(0.30)	(1.64)
SMB	0.14	0.23	0.22	0.22	0.35	0.37	0.36	0.34
	(2.35)	(1.71)	(2.01)	(1.77)	(2.15)	(2.60)	(2.17)	(2.64)
HML	0.36	0.13	0.17	0.16	−0.13	0.03	−0.06	0.05
	(5.39)	(0.87)	(1.48)	(1.10)	−(0.76)	(0.18)	−(0.35)	(0.33)
RMW	−0.09	−0.02	−0.25	−0.22	−0.34	−0.32	−0.07	−0.34
	−(1.07)	−(0.09)	−(1.40)	−(1.69)	−(2.15)	−(2.15)	−(0.40)	−(2.21)
CMA	0.38	0.48	0.37	0.44	0.31	0.04	0.25	0.57
	(3.41)	(2.42)	(1.75)	(2.45)	(1.37)	(0.13)	(1.20)	(2.66)
MOM	0.83	0.49	0.54	0.56	−0.25	0.24	−0.27	0.39
	(23.04)	(2.92)	(9.52)	(3.21)	−(1.71)	(2.49)	−(1.77)	(3.74)
R^2_{adj} (%)	75.87	17.65	26.79	25.26	10.88	16.19	8.77	18.56
DW	1.58	1.89	1.75	1.71	1.75	1.96	1.79	1.71

Notes: This table reports the risk-adjusted performance of the machine learning-based long-short strategies as well as the ensemble strategy over the 35-year out-of-sample testing period from 1987 to 2021. The coefficients of the OLS regressions of the portfolio excess returns against the Fama and French (2015) five-factor plus Carhart (1997) momentum factor returns are reported for both value-weighted and equal-weighted portfolios. The coefficients for *alpha* are annualized. Newey and West (1987) standard errors are applied to account for potential autocorrelation and heteroscedasticity. The corresponding t-values are reported in brackets. (Source: based on own analysis).

Another observation that can be made from Table 7 is that the six-factor model can explain 73.5% of the variation of the OLS-3 portfolio returns while its ability in explaining the variation in the returns of the machine learning portfolios is much lower. For instance, the regression with the NN4 portfolio returns has an adjusted R^2 of 24.5%. As a result, the neural network (NN4) has a higher alpha in both weighting schemes. This is in line with what Gu et al. (2020) find in their analysis.

Regarding the factor loadings of the different strategies, a clear agreement between the individual strategies cannot be observed. However, most strategies have a positive loading on the MKT, SMB, HML, and CMA factors while the loadings on RMW are generally negative. It is unsurprising that for almost all machine learning strategies we can observe highly significant loadings on the momentum factor as the momentum-related variables turned out to be one of the most informative variables when we compare the variable importance in terms of SHAP values in Section 5.2. For the strategy that is based on predictions from the NN4 model this is, however, less profound. A potential reason for this is the more democratic nature of the NN4 model by considering information from a broader set of features as the analysis of relative variable importance in Section 5.2 showed. When looking at the ensemble strategy that uses average predictions of all individual models to form the portfolios, it can be observed that the most important factors seem to be the MKT factor together with the SMB and MOM factors, as these have highly significant coefficients.

The qualitative results for the equal-weighted portfolios in Panel B are similar. In general, we can observe higher and more significant annualized alphas between 11.2% and 33.6%. In this case, the range for the adjusted R^2 of the factor regressions is even larger than for value-weighted portfolios. The six-factor model can now explain only 8.8% of the variation in the returns of the NN4-based strategy. The results for the factor regressions without micro-cap stocks in the sample are reported in Table 11 in Appendix C. Excluding micro-cap stocks from the investment universe leads to a decrease in the levels of the alpha coefficients as well as small changes in the loadings of the factors. In addition, the six-factor model seems to be better at explaining the variation in the strategy returns when removing micro-cap stocks from the sample. The adjusted R^2s are now between 20.9% and 84.7%. However, the machine learning-based investment strategies are still able to generate positive and significant alphas.

While the alpha relative to the six-factor model is a widely used measure for testing investment strategies, it is not sufficient to demonstrate superior performance for real-time

investors relative to other long-short strategies. For instance, Cederburg et al. (2020) show that while volatility-managed portfolios exhibit positive alphas in spanning regressions, they do not necessarily have superior out-of-sample Sharpe ratios compared to unmanaged portfolios. Therefore, in addition to the factor regressions, we compare the Sharpe ratios and cumulative log returns of the machine learning portfolios to long-short strategies that invest based on common risk factors. Table 8 reports the Sharpe ratios for long-short strategies based on the SMB, HML, RMW, CMA, and MOM factors compared to our best-performing model (NN4) and the ensemble strategy. It can be seen that none of the long-short strategies can beat our strategies. The best factor strategy is one that buys robust stocks and sells weak stocks (RMW), resulting in an annualized Sharpe ratio of 0.50.

Table 8: Annualized Sharpe Ratios of the Best-performing Machine Learning Models and Common Factor Long-Short Strategies

	SMB	HML	RMW	CMA	MOM	NN4	Ensemble
SR	0.11	0.13	0.50	0.38	0.35	0.73	0.81

Notes: This table reports the annualized Sharpe ratios of the long-short investment strategies based on the SMB, HML, RMW, CMA, and MOM factors, the NN4 model, and the ensemble strategy. All portfolios are value-weighted. (Source: based on own analysis, factor data from the Kenneth French Data Library (n.d.)).

The cumulative log returns of the corresponding strategies are shown in Figure 14. Consistent with the results in Table 8, the cumulative log returns of the machine learning portfolios are considerably higher than those based on common factor long-short strategies.

Summarizing this section, the factor analysis supports the previously found evidence for the profitability of the investment strategies based on machine learning models. However, a clear picture of why the machine learning-based strategies perform better cannot be drawn as the six-factor model has not much ability in explaining the machine learning strategies. Instead, the machine learning models might be able to capture information from other factors or find dynamic relationships between the factors and the returns that cannot be adequately represented by a linear risk model (Rasekhschaffe & Jones, 2019). Consistent with the results of the factor regressions, we further show that our long-short strategies outperform long-short strategies based on common risk factors in direct comparisons. This once again supports the usefulness and practicability of our results.

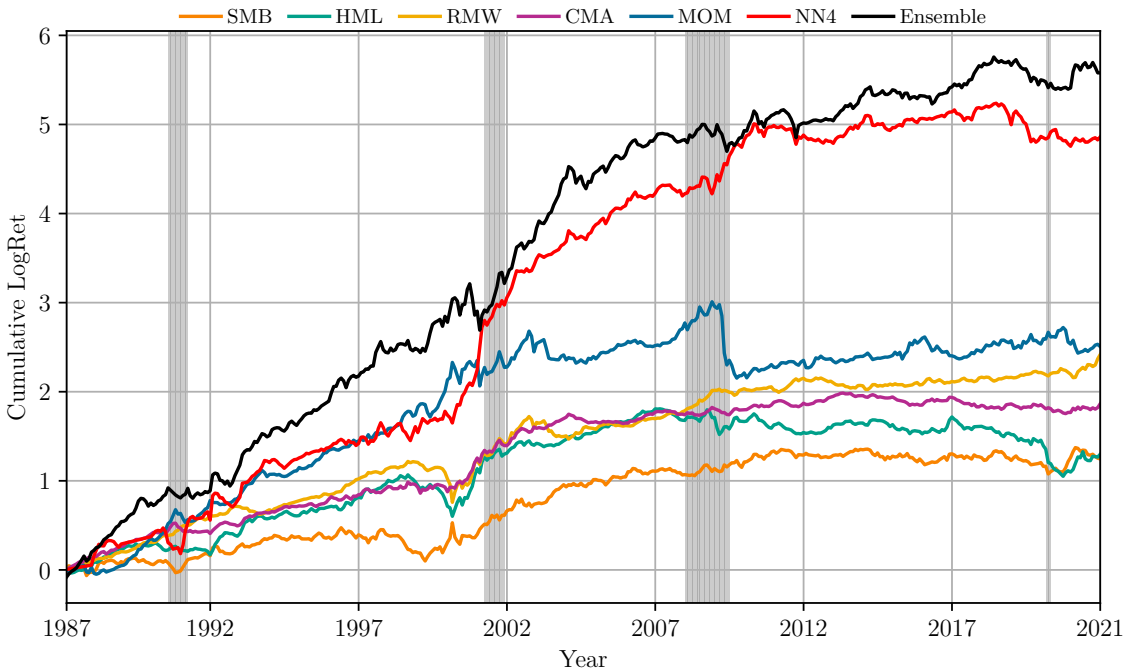


Figure 14: Cumulative Log Returns of Machine Learning Portfolios in Comparison with Long-Short Strategies Based on Common Factor Portfolios

This figure shows the performance in terms of cumulative log returns of the ensemble strategy, the NN4-based investment strategy, and long-short portfolios based on common risk factors from the six-factor model. All portfolios are value-weighted. The grey-shaded areas mark NBER recession periods in the U.S. (Source: data from own analysis, factor data from Kenneth French Data Library (n.d.), recession indicators from FRED (n.d.),).

6. Discussion

In this section, we provide a critical evaluation of the empirical results presented in Section 5. Specifically, we discuss the use of machine learning methods in finance and the factors that may have contributed to the differences between our results and those of Gu et al. (2020). Next, we address the practical implications of our findings and outline the limitations of our study. Finally, we propose areas for further research, given the limited scope of this thesis.

6.1. Critical Discussion of Findings

Machine Learning for Stock Return Prediction

The empirical results presented in the previous Section 5 showed that the use of machine learning in the prediction of stock returns can be very promising. With the ability of the models to incorporate a larger set of predictor variables and the ability to capture nonlinear relationships between the stock returns and the input variables, machine learning does have several advantages that can lead to successful applications in portfolio management (Israel et al., 2020; Bartram et al., 2021). However, when applying machine learning in finance, one must be aware of the risks and challenges when working with financial data (Israel et al., 2020; Bartram et al., 2021). For the larger and more complex machine learning models to excel, a relatively large amount of data is needed. Also, data quality is important as a lot of noise in the data can lead to unreliable models (Bartram et al., 2021). In that sense, finance is different from other areas where machine learning previously has performed well, such as image recognition or autonomous driving. Both areas are characterized by being big data environments and having high signal-to-noise ratios (Israel et al., 2020). A high signal-to-noise ratio means that the data contains high predictability. Due to the high randomness in financial data, the signal-to-noise ratio of stock returns is weak. In addition, finance is a rather small data environment. The amount of potential input variables might be large, but the number of observations of the stock returns is limited, especially for large (annual) frequencies of the return data. Furthermore, more data cannot be easily generated like in applications such as autonomous driving. If more training data is needed, more data can be collected by driving the cars more miles (Israel et al., 2020). In contrast, to get more return data, we have to wait for years and still have a rather small dataset. In our analysis, we use monthly data of a large set of stocks and thus limit the small data problem to some extent compared to other studies, but this might still not be enough (Israel et al., 2020).

The reason why the out-of-sample R^2s are relatively low, both in our analysis and in the financial literature, is directly linked to the low predictability of excess returns (low signal-to-noise ratios). When looking at the magnitude of our reported R^2s in the range from 0.15% to 0.26%, they seem to be very low and unimpressive. Returns are difficult to predict due to profit optimization and competition between traders. Traders will immediately exploit information that can reliably predict changes in stock prices. Hence, some predictability will disappear from the market and the only thing that moves

stock markets is unanticipated news which are random shocks to the market (i.e. noise) (Israel et al., 2020). This is the fundamental concept of the efficient market hypothesis introduced in Section 2.1. However, due to behavioral biases, transaction costs, and regulatory restrictions, some information might not be fully exploited by traders which leaves some predictability on the table. This predictability is usually small and difficult to capture (Israel et al., 2020). In addition, markets are dynamic and evolve over time resulting in lower predictability as well. Hence, with low-signal-to-noise ratios and evolving markets, it is common to have lower values for the predictive R^2 (Campbell & Thompson, 2008; Israel et al., 2020). Although the predictability of stock returns is low, it is important to recognize that even a small degree of predictability can lead to substantial economic gains (Kandel & Stambaugh, 1996; Xu, 2004; Campbell & Thompson, 2008; Rapach & Zhou, 2013).

Having established the reasons why there is low predictability and thus low predictive R^2s, we will compare our findings to those of the related literature in the following.

Comparison to Related Literature

While our study confirms the predictive power of machine learning models for U.S. stock returns, some of the results differ from those of previous studies. As explained in Section 5, the differences are mainly in the magnitude of the out-of-sample R^2s and the Sharpe ratios obtained in the investment strategies. Recall that our best model achieves an out-of-sample R^2 of 0.26%, while Gu et al. (2020) report a maximum out-of-sample R^2 of 0.40% for their best model. Differences are particularly observed for tree-based models and neural networks, while the multiple linear model and dimension-reduction methods perform very similarly. In the following, we critically analyze three important factors that may have influenced these results. We then address the question of whether the results of studies that use machine learning to predict stock returns can be replicated.

The difference in sample horizons is the first reason for the deviation in results between our study and Gu et al. (2020). Specifically, our out-of-sample period is five years longer than theirs. In general, a larger data sample can have a positive impact on the performance of tree-based methods and neural networks. With more data points, these models can learn more complex patterns and relationships, leading to more accurate predictions (James et al., 2021). It should be noted, however, that five additional years is very long compared to the 30-year out-of-sample period studied by Gu et al. (2020). This is especially true

since we have return data that is different from other datasets studied in machine learning applications as explained earlier. Even for large, diversified portfolios, there are significant fluctuations in returns due to unexpected news that do not recur over time but occur once (Israel et al., 2020). Consequently, a different out-of-sample period is an important reason for different results.

Second, we also use a different set of predictors than Gu et al. (2020). As explained in Section 3.1, we use four macroeconomic factors to form interaction terms, while Gu et al. (2020) use eight. As a result, they have 920 predictors, while we have 479. The decision to include four factors instead of eight is due to computational constraints and also the fact that the information from these additional four factors is already included in the firm characteristics. However, if important interaction terms are excluded from the model, it may miss important patterns and relationships, resulting in poorer performance. Thus, it could be that tree-based methods and neural networks perform better with a larger number of predictors. This is also supported by the fact that Leippold et al. (2022) obtain considerably higher predictive out-of-sample R^2s and Sharpe ratios when using a predictor set of more than 1,000 predictors. However, it must be remembered that they predict returns for the Chinese stock market instead of the U.S. stock market.

The third important factor for the different results is the dependence of tree-based models and neural networks on a random component. For instance, in random forests, the random seed controls the selection of features to be considered as splitting variables when growing the tree. Similarly, in XGBoost, the random seed is used to control the random subsampling of the training data and the random selection of the features. In neural networks, the random seed determines the initialization of the weights and provides a starting point for the optimization algorithm to find the weights that minimize the loss function. Since our dataset is small compared to Big Data problems, the influence of the random seed is not negligible. We have tried to minimize this influence using an ensemble approach. However, it is almost impossible to completely remove the influence of the random seed, especially since the available data is limited (Israel et al., 2020). This means that the models might not be trained sufficiently to produce stable out-of-sample results.

With this in mind, we now address the issue of replicability and robustness of the results of stock return prediction studies using machine learning techniques. Overall, our analysis has shown that reproducibility of results is challenging. Specifically, we showed that even slight variations in the methodology can lead to different results, which significantly complicates the replication process. In addition, the results of machine learning-based stock

return forecasts depend on a random component, such as feature selection or initialization of weights, which leads to variations in the results. Given the limited number of studies available and the divergence of our results from those of Gu et al. (2020), it is therefore difficult to say whether the magnitude of the results is robust. However, the qualitative results, e.g. that neural networks outperform the multiple linear model, are shown in many studies (Gu et al., 2020; Drobetz & Otto, 2021; Leippold et al., 2022). Therefore, we assume that the qualitative implications of our analysis are robust in different contexts.

6.2. Implications of Findings

The promising performance of machine learning-based investment strategies has important implications for many stakeholders. In the following, we mainly focus on professional investors, such as portfolio managers or asset managers. This is because the application of machine learning methods is not practical for retail investors due to the knowledge and computing resources needed for the implementation of the models. In contrast, for professional investors, machine learning is a promising tool that showed that it can deliver substantial improvements to portfolio performances. The increase in performance is probably mainly related to the more complex nature of the models that are able to capture nonlinearities. Furthermore, larger datasets including unstructured data can be used. However, due to the challenges mentioned in Section 6.1, it is much more difficult to leverage these advantages in finance. Also, even though we expect the results to be robust, we must point out one important limitation. As our study relies exclusively on the method of backtesting, we can only report information about past performance. Backtesting does not take into account unforeseen events and changes in future market conditions. Therefore, nothing can be said about the future performance of the trading strategies. Having said this, we describe three major implications for investors that we have identified in the remainder of this section.

First, the complex nature of machine learning models, especially neural networks, has direct implications for their use in portfolio management and asset management (Arnott et al., 2019). Neural networks are often seen as black boxes that are difficult to interpret. Hence, it is difficult to understand what the underlying drivers of the predictions of a model are. This can be a major concern for professional investors as they need to be able to explain to their investors how their models work and make predictions in order to gain their trust (Bartram et al., 2021). Furthermore, it is important to understand whether a

model is able to capture meaningful patterns in the data or whether it just captures noise. Asset managers have a preference for models that provide higher predictability compared to those with lower predictability. However, due to their responsibility to understand and communicate the risks associated with their clients' portfolios, they tend to favor models that are more understandable and easier to interpret (Israel et al., 2020; Bartram et al., 2021). To address the lack of interpretability and transparency of machine learning models, there have been major advancements in the last years in the field of *Explainable AI*. With the use of SHAP values, we apply one of these newer developments in our analysis and contribute to the literature by showing that SHAP values can indeed improve the interpretability of the models. Theoretically, it is possible to get an explanation for the prediction of every single stock in each month due to the local nature of SHAP values (Lundberg & Lee, 2017). Thus, it can be explained which variables lead to a specific return prediction.

Second, transaction costs also play a major role in the practical implementation of our strategy. Our empirical results reveal that machine learning models tend to have a higher turnover rate. It is important to note that our performance results do not account for transaction costs, and hence, some of the performance gains will be offset by higher trading costs. Higher transaction costs for machine learning-based strategies can have two possible reasons. First, we have shown that variables related to price trends are one of the most important ones for making predictions. This can lead to higher turnover due to more frequent rebalancing because of changing price trends (Bartram et al., 2021). Second, Israel et al. (2020) argue that the more complex a model, the higher the chance that it finds predictability in areas of the data that are more expensive or difficult to trade. Thus, it remains uncertain whether the strategies, which might involve buying lots of small firms, can be implemented on a large scale. A potential way to optimize trading costs is to apply machine learning, also called trading costs models, to optimize the portfolio implementation (Israel et al., 2020; Bartram et al., 2021). In addition, this application does not suffer as much from the small data and low signal-to-noise ratio problem. For instance, Jensen et al. (2022) propose a framework that integrates trading-cost-aware portfolio optimization with machine learning. Their model can form optimal portfolios while taking into account trading costs which leads to superior performance net of transaction costs compared to traditional portfolio optimization methods (Jensen et al., 2022).

Finally, another important implication of our findings is the promising performance of ensemble models that combine forecasts of different individual models. Together with

several other authors, we show that leveraging information about forecasts from several models leads to a higher and more stable portfolio performance (Borghi & De Rossi, 2020; Gu et al., 2020; Israel et al., 2020; Bartram et al., 2021). Hence, investors should use machine learning models as additional tools to the current models that are in place in order to utilize the advantages of machine learning methods described earlier.

Recapitulating the main points of this section, it can be concluded that investment strategies that are based on machine learning can be effective in predicting stock returns. However, the gains an investor achieves over traditional models are not revolutionary. In general, the developments in the application of machine learning in finance should be rather seen as an evolution and an important step forward in quantitative investing (Israel et al., 2020).

6.3. Further Research

Although this thesis has provided valuable insights into predicting returns using machine learning, there are several opportunities for further research that could improve our understanding. In this section, we will discuss some potential directions that could build on the results of our study.

One possible area for future research is to make the results more robust by modifying the ensemble approach. In our analysis, we have already formed ensembles by averaging the predictions of different random seeds and also averaging the predictions of the different models. However, several other ensemble approaches could improve the robustness of the models. For example, Rasekhschaffe and Jones (2019) propose to form ensembles by combining predictions based on different training windows and different predictor sets. Thereby, the variance of the forecasts can be further reduced which can lead to higher risk-adjusted returns (Rasekhschaffe & Jones, 2019).

Next, it would be interesting to vary the length of the forecast. Cochrane (2005) shows that as the length of the forecast horizon increases, an increasing fraction of the variation in stock returns becomes predictable. For example, when looking five years into the future, up to 60% of the variation in aggregate stock returns can be explained based on the dividend-price ratio, compared with 15% for a one-year horizon (Cochrane, 2005). This indicates that over a longer period of time, certain factors become more important in determining stock returns, making them more predictable. However, even though a longer time horizon typically has a better signal-to-noise ratio, there is a trade-off to consider.

Long-term forecasts also mean that the number of observations is small. This means that the machines do not have many observations on which they can be trained, leading to higher standard errors and greater variance in prediction errors. A key challenge for machine learning researchers in finance is therefore to determine the optimal point at which to balance signal strength with the number of observations to maximize profitability (Israel et al., 2020).

Including transaction costs in the analysis could further increase the realism of the study and provide a more accurate measure of the profitability of the proposed method. Drobetz and Otto (2021) follow the transaction cost approach proposed by Bollerslev et al. (2018) and Koijen et al. (2018), which determines portfolio transaction costs by multiplying the one-sided turnover and the half-spread. In this context, the one-sided turnover refers to the proportion of the portfolio that is sold or bought in a given period, while the half-spread is a measure of the average bid-ask spread for the stocks traded in the portfolio. Drobetz and Otto (2021) conclude that their neural network-based trading strategy performs best compared to other methods, even after accounting for transaction costs. It would therefore be interesting to investigate whether this is also true for our study.

In addition, two further, somewhat more distant research directions could be explored. First, it would be interesting to explore the generalizability of our method to other asset classes. While we focus on equities, our method could also be applied to other asset classes such as bonds, commodities, or real estate. However, this would require using a different predictor set that has direct influence on the specific asset class.

On the other hand, instead of predicting returns, machine learning models could also be used to predict fundamentals. Alberg and Lipton (2017) suggest that it is more effective to predict a company's fundamentals, such as earnings or sales than returns. This is because the signal-to-noise ratio is greater when predicting fundamentals, which better justifies the use of sophisticated machine learning algorithms. The predicted fundamentals can then be used to create stock portfolios for different trading strategies.

7. Conclusion

Machine learning is a powerful tool for analyzing complex datasets and has great potential in asset pricing. With a notable sample of more than 32,000 U.S. stocks and over 400 predictor variables, we study the performance of machine learning techniques in predicting stock returns between 1957 and 2021. We perform a comparative analysis of different machine learning models that include the multiple linear model, elastic net, dimension reduction via PCR and PLS, tree-based methods such as random forests and XGBoost, and neural networks. The three main objectives of this thesis are (1) to determine the model that performs best in predicting stock returns out-of-sample, (2) to identify the underlying firm characteristics and macroeconomic factors that drive the performance of the models, and (3) to test whether long-short investment strategies that are based on machine learning outperform a linear benchmark model and whether common risk factors can explain their performance.

Three main findings can be summarized from the analysis in our thesis. First, the analysis shows that neural networks are the best-performing models in terms of predictability. A neural network with four hidden layers achieves a monthly out-of-sample R^2 of 0.26% and outperforms all other models in our comparative analysis. Also, a pairwise comparison using the Diebold-Mariano test reveals that neural networks provide significantly more accurate predictions than the linear benchmark model. This superior performance can most likely be attributed to their ability to capture complex and nonlinear relationships in the data.

Second, we find that all machine learning methods roughly agree on a set of predictors that have the largest influence on the performance of the models. We identify three major groups of dominant predictor variables with the most influential variables being those related to price trends such as 1- and 12-month momentum. The next most powerful characteristics are measures of liquidity such as stock turnover and illiquidity followed by predictors that are related to investment and growth variables.

Third, we find that the use of machine learning methods can lead to substantial economic benefits for investors. In an out-of-sample backtest from 1987 to 2021 we compare the performance of long-short decile spread portfolios based on the next months' predictions of each method. In our specific case studied, the neural network-based strategy outperforms other models in terms of predictive R^2, Sharpe ratio, and alpha over the Fama and French (2015) and Carhart (1997) six-factor model. More specifically, the value-weighted

long-short strategy earns an annualized Sharpe ratio of 0.73 when using predictions from a neural network, compared to a Sharpe ratio of 0.64 from the linear benchmark model. An ensemble strategy that combines the predictions of all methods achieves an annualized Sharpe ratio of 0.81 and can thus provide an even stronger improvement in terms of Sharpe ratio compared to strategies that are based on individual model forecasts. In addition, the ensemble strategy yields a monthly predictive out-of-sample R^2 of 0.44% which is almost twice as large as the R^2 of the best individual model. This further supports our finding that investors can benefit from combining forecasts from various models. Finally, machine learning portfolios also beat long-short strategies based on common risk factors. Compared to a conventional momentum strategy, our best-performing model delivers a Sharpe ratio that is more than twice as high.

While the analysis shows promising results for the application of machine learning methods in asset pricing, there are also risks and challenges associated with using more sophisticated and complex models. First and foremost, higher predictive accuracy comes at the price of lower interpretability, which has direct implications for practical applications in portfolio management. Asset managers might be reluctant to use machine learning models if they are unable to understand precisely how their models work, even if they exhibit superior portfolio performance. Hence, by applying SHAP values, a recent advancement in the field of *Explainable AI*, we attempt to make machine learning models more interpretable. However, further research in this field is required to draw more meaningful conclusions. In addition, our study shows that investment strategies that are based on machine learning models have higher turnover rates. As a result, their performance will decrease when considering the strategies net of transaction costs.

In summary, this study suggests that the use of machine learning in finance is an evolutionary rather than a revolutionary change that has great potential to impact the field of asset pricing. However, further research is necessary to gain a complete understanding of the opportunities and challenges of applying machine learning to stock return prediction and to create more robust models.

Appendix

A. Detailed Description and Descriptive Statistics of Firm Characteristics

Table 9: Description and Descriptive Statistics of Firm Characteristics

No.	Acronym	Firm Characteristic	Frequency	Mean	Std
1	absacc	Absolute accruals	Annual	0.08	0.07
2	acc	Working capital accruals	Annual	0.00	0.08
3	aeavol	Abnormal earnings announcement volume	Quarterly	0.59	1.48
4	age	# years since first Compustat coverage	Annual	9.88	6.93
5	agr	Asset growth	Annual	-0.12	0.28
6	baspread	Bid-ask spread	Monthly	0.05	0.05
7	beta	Beta	Monthly	1.01	0.57
8	betasq	Beta squared	Monthly	1.37	1.46
9	bm	Book-to-market	Annual	1.15	2.79
10	bm_ia	Industry-adjusted book to market	Annual	-1.26	4.59
11	cash	Cash holdings	Quarterly	0.12	0.15
12	cashdebt	Cash flow to debt	Annual	0.05	3.05
13	cashpr	Cash productivity	Annual	-0.60	37.09
14	cfp	Cash flow to price ratio	Annual	0.05	0.69
15	cfp_ia	Industry-adjusted cash flow to price ratio	Annual	-0.08	2.56
16	chatoia	Industry-adjusted change in asset turnover	Annual	0.00	0.16
17	chcsho	Change in shares outstanding	Annual	0.08	0.24
18	chempia	Industry-adjusted change in employees	Annual	-0.09	0.48
19	chinv	Change in inventory	Annual	0.01	0.04
20	chmom	Change in 6-month momentum	Monthly	0.00	0.41
21	chpmia	Industry-adjusted change in profit margin	Annual	0.06	3.39
22	chtx	Change in tax expense	Quarterly	0.00	0.01
23	cinvest	Corporate investment	Quarterly	0.00	1.33
24	convind	Convertible debt indicator	Annual	0.10	0.29
25	currat	Current ratio	Annual	3.06	4.23
26	depr	Depreciation / PP&E	Annual	0.21	0.25
27	divi	Dividend initiation	Annual	0.02	0.13
28	divo	Dividend omission	Annual	0.02	0.13
29	dolvol	Dollar trading volume	Monthly	10.55	2.22
30	dy	Dividend to price	Annual	0.02	0.03
31	ear	Earnings announcement return	Quarterly	0.00	0.06
32	egr	Growth in common shareholder equity	Annual	0.11	0.45
33	ep	Earnings to price	Annual	0.01	0.21

Table 9: Description and Descriptive Statistics of Firm Characteristics (Continued)

No.	Acronym	Firm Characteristic	Frequency	Mean	Std
34	gma	Gross profitability	Annual	0.35	0.25
35	grCAPX	Growth in capital expenditures	Annual	0.55	2.13
36	grltnoa	Growth in long term net operating assets	Annual	0.07	0.10
37	herf	Industry sales concentration	Annual	0.13	0.08
38	hire	Employee growth rate	Annual	0.06	0.23
39	idiovol	Idiosyncratic return volatility	Monthly	0.06	0.03
40	ill	Illiquidity	Monthly	0.00	0.00
41	indmom	Industry momentum	Monthly	0.13	0.13
42	invest	Capital expenditures and inventory	Annual	0.07	0.13
43	lev	Leverage	Annual	1.69	3.23
44	lgr	Growth in long-term debt	Annual	0.18	0.53
45	maxret	Maximum daily return	Monthly	0.06	0.06
46	mom12m	12-month momentum	Monthly	0.12	0.43
47	mom1m	1-month momentum	Monthly	0.01	0.12
48	mom36m	36-month momentum	Monthly	0.27	0.64
49	mom6m	6-month momentum	Monthly	0.05	0.27
50	ms	Financial statement score	Quarterly	3.56	1.30
51	mvel1	Size	Monthly	1,546,091.13	7,837,979.46
52	mve_ia	Industry-adjusted size	Annual	-366.39	4,858.58
53	nincr	Number of earnings increases	Quarterly	0.92	1.06
54	operprof	Operating profitability	Annual	0.18	0.32
55	orgcap	Organizational capital	Annual	0.02	0.01
56	pchcapx_ia	Industry adjusted % change in capital expenditures	Annual	4.52	27.83
57	pchcurrat	% change in current ratio	Annual	0.04	0.41
58	pchdepr	% change in depreciation	Annual	0.06	0.34
59	pchgm_pchsale	% change in gross margin - % change in sales	Annual	-0.04	0.57
60	pchquick	% change in quick ratio	Annual	0.05	0.47
61	pchsale_pchinvt	% change in sales - % change in inventory	Annual	-0.02	0.46
62	pchsale_pchrect	% change in sales - % change in A/R	Annual	-0.03	0.43
63	pchsale_pchxsga	% change in sales - % change in SG&A	Annual	0.01	0.21
64	pchsaleinv	% change sales-to-inventory	Annual	0.07	0.48
65	pctacc	Percent accruals	Annual	-0.22	4.10
66	pricedelay	Price delay	Monthly	0.12	0.75

Table 9: Description and Descriptive Statistics of Firm Characteristics (Continued)

No.	Acronym	Firm Characteristic	Frequency	Mean	Std
67	ps	Financial statements score	Annual	4.07	1.24
68	quick	Quick ratio	Annual	2.24	3.81
69	rd	R&D increase	Annual	0.08	0.25
70	rd_mve	R&D to market capitalization	Annual	0.04	0.05
71	rd_sale	R&D to sales	Annual	0.23	1.72
72	realestate	Real estate holdings	Annual	0.25	0.12
73	retvol	Return volatility	Monthly	0.03	0.02
74	roaq	Return on assets	Quarterly	0.00	0.04
75	roavol	Earnings volatility	Quarterly	0.02	0.03
76	roeq	Return on equity	Quarterly	0.01	0.10
77	roic	Return on invested capital	Annual	-0.03	0.60
78	rsup	Revenue surprise	Quarterly	0.02	0.14
79	salecash	Sales to cash	Annual	34.96	90.89
80	saleinv	Sales to inventory	Annual	17.33	42.91
81	salerec	Sales to receivables	Annual	10.06	23.83
82	secured	Secured debt	Annual	0.54	0.35
83	securedind	Secured debt indicator	Annual	0.33	0.31
84	sgr	Sales growth	Annual	0.15	0.38
85	sin	Sin stocks	Annual	0.01	0.08
86	sp	Sales to price	Annual	1.98	2.41
87	std_dolvol	Volatility of liquidity (dollar trading volume)	Monthly	0.81	0.32
88	std_turn	Volatility of liquidity (share turnover)	Monthly	3.71	7.16
89	stdacc	Accrual volatility	Quarterly	3.21	22.23
90	stdcf	Cash flow volatility	Quarterly	7.55	51.53
91	tang	Debt capacity/firm tangibility	Annual	0.54	0.11
92	tb	Tax income to book income	Annual	-0.13	1.11
93	turn	Share turnover	Monthly	0.90	1.28
94	zerotrade	Zero trading days	Monthly	1.22	2.67

Notes: This table reports the definitions of all firm characteristics used in our analysis based on Gu et al. (2020) and Green et al. (2013). The data for the firm characteristics is obtained from Dacheng Xiu Data Library (n.d.). Mean and standard deviation are calculated before the normalization of the characteristics.

B. Gauss-Markov Assumptions

Following Woolridge (2020), the five Gauss-Markov assumptions are:

Assumption MLR.1 (Linear in Parameters)

The model in the population is given as

$$y = \theta_0 + \theta_1 z_1 + \theta_2 z_2 + ... + \theta_k z_k + u, \tag{B.1}$$

where $\theta_0, \theta_1, ..., \theta_k$ are the unknown parameters of interest and u is an unobserved random error.

Assumption MLR.2 (Random Sampling)

We have a random sample of P observations, $(z_{i1}, z_{i2}, ..., z_{ik}, y_i) : i = 1, 2, ..., P$, following the population model in assumption MLR.1.

Assumption MLR.3 (No Perfect Collinearity)

In the sample and therefore in the population, none of the predictors is constant, and there are no exact linear relationships among the predictors.

Assumption MLR.4 (Zero Conditional Mean)

The error u has an expected value of zero given any values of the independent variables. Expressed in mathematical terms,

$$\mathrm{E}(u|z_1, z_2, ..., z_k) = 0. \tag{B.2}$$

Assumption MLR.5 (Homoskedasticity)

The error u has the same variance given any value of the predictors. Expressed in mathematical terms,

$$\mathrm{Var}(u|z_1, z_2, ..., z_k) = \sigma^2. \tag{B.3}$$

C. Robustness Checks

Table 10: Long-Short Portfolio Performance (Excluding Micro-cap Stocks)

	OLS-3	Elastic Net	PCR	PLS	RF	XGBoost	NN4	Ensemble
Panel A: Value-Weighted Portfolios								
Mean (%)	10.74	10.22	9.64	9.73	8.75	10.29	9.47	13.64
Std (%)	18.71	16.34	15.55	16.16	12.31	15.15	11.13	16.32
SR	0.57	0.63	0.62	0.60	0.71	0.68	0.85	0.84
Max DD (%)	70.42	47.32	48.67	56.82	41.45	36.07	13.46	43.43
Max 1M Loss (%)	27.16	26.18	22.96	19.86	20.76	17.40	10.20	18.40
Turnover (%)	59.91	155.33	113.48	119.07	110.70	135.28	99.53	131.89
Panel B: Equal-Weighted Portfolios								
Mean (%)	19.55	14.59	12.63	14.04	14.50	13.82	13.32	19.17
Std (%)	19.73	18.00	17.25	17.41	11.34	17.68	11.81	18.15
SR	0.99	0.81	0.73	0.81	1.28	0.78	1.13	1.06
Max DD (%)	75.25	69.56	68.13	63.43	22.05	39.15	48.30	51.37
Max 1M Loss (%)	37.77	31.71	24.06	37.40	16.36	16.08	17.26	34.55
Turnover (%)	54.47	146.29	106.47	111.05	111.87	149.55	131.81	122.25

Notes: This table reports the performance of the machine learning-based long-short strategies over the 35-year out-of-sample testing period from 1987 to 2021. This analysis excludes micro-cap stocks (stocks below the 20th percentile on NYSE cap weights). The predictions of each model for the next month are sorted into deciles and we go long the stocks with the highest excess returns and short the stocks with the lowest excess returns. We report the annualized average realized returns (Mean), annualized standard deviation (Std), annualized Sharpe ratio (SR), maximum drawdown (Max DD), the most extreme negative monthly return (Max 1M Loss), and the average monthly turnover for all value-weighted (Panel A) and equal-weighted (Panel B) portfolios. (Source: based on own analysis, data for NYSE breakpoints from the Kenneth French Data Library (n.d.)).

Table 11: Factor Analysis of Machine Learning Portfolios (Excluding Micro-cap Stocks)

	OLS-3	Elastic Net	PCR	PLS	RF	XGBoost	NN4	Ensemble
Panel A: Value-Weighted Portfolios								
Alpha (%)	3.92	5.02	5.15	3.51	4.70	5.56	4.18	6.49
	(2.50)	(2.03)	(2.66)	(1.61)	(2.00)	(2.33)	(2.44)	(3.14)
MKT	0.05	0.23	0.11	0.24	0.17	0.25	0.20	0.25
	(1.07)	(2.88)	(2.05)	(3.69)	(3.26)	(3.01)	(4.49)	(3.77)
SMB	0.57	0.38	0.32	0.39	0.14	0.02	0.29	0.43
	(11.88)	(3.62)	(3.48)	(4.21)	(1.46)	(0.18)	(3.28)	(4.52)
HML	0.45	0.16	0.00	−0.02	0.19	0.25	0.15	0.20
	(6.97)	(0.89)	−(0.05)	−(0.16)	(1.57)	(2.50)	(1.61)	(1.96)
RMW	−0.29	−0.05	−0.23	−0.11	0.15	0.02	0.33	−0.09
	−(4.16)	−(0.27)	−(1.31)	−(0.65)	(1.05)	(0.13)	(2.98)	−(0.53)
CMA	0.23	0.29	0.52	0.57	0.20	−0.10	0.16	0.46
	(2.85)	(1.28)	(3.33)	(3.08)	(1.26)	−(0.51)	(1.19)	(2.69)
MOM	1.03	0.36	0.50	0.48	0.17	0.42	0.21	0.61
	(23.12)	(4.42)	(9.31)	(6.29)	(2.07)	(5.74)	(3.27)	(9.99)
R^2_{adj} (%)	83.20	20.12	34.99	32.78	10.50	17.75	21.33	43.85
DW	2.04	2.09	1.97	1.87	1.83	1.92	1.94	1.94
Panel B: Equal-Weighted Portfolios								
Alpha (%)	11.73	6.63	6.40	6.75	9.70	7.97	7.07	10.50
	(6.68)	(2.35)	(2.61)	(2.43)	(5.00)	(2.80)	(3.93)	(4.20)
MKT	−0.06	0.32	0.17	0.24	0.09	0.24	0.10	0.23
	−(1.59)	(3.61)	(3.07)	(3.80)	(1.82)	(2.39)	(2.50)	(3.67)
SMB	0.34	0.25	0.24	0.28	0.15	0.28	0.13	0.33
	(5.92)	(2.24)	(2.32)	(3.01)	(1.75)	(2.61)	(1.41)	(3.51)
HML	0.45	0.17	0.20	0.16	0.12	0.26	0.17	0.27
	(7.95)	(1.53)	(1.73)	(1.47)	(1.12)	(2.20)	(2.23)	(2.54)
RMW	0.05	0.13	−0.07	−0.04	0.32	0.10	0.68	0.08
	(0.59)	(0.76)	−(0.43)	−(0.31)	(3.17)	(0.64)	(5.20)	(0.58)
CMA	0.34	0.31	0.32	0.40	0.21	−0.11	0.17	0.36
	(3.75)	(1.89)	(1.82)	(2.56)	(1.62)	−(0.49)	(1.66)	(2.21)
MOM	1.11	0.56	0.65	0.67	0.31	0.51	0.27	0.81
	(22.67)	(4.66)	(8.47)	(6.21)	(5.92)	(5.38)	(3.83)	(9.46)
R^2_{adj} (%)	84.65	26.00	35.34	38.38	24.96	20.85	39.46	49.35
DW	1.62	2.20	1.93	1.95	1.80	1.91	1.76	1.96

Notes: This table reports the factor analysis of the machine learning-based long-short strategies as well as the ensemble strategy against the Fama and French (2015) and Carhart (1997) six-factor model. This analysis excludes micro-cap stocks (stocks below the 20th percentile on NYSE cap weights). The coefficients of the OLS regressions of the portfolio excess returns against the six-factor model are reported for both value-weighted and equal-weighted portfolios. The coefficients for *alpha* are annualized. Newey and West (1987) standard errors are applied to account for potential autocorrelation and heteroscedasticity. The corresponding t-values are reported in brackets. (Source: based on own analysis, data for NYSE breakpoints from the Kenneth French Data Library (n.d.)).

References

Alberg, J., & Lipton, Z. C. (2017). Improving Factor-Based Quantitative Investing by Forecasting Company Fundamentals. *Working Paper, Cornell University.*

Amit Goyal Data Library (n.d.). *Amit Goyal.* Retrieved February 13, 2023, from https://sites.google.com/view/agoyal145.

Arnott, R., Harvey, C. R., & Markowitz, H. (2019). A Backtesting Protocol in the Era of Machine Learning. *The Journal of Financial Data Science*, *1*(1), 64–74.

Baesens, B. (2014). *Analytics in a Big Data World: The Essential Guide to Data Science and its Applications* (2nd ed.). John Wiley & Sons.

Bartram, S. M., Branke, J., De Rossi, G., & Motahari, M. (2021). Machine Learning for Active Portfolio Management. *Journal of Financial Data Science*, *3*(3), 9–30.

Beckers, S., & Blair, B. (2002). Non–Parametric Forecasting for Conditional Asset Allocation. *Journal of Asset Management*, *3*(3), 213–228.

Black, F., Jensen, M. C., & Scholes, M. (1972). The Capital Asset Pricing Model: Some Empirical Tests. *Studies in the Theory of Capital Markets.*

Bollerslev, T., Chou, R. Y., & Kroner, K. F. (1992). ARCH Modeling in Finance - A Review of the Theory and Empirical Evidence. *Journal of Econometrics*, *52*, 5–59.

Bollerslev, T., Engle, R. F., & Nelson, D. B. (1994). ARCH Models. In R. Engle & D. McFadden (Eds.), *Handbook of econometrics* (pp. 2961–2984). Elsevier Science B.V.

Bollerslev, T., Hood, B., Huss, J., & Pedersen, L. H. (2018). Risk Everywhere: Modeling and Managing Volatility. *Review of Financial Studies*, *31*(7), 2730–2773.

Borghi, R., & De Rossi, G. (2020). The Artificial Intelligence Approach to Picking Stocks. In E. Jurczenko (Ed.), *Machine learning and asset management.* Wiley.

Boudoukh, J., Michaely, R., Richardson, M., & Roberts, M. R. (2007). On The Importance of Measuring Payout Yield: Implications for Empirical Asset Pricing. *Journal of Finance*, *62*(2), 877–915.

Box, G., & Jenkins, G. M. (1970). *Time Series Analysis: Forecasting and Control.* Holden-Day.

Breiman, L., Friedman, J., Stone, C. J., & Olshen, R. A. (1984). *Classification and Regression Trees* (1st ed.). Routledge.

Breiman, L. (2001). Random Forests. *Machine Learning*, *45*, 5–32.

Brochu, E., Cora, V. M., & De Freitas, N. (2010). A Tutorial on Bayesian Optimization of Expensive Cost Functions, with Application to Active User Modeling and Hierarchical Reinforcement Learning.

Butaru, F., Chen, Q., Clark, B., Das, S., Lo, A. W., & Siddique, A. (2016). Risk and Risk Management in The Credit Card Industry. *Journal of Banking and Finance*, *72*, 218–239.

Campbell, J. Y. (1987). Stock Returns and The Term Structure. *Journal of Financial Economics*, *18*, 373–399.

Campbell, J. Y. (2000). Asset Pricing at the Millennium. *Journal of Finance*, *55*(4), 1515–1567.

Campbell, J. Y., & Cochrane, J. H. (1999). By Force of Habit: A Consumption-based Explanation of Aggregate Stock Market Behavior. *Journal of Political Economy*, *107*(2), 205–251.

Campbell, J. Y., & Shiller, R. J. (1988a). Stock Prices, Earnings, and Expected Dividends. *The Journal of Finance*, *43*(3), 661–676.

Campbell, J. Y., & Shiller, R. J. (1988b). The Dividend-Price Ratio and Expectations of Future Dividends and Discount Factors. *The Review of Financial Studies*, *1*(3), 195–228.

Campbell, J. Y., & Thompson, S. B. (2008). Predicting Excess Stock Returns Out of Sample: Can Anything Beat the Historical Average? *Review of Financial Studies*, *21*(4), 1509–1531.

Campbell, J. Y., & Vuolteenaho, T. (2004). Inflation Illusion and Stock Prices. *The American Economic Review*, *94*(2), 19–23.

Carhart, M. M. (1997). On Persistence in Mutual Fund Performance. *Journal of Finance*, *52*(1), 57–82.

Cederburg, S., O'Doherty, M. S., Wang, F., & Yan, X. ((2020). On The Performance of Volatility-Managed Portfolios. *Journal of Financial Economics*, *138*(1), 95–117.

Chatfield, C. (2000). *Time-series Forecasting*. Chapman & Hall/CRC.

Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794.

Cochrane, J. H. (2005). *Asset Pricing: Revised Edition*. Princeton University Press.

Dacheng Xiu Data Library (n.d.). *Dacheng Xiu*. Retrieved February 13, 2023, from https://dachxiu.chicagobooth.edu.

De Gooijer, J. G., & Hyndman, R. J. (2006). 25 Years of Time Series Forecasting. *International Journal of Forecasting*, *22*(3), 443–473.

Deisenroth, M. P., Faisal, A. A., & Ong, C. S. (2020). *Mathematics for Machine Learning*. Cambridge University Press.

Dey, A. (2016). Machine Learning Algorithms: A Review. *International Journal of Computer Science and Information Technologies*, *7*(3), 1174–1179.

Diebold, F. X. (2015). Comparing Predictive Accuracy, Twenty Years Later: A Personal Perspective on the Use and Abuse of Diebold–Mariano Tests. *Journal of Business and Economic Statistics*, *33*(1), 1–9.

Diebold, F. X., & Mariano, R. S. (1995). Comparing Predictive Accuracy. *Journal of Business and Economic Statistics*, *13*(3), 134–144.

Dong, H., Guo, X., Reichgelt, H., & Hu, R. (2020). Predictive Power of ARIMA Models in Forecasting Equity Returns: A Sliding Window Method. *Journal of Asset Management*, *21*(6), 549–566.

Drobetz, W., & Otto, T. (2021). Empirical Asset Pricing via Machine Learning: Evidence from the European Stock Market. *Journal of Asset Management*, *22*(7), 507–538.

Eldan, R., & Shamir, O. (2016). The Power of Depth for Feedforward Neural Networks. In V. Feldman, A. Rakhlin, & O. Shamir (Eds.), *29th annual conference on learning theory* (pp. 907–940). PMLR.

Engle, R. F. (1982). Autoregressive Conditional Heteroscedasticity with Estimates of the Variance of United Kingdom Inflation. *Econometrica*, *50*(4), 987–1007.

Estrella, A., & Mishkin, F. S. (1996). The Yield Curve as a Predictor of U.S. Recessions. *Current Issues in Economics and Finance*, *2*(7).

Fama, E. F. (1970). Efficient Capital Markets: A Review of Theory and Empirical Work. *The Journal of Finance*, *25*(2), 383–417.

Fama, E. F. (1981). Stock Returns, Real Activity, Inflation, and Money. *The American Economic Review*, *71*(4), 545–565.

Fama, E. F. (1991). Efficient Capital Markets: II. *The Journal of Finance*, *46*(5), 1575–1617.

Fama, E. F., & French, K. R. (1988). Dividend Yields and Expected Stock Returns. *Journal of Financial Economics*, *22*(1), 3–25.

Fama, E. F., & French, K. R. (1993). Common Risk Factors in the Returns on Stocks and Bonds. *Journal of Financial Economics*, *33*, 3–56.

Fama, E. F., & French, K. R. (2015). A Five-Factor Asset Pricing Model. *Journal of Financial Economics*, *116*(1), 1–22.

Feng, G., He, J., & Polson, N. G. (2018). Deep Learning for Predicting Asset Returns.

FRED (n.d.). *NBER based Recession Indicators for the United States from the Period following the Peak through the Trough [USAREC]*. Retrieved March 8, 2023, from https://fred.stlouisfed.org/series/USREC.

Freyberger, J., Neuhierl, A., & Weber, M. (2020). Dissecting Characteristics Nonparametrically. *Review of Financial Studies*, *33*(5), 2326–2377.

Friedman, J. H. (2001). Greedy Function Approximation: A Gradient Boosting Machine. *The Annals of Statistics*, *29*(5), 1189–1232.

Gardner, E. S. (2006). Exponential Smoothing: The State of The Art -Part II. *International Journal of Forecasting*, *22*(4), 637–666.

Géron, A. (2019). *Hands-on Machine Learning with Scikit-Learn, Keras, and TensorFlow* (2nd ed.). O'Reilly Media, Inc.

Green, J., Hand, J. R., & Zhang, X. F. (2013). The Supraview of Return Predictive Signals. *Review of Accounting Studies*, *18*(3), 692–730.

Gu, S. (2021). *Machine Learning in Empirical Asset Pricing* [Doctoral dissertation, The University of Chicago].

Gu, S., Kelly, B., & Xiu, D. (2020). Empirical Asset Pricing via Machine Learning. *Review of Financial Studies*, *33*(5), 2223–2273.

Hastie, T., Tibshirani, R., & Friedman, J. H. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction* (2nd ed.). Springer.

Hodrick, R. J. (1992). Dividend Yields and Expected Stock Returns: Alternative Procedures for Inference and Measurement. *The Review of Financial Studies*, *5*(3), 357–386.

Hutchinson, J. M., Lo, A. W., & Poggio, T. (1994). A Nonparametric Approach to Pricing and Hedging Derivative Securities Via Learning Networks. *The Journal of Finance*, *49*(3), 851–889.

Ioffe, S., & Szegedy, C. (2015). Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. *International Conference on Machine Learning*, *37*, 448–456.

Israel, R., Kelly, B. T., & Moskowitz, T. J. (2020). Can Machines 'Learn' Finance? *Journal of Investment Management*, *18*(2), 23–36.

James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An Introduction to Statistical Learning: With Applications in R* (2nd ed.). Springer.

Jensen, T. I., Kelly, B. T., Malamud, S., & Pedersen, L. H. (2022). Machine Learning and the Implementable Efficient Frontier. *Swiss Finance Institute Research Paper*, *22-63*.

Kandel, S., & Stambaugh, R. F. (1996). On the Predictability of Stock Returns: An Asset-Allocation Perspective. *The Journal of Finance*, *51*(2), 385–424.

Keim, D. B., & Stambaugh, R. F. (1986). Predicting Returns in The Stock and Bond Markets. *Journal of Financial Economics*, *17*(2), 357–390.

Kelly, B. T., Pruitt, S., & Su, Y. (2019). Characteristics are Covariances: A Unified Model of Risk and Return. *Journal of Financial Economics*, *134*(3), 501–524.

Kenneth French Data Library (n.d.). *Kenneth R. French*. Retrieved March 8, 2023, from https://mba.tuck.dartmouth.edu/pages/faculty/ken.french/data_library.html.

Khandani, A. E., Kim, A. J., & Lo, A. W. (2010). Consumer Credit-risk Models via Machine-learning Algorithms. *Journal of Banking and Finance*, *34*(11), 2767–2787.

Kingma, D. P., & Ba, J. (2014). Adam: A Method for Stochastic Optimization. *International Conference on Learning Presentations*.

Koijen, R. S., Moskowitz, T. J., Pedersen, L. H., & Vrugt, E. B. (2018). Carry. *Journal of Financial Economics*, *127*(2), 197–225.

Kothari, S. P., & Shanken, J. (1997). Book-to-Market, Dividend Yield, and Expected Market Returns: A Time-series Analysis. *Journal of Financial Economics*, *41*, 16–203.

Kozak, S., Nagel, S., & Santosh, S. (2020). Shrinking the Cross-section. *Journal of Financial Economics*, *135*(2), 271–292.

Krasser, M. (2018). *Bayesian Optimization*. Retrieved April 8, 2023, from http://krasserm.github.io/2018/03/21/bayesian-optimization/.

Lamont, O. (1998). Earnings and Expected Returns. *The Journal of Finance*, *53*(5), 1563–1587.

Leippold, M., Wang, Q., & Zhou, W. (2022). Machine Learning in the Chinese Stock Market. *Journal of Financial Economics*, *145*(2), 64–82.

Leung, M. T., Daouk, H., & Chen, A.-S. (2000). Forecasting Stock Indices: A Comparison of Classification and Level Estimation Models. *International Journal of Forecasting*, *16*, 173–190.

Lewellen, J. (1999). The Time-series Relations among Expected Return, Risk, and Book-to-Market. *Journal of Financial Economics*, *54*, 5–43.

Lintner, J. (1965). The Valuation of Risk Assets and the Selection of Risky Investments in Stock Portfolios and Capital Budgets. *The Review of Economics and Statistics*, *47*(1), 13–37.

Lundberg, S. M., Erion, G., Chen, H., DeGrave, A., Prutkin, J. M., Nair, B., Katz, R., Himmelfarb, J., Bansal, N., & Lee, S. I. (2020). From Local Explanations to Global Understanding with Explainable AI for Trees. *Nature Machine Intelligence*, *2*(1), 56–67.

Lundberg, S. M., & Lee, S.-I. (2017). A Unified Approach to Interpreting Model Predictions. *Advances in Neural Information Processing Systems*, *30*.

Masters, T. (1993). *Practical Neural Network Recipies in C++*. Morgan Kaufmann.

Molnar, C. (2022). *Interpretable Machine Learning: A Guide for Making Black Box Models Explainable* (2nd ed.). https://christophm.github.io/interpretable-ml-book

Müller, A. C., & Guido, S. (2016). *Introduction to Machine Learning with Python: A Guide for Data Scientists*. O'Reilly Media, Inc.

Murphy, K. P. (2012). *Machine Learning: A Probabilistic Perspective*. MIT Press.

Nelson, C. R. (1976). Inflation and Rates of Return on Common Stocks. *The Journal of Finance*, *31*(2), 471–483.

Newey, W. K., & West, K. D. (1987). A Simple, Positive Semi-Definite, Heteroskedasticity and Autocorrelation Consistent Covariance Matrix. *Econometrica*, *55*(3).

Pai, P. F., & Lin, C. S. (2005). A Hybrid ARIMA and Support Vector Machines Model in Stock Price Forecasting. *Omega*, *33*(6), 497–505.

Pedersen, L. H. (2015). *Efficiently Inefficient: How Smart Money Invests and Market Prices Are Determined*. Princeton University Press.

Polk, C., Thompson, S., & Vuolteenaho, T. (2006). Cross-Sectional Forecasts of the Equity Premium. *Journal of Financial Economics*, *81*(1), 101–141.

Rapach, D., & Zhou, G. (2013). Forecasting Stock Returns. In *Handbook of economic forecasting* (pp. 328–383). Elsevier B.V.

Rapach, D. E., Strauss, J. K., & Zhou, G. (2010). Out-of-sample Equity Premium Prediction: Combination Forecasts and Links to the Real Economy. *Review of Financial Studies*, *23*(2), 821–862.

Rasekhschaffe, K. C., & Jones, R. C. (2019). Machine Learning for Stock Selection. *Financial Analysts Journal*, *75*(3), 70–88.

Scheuch, C., Voigt, S., & Weiss, P. (2023). *Tidy Finance with R* (1st ed.). Chapman & Hall/CRC. https://tidy-finance.org

Sharpe, W. F. (1964). Capital Asset Prices: A Theory of Market Equilibrium Under Conditions of Risk. *The Journal of Finance*, *19*(3), 425–442.

Tibshirani, R. (1996). Regression Shrinkage and Selection via the Lasso. *Journal of the Royal Statistical Society. Series B (Methodological)*, *58*(1), 267–288.

United States Department of Labor (n.d.). *Standard Industrial Classification (SIC) Manual*. Retrieved February 13, 2023, from https://www.osha.gov/data/sic-manual.

U.S. Department of the Treasury (n.d.). *Price, Yield and Rate Calculations for a Treasury Bill*. Retrieved February 13, 2023, from https://www.treasurydirect.gov/instit/annceresult/press/preanre/2004/ofcalc6decbill.pdf.

Wang, J.-H., & Leu, J.-Y. (1996). Stock Market Trend Prediction Using ARIMA-based Neural Networks. *Proceedings of International Conference on Neural Networks*.

Weigand, A. (2019). Machine Learning in Empirical Asset Pricing. *Financial Markets and Portfolio Management*, *33*(1), 93–104.

Welch, I., & Goyal, A. (2008). A Comprehensive Look at the Empirical Performance of Equity Premium Prediction. *Review of Financial Studies*, *21*(4), 1455–1508.

West, K. D. (2006). Forecast Evaluation. In G. Elliott, C. W. Granger, & A. Timmermann (Eds.), *Handbook of economic forecasting* (pp. 99–134). Elsevier B.V.

White, H. (1988). Economic Prediction Using Neural Networks: The Case of IBM Daily Stock Returns. *IEEE 1988 International Conference on Neural Networks*, 451–458.

Woolridge, J. M. (2020). *Introductory Econometrics: A Modern Approach* (7th ed.). Cengage Learning.

Xu, Y. (2004). Small Levels of Predictability and Large Economic Gains. *Journal of Empirical Finance*, *11*(2), 247–275.

Yao, J., Li, Y., & Tan, L. (2000). Option Price Forecasting Using Neural Networks. *Omega*, (28), 455–466.

Zhang, Z., Zohren, S., & Roberts, S. (2019). DeepLOB: Deep Convolutional Neural Networks for Limit Order Books. *IEEE Transactions on Signal Processing*, *67*(11), 3001–3012.

Zou, H., & Hastie, T. (2005). Regularization and Variable Selection via the Elastic Net. *Journal of the Royal Statistical Society. Series B (Statistical Methodology)*, *67*(2), 301–320.